Quick Start

Quickstart — Exact Commands (Turn-Key)

Preconditions (must be true before running anything)

- Repo has a Node runtime available (Replit default is fine).
- Your Roshi workspace root exists at:
  - `docs/roshi_v2/`
- Your config files exist at:
  - `roshi/domains.json`
  - `roshi/sources.json`
- Your template folder exists at:
  - `docs/roshi_v2/01_templates/`

If any of these are missing: **STOP** and run `roshi:init` first.

---

0) One-time install (first run only)

A) Add scripts to `package.json`

Add these scripts (exact names) so the agent can run the pipeline consistently:

```
npm pkg set scripts.roshi:init="node scripts/roshi-init.mjs"
npm pkg set scripts.roshi:gen="node scripts/roshi-generate.mjs"
npm pkg set scripts.roshi:seed="node scripts/roshi-seed.mjs"
npm pkg set scripts.roshi:draft="node scripts/roshi-draft.mjs"
npm pkg set scripts.roshi:review="node scripts/roshi-review.mjs"
npm pkg set scripts.roshi:lock="node scripts/roshi-lock.mjs"
npm pkg set scripts.roshi:verify="node scripts/roshi-verify.mjs"
```

Optional convenience script:

```
npm pkg set scripts.roshi:all="npm run roshi:gen && npm run roshi:seed && npm run roshi:draft"
```

Stop condition: If `package.json` cannot be edited or scripts can't be added, stop and fix repo permissions.

---

1) Initialize Roshi workspace (only if missing)

```
npm run roshi:init
```

Expected outputs:

- `docs/roshi_v2/00_product/`
- `docs/roshi_v2/00_registry/`
- `docs/roshi_v2/01_templates/`
- `docs/roshi_v2/02_domains/`
- `docs/roshi_v2/03_workflows/`
- `docs/roshi_v2/README.md`

Stop condition: If any of the above are not created, stop and print the script error.

---

2) Generate per-domain doc packs (shape only)

```
npm run roshi:gen
```

Expected outputs:
For every domain in `roshi/domains.json`, a folder is created under:

- `docs/roshi_v2/02_domains/<domain-slug>/`

Inside each folder, these files must exist:

- `DDES_<domain>.md`
- `UX_Foundations_<domain>.md`
- `UI_Constraints_<domain>.md`
- `BELS_<domain>.md`
- `DIM_<domain>.md`
- `SCREENMAP_<domain>.md`
- `TESTPLAN_<domain>.md`

Rules:

- `roshi:gen` must be **idempotent**:
    - never overwrites existing domain docs
    - only creates missing files

Stop condition: If any domain folder is missing any file above, stop and report missing files.

---

3) Seed starter scaffolding (safe prefill only)

`npm run roshi:seed`

What seed is allowed to do:

- Fill placeholders and starter tables **only if the file is empty/unstarted**
- Insert action prefix stubs
- Insert blank policy/state/scenario tables

What seed is NOT allowed to do:

- Invent real rules
- Overwrite authored content

Stop condition: If `seed` modifies a non-empty file without a “seed marker,” stop and revert.

---

4) Draft truth candidates (extractive-only)

`npm run roshi:draft`

Draft reads from sources defined in `roshi/sources.json`, typically:

- `docs/roshi_v2/00_product/RPBS_Product.md`
- `docs/roshi_v2/00_product/REBS_Product.md`
- `docs/roshi_v2/00_registry/domain-map.md`
- `docs/roshi_v2/00_registry/reason-codes.md`

- `docs/roshi_v2/00_registry/action-vocabulary.md`
- optional per-domain notes folder (if configured)

Draft outputs (per domain):

- Updates `BELS_<domain>.md` with **truth candidates**
- Creates or updates `OPEN_QUESTIONS_<domain>.md`

Hard constraints (must enforce):

- If not explicitly supported by sources → write `UNKNOWN` and add to Open Questions.
- Every policy rule / state transition must include `SourceRef`.

Stop condition: If draft creates rules without SourceRef, stop and treat output as invalid.

---

5) Review packet (optional but recommended)

`npm run roshi:review`

Expected output:

- Summary of:
    - count of `UNKNOWN` entries per domain
    - conflicts detected between sources
    - missing Reason Codes referenced by BELS
    - missing required sections/tables

Stop condition: If review finds conflicts or missing reason codes, do not lock.

---

6) Lock a domain (ERC creation)

Lock a single domain when (and only when):

- BELS has **no critical UNKNOWNs**
- policy rules have reason codes + messages
- state machines have deny codes
- minimum acceptance scenarios exist

Run:

```
npm run roshi:lock -- --domain <domain-slug> --version v1
```

Expected outputs:

- `docs/roshi_v2/02_domains/<domain-slug>/ERC_<domain>_v1.md`

Optional:

- `docs/roshi_v2/02_domains/<domain-slug>/LOCK_HASHES.json` (if enabled)

Stop condition: If the domain still contains `UNKNOWN` in locked sections, do not lock.

---

7) Verify the system (gate check)

```
npm run roshi:verify
```

Verify must fail the run if:

- required files are missing
- a locked domain references undefined Reason Codes
- a locked domain contains `UNKNOWN` in policy/state/scenarios
- required template sections are absent

Expected output:

- PASS / FAIL + a file-by-file actionable report

Domain-scoped runs (exact commands)

Use these when iterating one domain at a time:

```
npm run roshi:gen  -- --domain <domain-slug>
npm run roshi:seed -- --domain <domain-slug>
npm run roshi:draft -- --domain <domain-slug>
npm run roshi:lock -- --domain <domain-slug> --version v1
```

Golden “daily loop” (what you actually do)

```
npm run roshi:draft -- --domain <domain-slug>
# edit docs to resolve UNKNOWNs
npm run roshi:review -- --domain <domain-slug>
npm run roshi:lock  -- --domain <domain-slug> --version v1
npm run roshi:verify
```

Safety rules (must be enforced)

- Never regenerate or overwrite authored domain docs.

- Never “invent” rules. Missing rules become `UNKNOWN`.
- Only ERC-locked docs are “truth” for implementation.
- If verify fails, do not build.

Quickstart Appendix — Config Files

Quickstart Appendix — Config Files (Exact Formats)

File A) `roshi/domains.json` (domain registry for generation)

Purpose: single source for which domains exist, their slugs, and action prefixes.

Location (required):

- `roshi/domains.json`

Exact format (JSON):

```
{
  "roshi_root": "docs/roshi_v2",
  "domains_dir": "02_domains",
  "templates_dir": "01_templates",
  "domains": [
    {
      "name": "Identity & Access",
      "slug": "identity-access",
      "prefix": "identity",
      "type": "business"
    },
    {
      "name": "AccessControl",
      "slug": "access-control",
      "prefix": "access",
      "type": "crosscutting"
    }
  ]
}
```

Field rules (must validate):

- `roshi_root` must exist.
- `domains_dir` must exist under `roshi_root`.

- `templates_dir` must exist under `roshi_root`.
- Each domain must include:
  - `name` (string, human readable)
  - `slug` (kebab-case, unique)
  - `prefix` (lowercase, unique; used for action namespace)
  - `type` in: `business | controlplane | crosscutting`
- No duplicates allowed for `slug` or `prefix`.

Stop conditions:

- If any domain entry is missing required fields → FAIL.
- If `slug` or `prefix` duplicates → FAIL.
- If any `slug` folder already exists and script would overwrite → FAIL (must be no-overwrite).

File B) `roshi/sources.json` (authoritative input sources for draft)

Purpose: tells `roshi:draft` what files it is allowed to use as "truth sources."

Location (required):

- `roshi/sources.json`

Exact format (JSON):

```
{
  "roshi_root": "docs/roshi_v2",
  "sources": [
    "00_product/RPBS_Product.md",
    "00_product/REBS_Product.md",
```

```
    "00_registry/domain-map.md",
    "00_registry/reason-codes.md",
    "00_registry/action-vocabulary.md",
    "00_registry/glossary.md"
  ],
  "domain_notes": {
    "enabled": true,
    "dir": "notes/domains",
    "pattern": "{slug}.md"
  },
  "sourceref": {
    "required": true,
    "format": "HeadingPath",
    "example": "RPBS > Hard Rules Catalog > Permissions /
AccessControl"
  }
}
```

Rules:

- Every path in `sources[]` is relative to `docs/roshi_v2/`
- All listed files must exist or `draft` must FAIL.

Domain notes (optional but recommended):

- If enabled, the draft script may read:
  - `notes/domains/<domain-slug>.md`
- Domain notes are **allowed** to add detail, but they must follow the “no invention” rule:
  - If a note conflicts with RPBS/REBS/Domain Map → it is ignored and flagged as a conflict.

Stop conditions:

- Any missing file in `sources[]` → FAIL.

- Conflicts detected → draft must produce an `OPEN_QUESTIONS` entry and mark affected rules as `UNKNOWN`.

Appendix — Template Filename Contract (required)

These template files must exist under:

- `docs/roshi_v2/01_templates/`

Exact required template filenames:

- `DDES.template.md`
- `UX_Foundations.template.md`
- `UI_Constraints.template.md`
- `BELS.template.md`
- `DIM.template.md`
- `SCREENMAP.template.md`
- `TESTPLAN.template.md`
- `ERC.template.md`

Stop condition: If any are missing → FAIL.

Appendix — Generated Domain Doc Filename Contract (required)

For each domain `slug`, `roshi:gen` must create (if missing) these files under:

- `docs/roshi_v2/02_domains/<slug>/`

Required files:

- `DDES_<slug>.md`
- `UX_Foundations_<slug>.md`
- `UI_Constraints_<slug>.md`
- `BELS_<slug>.md`
- `DIM_<slug>.md`
- `SCREENMAP_<slug>.md`
- `TESTPLAN_<slug>.md`

Lock file (created by `roshi:lock`):

- `ERC_<slug>_v#.md`

Stop condition: If any required file is missing after gen → FAIL.

Appendix — “No Overwrite” File Safety Policy (must enforce)

All scripts must follow this policy:

`roshi:gen`

- Allowed: create missing files/folders
- Forbidden: overwrite existing domain docs

`roshi:seed`

- Allowed: fill only empty/unstarted docs OR docs with a `<!-- SEED_OK -->` marker
- Forbidden: modify authored content

`roshi:draft`

- Allowed: write “truth candidates” only into clearly marked sections (recommended marker `<!-- DRAFT_SECTION_START -->`)
- Forbidden: overwrite content in a section marked `LOCKED`

`roshi:lock`

- Allowed: create ERC file referencing inputs
- Forbidden: lock if required sections contain `UNKNOWN` in policy/state tables

---

Appendix — Output Report Contract (agent must print this)

Every script run must print:

- Files created
- Files modified
- Files skipped (because no-overwrite)
- Any FAIL reasons clearly

Example report header:

- `ROSHI_REPORT: created=X modified=Y skipped=Z failed=N`

Quickstart Appendix — CLI Argument Contract

Quickstart Appendix — CLI Argument Contract (Exact)

Global rules (apply to all Roshi scripts)

- All scripts must support:
  - `--domain <slug>` (optional; if omitted, operate on all domains)
  - `--dry-run` (optional; prints changes without writing)
- If `--domain` is provided and slug is not found in `roshi/domains.json` → **FAIL**
- If `--dry-run` is set → **no files may be modified**
- All scripts must print a `ROSHI_REPORT` summary

`roshi:init`

Command

```
npm run roshi:init
```

Args: none
Behavior

- Creates missing Roshi folder tree + baseline registry files + templates folder
- Must be idempotent

Fail if

- Cannot create `docs/roshi_v2/`

---

roshi:gen

Command

```
npm run roshi:gen
npm run roshi:gen -- --domain <slug>
npm run roshi:gen -- --domain <slug> --dry-run
```

Behavior

- Creates the per-domain doc pack files (shape only)
- Never overwrites existing files

Fail if

- Any required template file is missing
- Would overwrite an existing file

---

roshi:seed

Command

```
npm run roshi:seed
npm run roshi:seed -- --domain <slug>
npm run roshi:seed -- --domain <slug> --dry-run
```

Behavior

- Prefills only empty/unstarted docs or docs with `<!-- SEED_OK -->`
- Never invents real rules

Fail if

- Attempts to edit a non-empty doc without seed marker

---

roshi:draft

Command

```
npm run roshi:draft
npm run roshi:draft -- --domain <slug>
npm run roshi:draft -- --domain <slug> --dry-run
```

Behavior

- Extractive-only drafting from `roshi/sources.json`
- Writes candidates + creates/updates `OPEN_QUESTIONS_<slug>.md`
- Any unsupported rule becomes `UNKNOWN` + Open Questions

Fail if

- Any source file is missing
- A draft rule is written without `SourceRef`
- A draft attempts to modify a section marked `LOCKED`

---

roshi:review

Command

```
npm run roshi:review
npm run roshi:review -- --domain <slug>
```

Behavior

- Produces summary of:
  - UNKNOWN counts
  - conflicts
  - missing reason codes
  - missing required tables/sections

Fail if

- None (should report issues, not crash) unless files are unreadable

---

`roshi:lock`

Command

```
npm run roshi:lock -- --domain <slug> --version v1
npm run roshi:lock -- --domain <slug> --version v1 --dry-run
```

Required args

- `--domain <slug>`
- `--version v#` (format: `v1`, `v2`, etc.)

Behavior

- Creates `ERC_<slug>_<version>.md` in the domain folder
- Optionally writes hashes for locked inputs

Fail if

- Domain still contains `UNKNOWN` in:
    - Policy Rules table
    - State Machine transitions
    - Acceptance Scenarios required set
- Reason codes referenced by BELS are missing from registry
- Required input docs listed in ERC are missing

`roshi:verify`

Command

```
npm run roshi:verify
npm run roshi:verify -- --domain <slug>
```

Behavior

- Fails the run if:
    - required files missing
    - locked domains contain UNKNOWN in critical sections
    - undefined reason codes referenced
    - missing scenario minimums
    - missing required template sections (structure regression)

Exit codes

- `0` = pass
- `1` = fail

Standard output contract (must print)

All scripts must print:

- `ROSHI_REPORT`
    - `created: []`
    - `modified: []`
    - `skipped: []`
    - `warnings: []`
    - `errors: []`
    - `status: PASS|FAIL`

If FAIL: print actionable next steps.

One Prompt to Start a New App

TASK: Start a brand-new app using the Roshi v2 system, end-to-end, with zero drift.

GUARDRAILS (NON-NEGOTIABLE):
- No invention: if a rule/decision is not explicitly supported by provided sources, write UNKNOWN and add it to Open Questions.
- No overwrite: if docs already exist, do not overwrite authored content; only create missing files.
- Do not restructure the repo, rename routes, move screens, or change navigation contracts.
- Server is source of truth; client gating is UX only.
- Every deny must include a Reason Code + user-facing message guidance.
- Every policy/state/scenario row must include SourceRef.
- Output a ROSHI_REPORT listing created/modified/skipped/errors at the end.

PHASE 0 — DISCOVERY (READ ONLY)
1) Inspect the repository structure:
  - Identify backend folder (server/api) and frontend folder (client/app).
  - Identify existing auth, access control, errors, audit, events, and tests (if any).
2) Create or update: docs/roshi_v2/00_registry/repo-mapping.md
  - Fill all integration point rows (FOUND/MISSING).
  - If MISSING, propose minimal creation plan (adapters preferred).

PHASE 1 — ROSHI WORKSPACE INIT (DOC SYSTEM)
3) Initialize Roshi workspace if missing:
  - Create docs/roshi_v2 folder tree:
    00_product/, 00_registry/, 01_templates/, 02_domains/, 03_workflows/
  - Create baseline global docs if missing (do not overwrite if they exist):
    - 00_product/RPBS_Product.md
    - 00_product/REBS_Product.md
    - 00_product/PROJECT_OVERVIEW.md (as a template if empty)
    - 00_registry/domain-map.md
    - 00_registry/glossary.md
    - 00_registry/action-vocabulary.md
    - 00_registry/reason-codes.md
    - 03_workflows/Prompt_Playbook.md (optional)
  - Create docs/roshi_v2/01_templates/TEMPLATE_RULES.md and TEMPLATE_VERSION.md if missing.

PHASE 2 — TEMPLATE HARDENING (TEMPLATES ONLY)
4) Ensure templates exist and meet requirements (do not touch generated domain docs):
  Required templates under docs/roshi_v2/01_templates:
  - DDES.template.md
  - UX_Foundations.template.md
  - UI_Constraints.template.md
  - BELS.template.md
  - DIM.template.md

- SCREENMAP.template.md
- TESTPLAN.template.md
- ERC.template.md

Confirm these templates include:
- SourceRef requirements
- UNKNOWN + Open Questions policy
- Policy evaluation order
- State machine + transitions
- Error & recovery table
- Acceptance scenarios table
- Step-up section (for controlplane domains)

If missing, create/update templates ONLY.

PHASE 3 — DOMAIN REGISTRY (LIST + MINIMUMS)

5) Create/Update roshi/domains.json (do not invent domains; use a sensible universal default set if none provided, but mark as DRAFT).
- Include: name, slug, prefix, type (business/controlplane/crosscutting)
- Add minimums config:
  business: actions>=8, commands>=3, policyRules>=10, scenarios>=12, tests>=12
  controlplane: actions>=6, commands>=3, policyRules>=8, scenarios>=10, tests>=10, requiresStepUp=true, auditOnWrites=true
  crosscutting: tests>=8

6) Sync docs/roshi_v2/00_registry/domain-map.md to match domains.json.

PHASE 4 — INPUTS PACK TEMPLATE (ASK USER FOR CONTENT)

7) Populate docs/roshi_v2/00_product/PROJECT_OVERVIEW.md with the required headings (template only) IF it is empty.

8) STOP and print an “INPUTS REQUIRED” block telling me exactly what to fill in:
- App name + version
- App type/modes
- Core objects + lifecycles
- Roles summary
- Onboarding journey steps
- Navigation surfaces
- 20–50 Key Rules truth statements
- Trust & Safety baseline
- Ops controls baseline

(Do not proceed to draft until this is filled.)

PHASE 5 — GEN (SHAPE ONLY)

9) Once PROJECT_OVERVIEW is filled, generate per-domain doc packs (shape only):
- Create docs/roshi_v2/02_domains/<slug>/
- Create missing files only:
  DDES_<slug>.md

UX_Foundations_<slug>.md
UI_Constraints_<slug>.md
BELS_<slug>.md
DIM_<slug>.md
SCREENMAP_<slug>.md
TESTPLAN_<slug>.md

PHASE 6 — DRAFT (LOGIC CANDIDATES, EXTRACTIVE ONLY)
10) Run roshi:draft for the onboarding-critical domains first (do not build code yet):
- identity-access (account creation, login, onboarding)
- access-control (authorization gate)
- trust-safety (restrictions baseline)
- ops-control-plane (admin controls baseline)

Draft rules must be extractive-only and include SourceRef.
Missing info becomes UNKNOWN + Open Questions.

PHASE 7 — REVIEW PACKETS (GAP LIST)
11) Produce REVIEW_PACKET_<slug>.md for each drafted domain:
- UNKNOWN list + locations
- conflicts
- missing reason codes
- minimum counts vs required minimums
- next edits required to lock

OUTPUT REQUIREMENTS:
- Print ROSHI_REPORT with:
  - Files created
  - Files modified
  - Files skipped
  - Errors/warnings
- Print “INPUTS REQUIRED” section if PROJECT_OVERVIEW is not filled.
- Do NOT lock ERC.
- Do NOT implement app code in this run (docs-only bootstrap).

Inputs Pack next

Inputs Pack Spec (Turn-Key)

Purpose

The Inputs Pack is the **minimum structured truth bundle** required for `roshi:draft` to generate real domain logic candidates (BELS) **without invention**.

Rule: If information is missing, the system must output `UNKNOWN` + log it in Open Questions.

A) Required Input Files (minimum set)

These files must exist before `roshi:draft` is allowed to run:

1. `docs/roshi_v2/00_product/RPBS_Product.md`
2. `docs/roshi_v2/00_product/REBS_Product.md`
3. `docs/roshi_v2/00_registry/domain-map.md`
4. `docs/roshi_v2/00_registry/action-vocabulary.md`
5. `docs/roshi_v2/00_registry/reason-codes.md`
6. `docs/roshi_v2/00_registry/glossary.md`
7. `docs/roshi_v2/00_product/PROJECT_OVERVIEW.md` ✅ (the new one)

Stop condition: If any required file is missing → `roshi:draft` must FAIL.

B) `PROJECT_OVERVIEW.md` (single doc the agent needs per app)

B1) File location (required)

docs/roshi_v2/00_product/PROJECT_OVERVIEW.md

B2) Required structure (must follow headings)

```
# PROJECT OVERVIEW — {{APP_NAME}} (v{{VERSION}})

## 1) App Type & Modes
- App type(s): (social/community, marketplace, fintech, streaming,
internal tools, etc.)
- Platforms: (iOS/Android/Web)
- Auth model: (email/password, phone OTP, SSO, wallet, etc.)
- Realtime: YES/NO (chat, rooms, livestream, etc.)

## 2) Core Objects (What exists in the product)
List the canonical objects users interact with.
For each object: what it is, who can create it, and its lifecycle
state.
- Object:
  - Description:
  - Created by:
  - Owned by:
  - Visibility:
  - Lifecycle states: (draft/published/archived/etc.)
  - Notes:

## 3) Roles & Access Summary (human readable)
List each role and what they broadly can do.
- Role:
  - Can view:
  - Can create:
  - Can moderate/manage:
  - Restrictions that override role:

## 4) Onboarding Journey (minimum ship)
Define the onboarding funnel as concrete checkpoints.
- Entry point:
- Steps:
```

```
  - Step 1:
  - Step 2:
- “Onboarded” definition:
- When privileges unlock:
- Failure recovery paths:

## 5) Navigation Surfaces (high-level)
List the top-level sections/screens users can reach.
- Surface:
  - Purpose:
  - Primary actions:
  - Role gates:

## 6) Key Rules (Executive summary)
Write 20–50 short rules as truth statements (these map into BELS).
Format: `RULE: <truth>`
- RULE:
- RULE:

## 7) Trust & Safety Baseline
- Report flows:
- Block flows:
- Enforcement types: (mute/suspend/ban/etc.)
- Visibility rules when restricted:
- Moderator/admin capabilities:
- Audit requirements:

## 8) Notifications & Activity (baseline)
- What triggers notifications:
- What is logged to ActivityLog:
- Preferences users can change:

## 9) Operations Controls (baseline)
List the controls needed for production.
- Feature flags:
- Kill switches:
- Role management:
- Content moderation controls:
```

```
- Minimum analytics needed:

## 10) Payments / Marketplace (if applicable)
If not applicable, explicitly write: “NOT IN SCOPE”
- Purchase types:
- Payment events:
- Refund rules:
- Ledger requirements:

## 11) Compliance / Data Rights (baseline)
- Export: YES/NO scope
- Delete: YES/NO scope
- Retention: baseline
- Sensitive data categories:

## 12) Domain Notes Index (optional but recommended)
If you have domain notes files, list them here so it’s explicit:
- notes/domains/<slug>.md — purpose
```

B3) Content rules (strict)

- Must be written as **truth statements**, not aspirations.
- If something is undecided, label it as:
    - `UNKNOWN:` and add an open question.
- Avoid implementation details (no DB schema here).
- Must align with RPBS. If conflict exists, RPBS wins.

C) Optional “Domain Notes” files (for extra detail per domain)

C1) Location (if enabled)

`notes/domains/<domain-slug>.md`

C2) Required structure

```
# Domain Notes — <Domain Name> (<domain-slug>)

## 1) Owned Objects
- …

## 2) Actions (verbs users take)
- …

## 3) Permissions (role + conditions)
- …

## 4) State Machines
- …

## 5) Edge Cases / Abuse
- …

## 6) Open Questions
- …
```

Rules

- Domain notes may add details, but cannot override RPBS/REBS/Domain Map.
- Conflicts must be flagged as conflicts and produce `UNKNOWN` outputs.

D) How Inputs Pack feeds Roshi outputs (mapping table)

Input Source	Feeds	Output Docs
RPBS Hard Rules + Non-negotiables	permissions + safety constraints	BELS, TESTPLAN
PROJECT_OVERVIEW Core Objects + Journey	object lifecycles + actions + screens	DDES, SCREENMAP, BELS
Domain Map	ownership + boundaries + dependencies	DDES, DIM
Action Vocabulary	action namespace consistency	BELS
Reason Codes	deny consistency + UX messaging	BELS, UI Constraints
Domain Notes	domain-specific edges	BELS, DIM, TESTPLAN

E) Stop Conditions (must enforce)

`roshi:draft` must FAIL if:

- PROJECT_OVERVIEW.md is missing
- required headings are missing
- RPBS/REBS/Domain Map are missing
- draft outputs contain rules without SourceRef

Minimal promise

If you provide:

- RPBS (fortified)

- REBS (fortified)
- Domain Map + registries
- PROJECT_OVERVIEW (structured)

Then `roshi:draft` can reliably produce:

- action vocab candidates
- policy candidates
- state machine candidates
- acceptance scenarios candidates
  …and anything missing becomes explicit UNKNOWN + Open Questions (no silent gaps).

Integration Points Contract

Integration Points Contract (Turn-Key)

Purpose

This contract defines **where domain logic MUST live** in the codebase and **how it MUST be called**, so Roshi outputs (BELS/DIM/TESTPLAN/ERC) translate into production code without duplication, drift, or random folder creation.

Rule: If an agent cannot find the required integration point, it must **create it** in the specified location (minimal, no breaking changes).

A) Required Integration Surfaces (universal)

The system must expose these integration surfaces:

1. **AccessControl** (single authorization gate)
2. **Errors** (standard error envelope + reason codes)
3. **AuditLog** (tamper-resistant admin/user action logs)
4. **Events** (domain event publishing + consuming)
5. **Commands/Handlers** (write paths: command handlers)
6. **State Machines** (entity lifecycles + transitions)
7. **Read Models** (query functions for UI/API reads)
8. **Validation** (input validation + invariants)
9. **Rate Limiting & Abuse Controls**
10. **Observability** (logs/metrics/tracing hooks)

B) Standard Repo Locations (universal layout)

> If your repo differs, keep the spirit: one “core” layer + domain modules.
> The agent must map these to your structure and document the mapping in `domain-map.md`.

B1) Server (source of truth)

Preferred structure:

```
server/
  src/
    core/
      access/
        accessControl.ts
        actions.ts
      errors/
        errorCodes.ts
        errorEnvelope.ts
      audit/
        auditLog.ts
        auditTypes.ts
      events/
        eventBus.ts
        eventTypes.ts
      observability/
        logger.ts
        metrics.ts
    domains/
      <domain-slug>/
        index.ts
        contracts.ts
        handlers/
          commands.ts
        state/
          machine.ts
        reads/
```

```
        queries.ts
      validators/
        input.ts
        invariants.ts
      events/
        emits.ts
        consumes.ts
      tests/
        *.test.ts
```

B2) Client (UX gating only)

Preferred structure:

```
client/
  src/
    core/
      access/
        canClient.ts
      errors/
        presentError.ts
      telemetry/
        track.ts
    domains/
      <domain-slug>/
        screens/
        components/
        hooks/
        api/
```

Hard rule: Client checks may **hide/disable** but server must enforce.

C) AccessControl Contract (single gate)

C1) Required exported API (server)

`server/src/core/access/accessControl.ts`

```
can(actor, action, resource, ctx) -> { allow: boolean, reasonCode?: string, userMessage?: string, audit?: boolean }
```

C2) Required inputs

- **actor:** `{ userId, role, status, enforcementFlags, ... }`
- **action:** string from Action Vocabulary: `<domain>.<resource>.<verb>`
- **resource:** `{ type, id, ownerId?, visibility?, state?, ... }`
- **ctx:** `{ ip?, deviceId?, requestId?, correlationId?, now?, ... }`

C3) Enforcement rule

All write paths MUST call AccessControl **before any mutation**.

Forbidden:

- domain-specific “random if checks” deciding allow/deny without `can()`

C4) Mapping to Roshi docs

- BELS Policy Rules table = AccessControl rules
- Reason Codes = deny envelopes
- RuleIDs should be referenced in code comments or test names

---

D) Error Envelope Contract (reason-code consistent)

D1) Standard error envelope (server responses)

`server/src/core/errors/errorEnvelope.ts`

Required shape:

```
{
  "ok": false,
  "error": {
    "code": "REASON_CODE",
    "message": "User-facing message",
    "details": { "field": "..." },
    "correlationId": "..."
  }
}
```

D2) Reason codes source

- All `code` values must exist in:
  - `docs/roshi_v2/00_registry/reason-codes.md`
- Any missing code → FAIL `roshi:verify`

D3) UI rule

Client must present:

- reason + next step
- never show internal debug details

E) AuditLog Contract (mandatory for sensitive actions)

E1) Required exported API

`server/src/core/audit/auditLog.ts`

```
audit(eventType, actor, target, metadata, ctx) -> void
```

E2) Minimum metadata fields

- `eventType`
- `actor.userId`
- `target.type`, `target.id`
- `timestamp`
- `correlationId`
- `reasonCode` (if enforcement/deny/admin action)
- `policyRuleId` (if available)

E3) Mapping to Roshi docs

- REBS “Audit categories” defines which actions require audit.
- BELS “Audit?” column sets audit requirements per rule/action.

F) Events Contract (DIM-driven integration)

F1) Required exported API

`server/src/core/events/eventBus.ts`

```
publish(eventType, payload, meta) -> void
subscribe(eventType, handler) -> void
```

F2) Event schema rule

- Events must be declared in `eventTypes.ts`
- Payload must have:
  - `eventId`
  - `occurredAt`
  - `correlationId`
  - `producerDomain`
  - entity identifiers

F3) Mapping to Roshi docs

- DIM “Event Catalog (emits)” = events published
- DIM “Consumed Events” = subscriptions + effects

G) Command Handler Contract (write path discipline)

G1) What is a command handler?

A function that performs a domain mutation with this order:

1. Validate inputs

2. Load required resources
3. `AccessControl.can()`
4. Enforce invariants
5. Apply state machine transition (if relevant)
6. Write to DB (owned entities only)
7. Emit events
8. Audit if required
9. Return result envelope

G2) Required location

`server/src/domains/<slug>/handlers/commands.ts`

G3) Idempotency

If BELS says idempotent:

- command must accept an idempotency key
- must safely retry without duplicates

H) State Machine Contract (BELS-driven)

H1) Required location

`server/src/domains/<slug>/state/machine.ts`

H2) Required exported API

```
transition(entity, transitionName, actor, ctx) -> { ok, newState } | { ok:false, reasonCode }
```

H3) Mapping to Roshi docs

- BELS State Machines table defines:
    - states
    - transitions
    - guards
    - deny reason codes

I) Read Model Contract (queries are not business logic)

I1) Required location

`server/src/domains/<slug>/reads/queries.ts`

I2) Rule

- Reads may aggregate across domains, but must not write.
- Sensitive visibility rules must be applied consistently (often via AccessControl helpers).

J) Validation & Invariants Contract

J1) Input validation location

`server/src/domains/<slug>/validators/input.ts`

J2) Invariants location

`server/src/domains/<slug>/validators/invariants.ts`

Rule: Invariants must match DDES invariants and BELS acceptance scenarios.

K) Rate limiting & Abuse Controls

K1) Required location

`server/src/core/access/rateLimits.ts` (or `core/security/`)

- Domain-specific limit maps allowed, but must register here.

K2) Mapping to docs

- RPBS “Minimum Viable Safety” + DDES “Abuse considerations” + TESTPLAN “Abuse tests”

L) Observability Contract (production traceability)

L1) Required fields on logs

Every request/command must have:

- `correlationId`
- `actorId` (if authenticated)
- `action`
- `domain`
- `resourceId` (when applicable)

L2) Required location

`server/src/core/observability/logger.ts`

L3) Mapping to docs

- DDES “Observability requirements”
- REBS “Observability requirements”
- TESTPLAN “Observability verification”

M) Client Integration (UX gating + presentation)

M1) Client `canClient()` rule

Client can call:

- `canClient(action, resource)` to decide hide/disable
  But must assume server can still deny.

M2) Error presentation

Client must map Reason Codes to:

- message + next step pattern
- consistent UI component (banner/toast/dialog)

N) Drift Prevention Rules (hard)

N1) No duplicate enforcement

- If AccessControl exists, no other module may become a “second policy engine.”
- Domain checks must call AccessControl or be migrated.

N2) No cross-domain DB writes

- Writes only to owned entities.
- Cross-domain side effects must use events or orchestrators.

N3) ERC governs build

- If ERC exists for a domain, code must match it.
- If meaning changes, create a new ERC version.

O) Verification Requirements (tie into roshi:verify)

`roshi:verify` must fail if:

- write handlers do not call AccessControl
- reason codes referenced in code are not in registry
- locked domain docs contain UNKNOWN in critical tables
- events used in code are not declared in eventTypes.ts (if you adopt that pattern)
- audit-required actions have no audit emission

Mapping Contract for Existing Repos

Mapping Contract for Existing Repos (Turn-Key)

Purpose

Most real repos won't match the "ideal" folder layout. This contract forces the agent to:

- **map Roshi integration points to your existing structure**
- **avoid refactors**
- **avoid duplication**
- **document exactly where each integration surface lives**

Rule: If the repo already has an equivalent integration surface, the agent must **reuse it**. If it does not exist, the agent must **create it** in the least invasive way.

A) Mandatory Mapping Document (must be created/updated)

Location (required)

`docs/roshi_v2/00_registry/repo-mapping.md`

Required structure (must follow headings)

```
# Repo Mapping — {{APP_NAME}} (v{{VERSION}})

## 1) Repo Overview
- Frontend location:
- Backend location:
- Shared code location:
- Test location(s):
- CI entrypoint(s):

## 2) Canonical Integration Points (Mapped)
```

Fill every row. If missing, mark MISSING and include creation plan.

Integration Surface	Required API	Ideal Location	Actual Location	Status (FOUND/MISSING)	Notes
AccessControl	can(actor, action, resource, ctx)	server/src/core/access/			
Errors	error envelope + reason codes	server/src/core/errors/			
AuditLog	audit(eventType, actor, target, meta, ctx)	server/src/core/audit/			
Events	publish/subscribe	server/src/core/events/			
Command Handlers	domain write handlers	server/src/domains/<slug>/handlers/			
State Machines	transition()	server/src/domains/<slug>/state/			
Read Models	query funcs	server/src/domains/<slug>/reads/			
Validation	input + invariants	server/src/domains/<slug>/validators/			
Rate Limits	shared limiter	server/src/core/access/			
Observability	logger/metrics	server/src/core/observability/			
Client Gating	canClient()	client/src/core/access/			
UI Error Present	presentError()	client/src/core/errors/			

3) Domain Folder Mapping
Map the Roshi domains to existing modules.

Roshi Domain Slug	Existing Module/Folder	Notes (merge/split/keep)
identity-access		
access-control		
…		

4) What We Will NOT Do (Safety)

```
- No renames of existing routes
- No moving screens between stacks
- No folder reshuffles
- No “replace everything” rewrites

## 5) Creation Plan for Missing Surfaces (if any)
For each MISSING row above:
- What file(s) will be created:
- Minimal exported API:
- What existing code will call it first:
- How we prevent duplication:

## 6) Migration Plan for Duplicates (if any)
If enforcement exists in multiple places:
- List the duplicate locations:
- Shadow mode plan:
- Cutover criteria:
- Deletion criteria (only after tests pass):
```

Stop condition: If the mapping doc is incomplete (any row blank), the agent must stop and fill it before building logic.

B) Discovery Rules (how the agent finds “actual locations”)

B1) Search patterns (server)

Agent must search for these common patterns to locate existing integration points:

- `can(` / `authorize` / `permission` / `policy`
- `ReasonCode` / `error.code` / `FORBIDDEN`
- `audit` / `audit_log` / `admin_action`

- `publish(` / `emit` / `event` / `subscribe`
- `transition` / `state` / `status` / `lifecycle`
- `rateLimit` / `throttle`
- `logger` / `metrics` / `correlationId`

B2) Preference order (reuse > wrap > create)

When something exists:

1. **Reuse** existing module if it already behaves like the contract
2. **Wrap** it behind the Roshi contract API (adapter layer)
3. **Create** minimal new module only if neither exists

C) Adapter Layer Rule (prevents breaking changes)

If the repo already has a function but with a different signature, do NOT rewrite all callers.

Instead:

- create an adapter that conforms to Roshi's required API
- keep existing code intact

Example:

- Existing: `authorize(user, perm, obj)`
- New adapter: `accessControl.can(actor, action, resource, ctx)` calls `authorize()` internally

Stop condition: If the agent proposes “refactor every call site,” stop and require adapter approach.

D) Duplicate Enforcement Rule (how to prevent breakage)

If existing authorization checks are scattered:

- Do NOT delete them immediately.
- Do NOT try to unify everything in one pass.

D1) Shadow mode cutover (required)

For a period:

- compute decision using both:
  - old check
  - new `AccessControl.can()`
- log differences (with correlationId)
- enforce **old** decision until diff rate is near zero

D2) Cutover criteria

Cutover to new enforcement only when:

- diff rate is acceptably low
- denial reason codes match expected patterns
- tests cover allow/deny + edge cases

D3) Deletion criteria

Only remove old checks when:

- tests pass
- no shadow diffs for a defined period
- ERC for that domain is locked

E) Repo Safety Constraints (non-negotiable)

E1) No broad renames

- Do not rename folders, routes, or param keys.
- Do not “restructure the app.”

E2) No cross-domain DB writes

- If existing code violates this, document it in `repo-mapping.md` and propose a staged fix.

E3) One surface per concern

- One AccessControl
- One Error envelope
- One Audit sink
- One Event bus
  (Adapters allowed, duplicates not.)

F) Verification Hook (ties into `roshi:verify`)

`roshi:verify` must fail if:

- `repo-mapping.md` missing or incomplete
- A locked domain has no mapped handler location
- Reason codes are referenced in code but absent from registry
- Commands mutate without calling AccessControl (or explicitly documented legacy mode)

G) Minimal “first adoption” sequence (recommended)

When integrating Roshi into an existing repo, do this order:

1. Create `repo-mapping.md`
2. Implement AccessControl adapter (no behavior change)
3. Implement Error envelope adapter
4. Add Audit sink adapter
5. Pick 1 low-risk domain and implement in shadow mode
6. Lock ERC and cutover
7. Repeat domain-by-domain

Tab 34

Golden Example Domain Pack + Code Mapping

Golden Example Domain Pack + Code Mapping (Turn-Key)

This section gives the agent a **single "gold standard" reference** showing exactly how Roshi docs become real logic + tests. Use this as the pattern for every domain.

Golden Example Domain

Domain: Community Messaging (chat messages in a room/channel)
Slug: `community-messaging`
Prefix: `community`
Type: business domain

---

1) Golden Example — Domain Doc Pack (Filled)

1.1 `DDES_community-messaging.md` (excerpt)

```
# DDES — Community Messaging ({{APP_NAME}})

## 0) Snapshot
- Domain Slug: community-messaging
- Domain Prefix: community
- Owner Type: Business Domain
- Primary Roles: Visitor, Member, Moderator, Admin

## 1) Domain Purpose
Enable users to send and read messages inside a room/channel with
role-gated permissions, safety enforcement, and auditability for
moderation actions.

## 2) System of Record Ownership
### OWNS
- Entities: Message
```

- Policies: message create/edit/delete rules
- State machines: message lifecycle
- Logs: message moderation audit events (for mod/admin actions)

DOES NOT OWN
- Room membership truth (owned by Community Spaces / Membership domain)
- Enforcement truth (owned by TrustSafety restrictions)
- Notifications delivery (owned by Notifications)

3) Public Contracts
APIs
- POST /v1/community/rooms/:roomId/messages
- GET /v1/community/rooms/:roomId/messages
- DELETE /v1/community/messages/:messageId (moderation/owner)

Events Emitted
- community.message.created
- community.message.deleted
- community.message.flagged (optional)

Queries
- listMessages(roomId, cursor)
- getMessage(messageId)

4) Core Entities
Entity	Description	Primary IDs	PII?	Lifecycle?
Message	Chat message in a room	messageId	Possibly	YES

5) Aggregates & Invariants
- INV-001: A message must belong to exactly one roomId.
- INV-002: A muted/suspended user cannot create messages.
- INV-003: Deleting a message preserves an audit trail when performed by a moderator/admin.

9) Failure & Recovery
- Writes are idempotent via clientMessageId (optional).

```
- Duplicate send must not create duplicates.
- If event publish fails after DB write: log + retry async (outbox
recommended).

## 10) Abuse & Safety
- Rate limit message.create per user per room.
- Support reporting and moderation actions.
```

1.2 `BELS_community-messaging.md` (filled minimum)

```
# BELS — Community Messaging ({{APP_NAME}})
(See TEMPLATE_RULES.md — no invention, SourceRef required)

## 1) Action Namespace
- Prefix: community

## 2) Action Vocabulary
| Action | Actor(s) | Resource | Description | SourceRef | Status |
|---|---|---|---|---|---|
| community.message.create | authenticated | Message | Send a message
in room | PROJECT_OVERVIEW > Key Rules | DRAFT |
| community.message.read | authenticated | Message | Read messages in
room | PROJECT_OVERVIEW > Nav Surfaces | DRAFT |
| community.message.delete | owner/mod/admin | Message | Delete
message | RPBS > Hard Rules > TrustSafety | DRAFT |

## 3) Command Handlers
| Command | Actions served | Inputs | Validations | Writes | Emits
Events | Idempotency | SourceRef |
|---|---|---|---|---|---|---|---|
| SendMessage | community.message.create | roomId, text,
clientMessageId? | text length, room exists, membership visible |
Message | message.created | clientMessageId | PROJECT_OVERVIEW > Key
Rules |
| DeleteMessage | community.message.delete | messageId, reason? |
exists, permission | Message(state=deleted) | message.deleted | N/A |
RPBS > Minimum Safety |
```

4) Policy Rules

RuleID	Action	Allow When (conditions)	Deny ReasonCode	Deny User Message	Audit?	SourceRef
R-001	community.message.read	actor is authenticated AND room is visible to actor	ROOM_NOT_VISIBLE	“You can’t view this room.”	NO	PROJECT_OVERVIEW > Roles
R-002	community.message.create	actor authenticated AND room visible AND NOT muted/suspended AND room allows posting	USER_RESTRICTED	“You can’t send messages right now.”	NO	RPBS > Min Safety
R-003	community.message.delete	actor is message owner within edit window	NOT_OWNER	“You can only delete your own message.”	NO	PROJECT_OVERVIEW > Key Rules
R-004	community.message.delete	actor is Moderator/Admin	FORBIDDEN	“You don’t have permission.”	YES	RPBS > Min Safety

Policy Evaluation Order

1) Auth check
2) Account status check (locked/disabled)
3) TrustSafety restrictions (muted/suspended)
4) Room visibility check
5) Role + conditions allow
6) Default deny

5) State Machine — Message

State	Meaning	Visible?
active	normal message	YES
deleted	removed from feed	YES (placeholder) OR NO (hard delete)

Transition	From	To	Guard Conditions	Side Effects	Deny ReasonCode	SourceRef
delete_by_owner	active	deleted	actor=owner AND within window	emit message.deleted	NOT_OWNER	PROJECT_OVERVIEW > Key Rules

```
| delete_by_mod | active | deleted | actor is Mod/Admin | audit
moderation | FORBIDDEN | RPBS > Min Safety |

## 6) Error & Recovery
| Scenario | Error Code | User Message | Retry? | Recovery Path |
SourceRef |
|---|---|---|---|---|---|
| Duplicate send | DUPLICATE_REQUEST | “Already sent.” | NO | Show
delivered state | REBS > Error Model |
| Rate limited | RATE_LIMITED | “You’re sending too fast.” | YES |
Cooldown UI | RPBS > System Limits |

## 8) Acceptance Scenarios
| ID | Given | When | Then | Type | SourceRef |
|---|---|---|---|---|---|
| S-001 | Member in visible room | send message | message appears,
event emitted | Allow | PROJECT_OVERVIEW |
| S-002 | Muted user | send message | deny USER_RESTRICTED | Deny |
RPBS |
| S-003 | Visitor in public room (read-only) | send message | deny
FORBIDDEN or READ_ONLY | Deny | PROJECT_OVERVIEW |
| S-004 | Owner deletes own message in window | delete | message
marked deleted | Allow | PROJECT_OVERVIEW |
| S-005 | Non-owner tries delete | delete | deny NOT_OWNER | Deny |
PROJECT_OVERVIEW |
| S-006 | Moderator deletes any message | delete | success + audit
event created | Allow | RPBS |
```

1.3 `DIM_community-messaging.md` (filled minimum)

```
# DIM — Community Messaging ({{APP_NAME}})

## 1) Interaction Matrix
| Use Case | Primary Action | Owning Write Domain | Consult | Writes |
Emits | Side Effects |
|---|---|---|---|---|---|---|
```

```
| Send message | community.message.create | community-messaging |
identity-access, trust-safety, community-spaces | Message |
message.created | notifications, activitylog |

## 2) Event Catalog (emits)
| Event | Emitted When | Payload | Consumers |
|---|---|---|---|
| community.message.created | message stored | messageId, roomId,
actorId | notifications, activitylog |
| community.message.deleted | message deleted | messageId, actorId,
reason? | activitylog, search-index |

## 3) Consumed Events
| Event | Producer | Effect |
|---|---|---|
| trust.user.restricted | trust-safety | enforce on can() via
restriction flags |
```

1.4 `SCREENMAP_community-messaging.md` (filled minimum)

```
# SCREENMAP — Community Messaging ({{APP_NAME}})

## 1) Screen Inventory
| Screen | Route/Key | Purpose | Primary Actions (BELS) | RuleIDs |
|---|---|---|---|---|
| RoomChatScreen | /rooms/:roomId | read + send messages | read,
create | R-001, R-002 |
| MessageActionsSheet | modal | delete/report | delete | R-003, R-004
|

## 3) UI Actions Map
| UI Element | Screen | Triggers | Fields | Validation | Error UX |
|---|---|---|---|---|---|
| Send button | RoomChatScreen | community.message.create | text |
non-empty, maxLen | banner w/ reason+next step |
```

```
| Delete option | ActionsSheet | community.message.delete | reason? |
n/a | confirm + result toast |
```

1.5 `TESTPLAN_community-messaging.md` (filled minimum)

```
# TESTPLAN — Community Messaging ({{APP_NAME}})

## 2) Scenario Coverage (from BELS)
- Must cover: S-001..S-006

## 3) Permission Tests
| RuleID | Action | Allow tests | Deny tests | ReasonCode verified? |
|---|---|---|---|---|
| R-002 | message.create | S-001 | S-002,S-003 | YES |

## 5) Abuse / Risk Tests
- RATE_LIMITED: spam send loop triggers cooldown
- replay/idempotency: duplicate clientMessageId does not create
duplicates

## 6) Observability
- verify correlationId in logs for SendMessage
- verify audit log exists for moderator delete
```

1.6 `ERC_community-messaging_v1.md` (lock excerpt)

```
# ERC — Community Messaging ({{APP_NAME}}) v1

## Locked Inputs
- RPBS: ../00_product/RPBS_Product.md
- REBS: ../00_product/REBS_Product.md
- Domain Map: ../00_registry/domain-map.md
- DDES: ./DDES_community-messaging.md
- BELS: ./BELS_community-messaging.md
```

```
- DIM: ./DIM_community-messaging.md
- SCREENMAP: ./SCREENMAP_community-messaging.md
- TESTPLAN: ./TESTPLAN_community-messaging.md

## Frozen Decisions
- Action namespace: community.message.*
- Policy rules: R-001..R-004
- Message states: active, deleted
- Audit required: moderator/admin delete
```

---

2) Code Mapping (How the docs become implementation)

2.1 File mapping (server)

This is where each doc section goes in code.

Doc Source	Code Target	File
Action Vocabulary	action constants	`server/src/core/access/actions.ts`
Policy Rules	`can()` evaluation	`server/src/core/access/accessControl.ts`
Command Handlers	write logic	`server/src/domains/community-messaging/handlers/commands.ts`
State Machine	transition guards	`server/src/domains/community-messaging/state/machine.ts`
DIM Events	publish/consume	`server/src/domains/community-messaging/events/emits.ts`
Reason Codes	enum/list	`server/src/core/errors/errorCodes.ts`
Error Envelope	response builder	`server/src/core/errors/errorEnvelope.ts`

Audit requirements	audit calls	`server/src/core/audit/auditLog.ts`
TESTPLAN scenarios	tests	`server/src/domains/community-messaging/tests/*.test.ts`

2.2 Example: Actions constants (`actions.ts`)

```
// server/src/core/access/actions.ts
export const Actions = {
  COMMUNITY_MESSAGE_CREATE: "community.message.create",
  COMMUNITY_MESSAGE_READ: "community.message.read",
  COMMUNITY_MESSAGE_DELETE: "community.message.delete",
} as const;
```

2.3 Example: AccessControl `can()` (policy evaluation order)

```
// server/src/core/access/accessControl.ts
import { Actions } from "./actions";
import { ReasonCodes } from "../errors/errorCodes";

export function can(actor, action, resource, ctx) {
  // 1) Auth
  if (!actor?.userId) return deny(ReasonCodes.UNAUTHENTICATED, "Please
sign in.");

  // 2) Account status
  if (actor.status === "locked" || actor.status === "disabled") {
    return deny(ReasonCodes.ACCOUNT_DISABLED, "Your account can’t
perform this action right now.");
  }

  // 3) Restrictions (TrustSafety)
  if (action === Actions.COMMUNITY_MESSAGE_CREATE &&
actor.restrictions?.muted) {
```

```
    return deny(ReasonCodes.USER_RESTRICTED, "You can’t send messages
right now.");
  }

  // 4) Visibility
  if (action === Actions.COMMUNITY_MESSAGE_READ || action ===
Actions.COMMUNITY_MESSAGE_CREATE) {
    if (!resource?.roomVisibleToActor) {
      return deny(ReasonCodes.ROOM_NOT_VISIBLE, "You can’t view this
room.");
    }
  }

  // 5) Role + conditions
  if (action === Actions.COMMUNITY_MESSAGE_DELETE) {
    if (actor.role === "admin" || actor.role === "moderator") {
      return allow({ audit: true });
    }
    if (resource?.ownerId === actor.userId &&
resource?.withinDeleteWindow) {
      return allow({ audit: false });
    }
    return deny(ReasonCodes.NOT_OWNER, "You can only delete your own
message.");
  }

  // Default deny
  return deny(ReasonCodes.FORBIDDEN, "You don’t have permission.");
}

function allow(extra = {}) {
  return { allow: true, ...extra };
}

function deny(code, userMessage) {
  return { allow: false, reasonCode: code, userMessage };
}
```

Mapping proof: This directly mirrors BELS:

- evaluation order
- RuleIDs behavior
- reason codes + messages
- audit flag for moderator delete

2.4 Example: Command handler (write path discipline)

```
// server/src/domains/community-messaging/handlers/commands.ts
import { can } from "../../../core/access/accessControl";
import { Actions } from "../../../core/access/actions";
import { ok, fail } from "../../../core/errors/errorEnvelope";
import { publish } from "../../../core/events/eventBus";
import { audit } from "../../../core/audit/auditLog";

export async function sendMessage({ actor, roomId, text,
clientMessageId, ctx, db }) {
  // 1) Validate
  if (!text || text.trim().length === 0) return fail("INVALID_INPUT",
"Message can't be empty.", ctx);
  if (text.length > 2000) return fail("INVALID_INPUT", "Message is too
long.", ctx);

  // 2) Load resources needed for can()
  const room = await db.rooms.get(roomId);
  const roomVisibleToActor = await db.rooms.isVisibleToActor(roomId,
actor.userId);

  // 3) Authorize
  const decision = can(actor, Actions.COMMUNITY_MESSAGE_CREATE, {
roomId, roomVisibleToActor }, ctx);
  if (!decision.allow) return fail(decision.reasonCode,
decision.userMessage, ctx);
```

```
  // 4) Write (owned entity only)
  const message = await db.messages.create({
    roomId,
    actorId: actor.userId,
    text,
    clientMessageId: clientMessageId ?? null
  });

  // 5) Emit event
  publish("community.message.created", {
    messageId: message.id,
    roomId,
    actorId: actor.userId,
    occurredAt: new Date().toISOString(),
    correlationId: ctx.correlationId,
    producerDomain: "community-messaging"
  });

  return ok({ message }, ctx);
}

export async function deleteMessage({ actor, messageId, ctx, db,
reason }) {
  // Load message
  const msg = await db.messages.get(messageId);
  if (!msg) return fail("NOT_FOUND", "Message not found.", ctx);

  const withinDeleteWindow = isWithinDeleteWindow(msg.createdAt,
ctx.now);

  // Authorize
  const decision = can(actor, Actions.COMMUNITY_MESSAGE_DELETE, {
    ownerId: msg.actorId,
    withinDeleteWindow
  }, ctx);

  if (!decision.allow) return fail(decision.reasonCode,
decision.userMessage, ctx);
```

```
  // State transition (simple: mark deleted)
  await db.messages.markDeleted(messageId);

  // Audit if required
  if (decision.audit) {
    audit("MODERATION.MESSAGE_DELETE", actor, { type: "message", id:
messageId }, { reason }, ctx);
  }

  publish("community.message.deleted", {
    messageId,
    actorId: actor.userId,
    occurredAt: new Date().toISOString(),
    correlationId: ctx.correlationId,
    producerDomain: "community-messaging"
  });

  return ok({ deleted: true }, ctx);
}

function isWithinDeleteWindow(createdAt, now) {
  const ms = new Date(now).getTime() - new Date(createdAt).getTime();
  return ms <= 15 * 60 * 1000; // example: 15 minutes
}
```

2.5 Tests map directly to BELS scenario IDs

```
// server/src/domains/community-messaging/tests/bels_scenarios.test.ts
import { sendMessage, deleteMessage } from "../handlers/commands";

test("S-001 allow: member sends message", async () => { /* ... */ });
test("S-002 deny: muted user cannot send", async () => { /* expect
USER_RESTRICTED */ });
test("S-005 deny: non-owner cannot delete", async () => { /* expect
NOT_OWNER */ });
```

```
test("S-006 allow: moderator delete audits", async () => { /* expect
audit called */ });
```

Rule: Test names include `S-###` so traceability is automatic.

---

3) How to use this golden example

When building any domain:

1. Copy the **pattern** (not the content)
2. Ensure BELS tables are complete (policy + state + scenarios)
3. Implement:
    - `Actions`
    - `can()`
    - command handlers
    - events
    - audit
    - tests mapping scenario IDs

Stop condition: If a domain has handlers without matching BELS scenarios/tests, the build is incomplete.

Control Plane Domain Pack + Code Mapping

Golden Example — Control Plane Domain Pack + Code Mapping (Turn-Key)

Golden Control Plane Domain

Domain: Operations Control Plane (in-app admin controls)
Slug: `ops-control-plane`
Prefix: `ops`
Type: controlplane domain
Primary actors: Admin, Moderator (optionally Support/Operator)

Core idea: This domain does not "own business features." It owns **controls** that influence them: feature flags, throttles, moderation tools, emergency locks, configuration.

---

1) `DDES_ops-control-plane.md` (filled minimum)

```
# DDES — Ops Control Plane ({{APP_NAME}})

## 0) Snapshot
- Domain Slug: ops-control-plane
- Prefix: ops
- Owner Type: Control Plane
- Audience: Admin/Moderator/Operator
- Risk Level: HIGH (affects production behavior)

## 1) Domain Purpose
Provide safe, audited, role-gated controls to manage the application
in production:
- Feature flags and remote config
- Emergency controls (kill switches, lockdown modes)
- Safety throttles (rate-limit overrides)
- Moderation views (queue + enforcement tools)
- Operational visibility (audit trail, key health checks)
```

2) System of Record Ownership
OWNS
- OpsConfig (remote config entries)
- FeatureFlag (flag values + rollout scopes)
- OpsIncident (emergency mode + notes)
- OpsActionAudit (structured admin action logs; may be implemented as global audit sink)

DOES NOT OWN
- Business domain data (messages, proposals, payments, etc.)
- Auth identities (Identity domain owns user truth)
- Restrictions truth (TrustSafety owns restrictions; ops triggers actions via contracts)

3) Public Contracts
APIs
- GET /v1/ops/flags
- PATCH /v1/ops/flags/:flagKey
- POST /v1/ops/incidents/lockdown/start
- POST /v1/ops/incidents/lockdown/end
- PATCH /v1/ops/config/:key
- GET /v1/ops/audit (query)

Events Emitted
- ops.flag.changed
- ops.lockdown.started
- ops.lockdown.ended
- ops.config.changed

Cross-Domain Calls (allowed)
- TrustSafety: applyEnforcement(userId, type, reason)
- Notifications: sendOperationalNotice(targets, message)

4) Core Entities
Entity	Description	IDs	Lifecycle
FeatureFlag	boolean or variant flag	flagKey	active always
OpsConfig	key/value config	key	active always

```
| OpsIncident | emergency mode record | incidentId | open/closed |
| OpsAuditEvent | immutable record | auditId | append-only |

## 5) Invariants
- INV-001: All ops writes require AccessControl allow + audit.
- INV-002: High-risk actions require step-up auth.
- INV-003: Lockdown mode must be globally enforced by business domains
via a shared check.
- INV-004: Audit events are append-only; no edits/deletes.

## 6) Failure & Recovery
- Writes must be idempotent via requestId/correlationId.
- If config write succeeds but event publish fails: record “pending
emit” and retry (outbox recommended).

## 7) Abuse & Safety
- No silent admin power: every change visible in audit feed.
- Rate limit ops writes even for admins (defense-in-depth).
- Restrict read access: audit feed may include sensitive metadata.

## 8) Observability
- Every ops action logs: actorId, action, target, previousValue,
newValue, correlationId.
- Emit metrics: ops_changes_total, lockdown_active,
failed_ops_writes_total.
```

---

2) `BELS_ops-control-plane.md` (filled minimum)

```
# BELS — Ops Control Plane ({{APP_NAME}})
(See TEMPLATE_RULES.md — no invention, SourceRef required)

## 1) Action Namespace
- Prefix: ops
```

2) Action Vocabulary

Action	Actor(s)	Resource	Description	SourceRef	Status
ops.flag.read	admin/mod	FeatureFlag	View flags list	PROJECT_OVERVIEW > Ops Controls	DRAFT
ops.flag.update	admin	FeatureFlag	Change flag value	RPBS > Hard Rules > Ops	DRAFT
ops.config.update	admin	OpsConfig	Change remote config	RPBS > Hard Rules > Ops	DRAFT
ops.lockdown.start	admin	OpsIncident	Start emergency lockdown	RPBS > System Limits & Safety	DRAFT
ops.lockdown.end	admin	OpsIncident	End emergency lockdown	RPBS > System Limits & Safety	DRAFT
ops.audit.read	admin/mod	OpsAuditEvent	View audit trail	REBS > Observability/Audit	DRAFT

3) Command Handlers

Command	Actions served	Inputs	Validations	Writes	Emits Events	Idempotency	SourceRef
UpdateFlag	ops.flag.update	flagKey, value, scope?	flag exists, value type ok	FeatureFlag	ops.flag.changed	requestId	RPBS > Ops Controls
UpdateConfig	ops.config.update	key, value	allowlist key, validate type	OpsConfig	ops.config.changed	requestId	RPBS > Ops Controls
StartLockdown	ops.lockdown.start	reason, scope?	reason required	OpsIncident(open) + global lock	ops.lockdown.started	requestId	RPBS > Min Safety
EndLockdown	ops.lockdown.end	incidentId	incident open	OpsIncident(closed)	ops.lockdown.ended	requestId	RPBS > Min Safety

4) Step-Up Requirements (high-risk actions)

Action	Step-up Required?	Step-up Window	ReasonCode if missing	SourceRef

ops.flag.update	YES (for certain flags)	10 min	STEP_UP_REQUIRED	REBS > Security Baseline
ops.config.update	YES	10 min	STEP_UP_REQUIRED	REBS > Security Baseline
ops.lockdown.start	YES (always)	10 min	STEP_UP_REQUIRED	REBS > Security Baseline
ops.lockdown.end	YES (always)	10 min	STEP_UP_REQUIRED	REBS > Security Baseline

5) Policy Rules

RuleID	Action	Allow When	Deny ReasonCode	Deny User Message	Audit?	SourceRef
OPS-R-001	ops.flag.read	actor role in {Admin, Moderator}	FORBIDDEN	“You don’t have access to ops.”	YES	RPBS > Role Powers
OPS-R-002	ops.flag.update	actor role=Admin AND stepUpValid	STEP_UP_REQUIRED/FORBIDDEN	“Re-authenticate to continue.”	YES	REBS > Security
OPS-R-003	ops.config.update	actor role=Admin AND stepUpValid AND key allowed	INVALID_CONFIG_KEY	“That setting can’t be changed.”	YES	RPBS > Ops Controls
OPS-R-004	ops.lockdown.start	actor role=Admin AND stepUpValid	STEP_UP_REQUIRED/FORBIDDEN	“Re-authenticate to continue.”	YES	REBS > Security
OPS-R-005	ops.lockdown.end	actor role=Admin AND stepUpValid	STEP_UP_REQUIRED/FORBIDDEN	“Re-authenticate to continue.”	YES	REBS > Security
OPS-R-006	ops.audit.read	actor role in {Admin, Moderator}	FORBIDDEN	“You don’t have access to ops.”	YES	REBS > Audit

Policy Evaluation Order

1) Auth check
2) Account status check
3) Role gate (admin/mod)
4) Step-up check (if required for action)
5) Key allowlist + validation (for config)
6) Default deny

6) State Machine — OpsIncident

State	Meaning
open	lockdown active
closed	lockdown ended

Transition	From	To	Guard	Side Effects	Deny ReasonCode	SourceRef
start_lockdown	closed/none	open	admin + stepUpValid	set globalLockdown=true	STEP_UP_REQUIRED	REBS
end_lockdown	open	closed	admin + stepUpValid	set globalLockdown=false	STEP_UP_REQUIRED	REBS

7) Error & Recovery

Scenario	Error Code	User Message	Retry?	Recovery
Step-up expired	STEP_UP_REQUIRED	“Re-authenticate to continue.”	YES	trigger step-up flow
Not allowed role	FORBIDDEN	“You don’t have access.”	NO	none
Invalid config key	INVALID_CONFIG_KEY	“That setting can’t be changed.”	NO	none
Rate limited	RATE_LIMITED	“Too many admin changes.”	YES	cooldown

8) Acceptance Scenarios

ID	Given	When	Then	Type
OPS-S-001	Moderator	reads ops flags	allow	Allow
OPS-S-002	Member	reads ops flags	deny FORBIDDEN	Deny
OPS-S-003	Admin no step-up	start lockdown	deny STEP_UP_REQUIRED	Deny
OPS-S-004	Admin step-up valid	start lockdown	lockdown=open, event emitted, audit created	Allow
OPS-S-005	Admin step-up valid	update config key not allowlisted	deny INVALID_CONFIG_KEY	Deny

```
| OPS-S-006 | Admin step-up valid | update feature flag | success +
audit + event | Allow |
```

---

3) `DIM_ops-control-plane.md` (filled minimum)

```
# DIM — Ops Control Plane ({{APP_NAME}})

## Interaction Matrix
| Use Case | Primary Action | Owning Write Domain | Consult | Writes |
Emits | Side Effects |
|---|---|---|---|---|---|---|
| Change feature flag | ops.flag.update | ops-control-plane |
access-control | FeatureFlag | ops.flag.changed | affects business
domains behavior |
| Start lockdown | ops.lockdown.start | ops-control-plane |
access-control | OpsIncident | ops.lockdown.started | business domains
enforce global deny |

## Event Catalog (emits)
- ops.flag.changed (flagKey, newValue, actorId, correlationId)
- ops.lockdown.started (incidentId, reason, actorId, correlationId)
- ops.lockdown.ended (incidentId, actorId, correlationId)

## Consumers
- Business domains read global lockdown state before processing
writes.
- Notifications may send operational message if configured.
```

---

4) `SCREENMAP_ops-control-plane.md` (filled minimum)

```
# SCREENMAP — Ops Control Plane ({{APP_NAME}})

## Screen Inventory
| Screen | Key | Purpose | Primary Actions (BELS) |
|---|---|---|---|
| OpsDashboard | ops/home | entry | ops.flag.read, ops.audit.read |
| FeatureFlagsScreen | ops/flags | view/edit flags | ops.flag.read,
ops.flag.update |
| RemoteConfigScreen | ops/config | edit allowlisted config |
ops.config.update |
| LockdownScreen | ops/lockdown | start/end lockdown |
ops.lockdown.start, ops.lockdown.end |
| AuditLogScreen | ops/audit | view audit trail | ops.audit.read |
| StepUpModal | ops/stepup | re-auth confirm | (step-up flow) |

## UI Actions Map
| UI Element | Screen | Triggers | Notes |
|---|---|---|---|
| Toggle flag | FeatureFlagsScreen | ops.flag.update | requires
step-up for protected flags |
| Save config | RemoteConfigScreen | ops.config.update | allowlist
enforced |
| Start lockdown | LockdownScreen | ops.lockdown.start | confirmation
required |
| End lockdown | LockdownScreen | ops.lockdown.end | confirmation
required |
```

---

5) `TESTPLAN_ops-control-plane.md` (filled minimum)

```
# TESTPLAN — Ops Control Plane ({{APP_NAME}})

## Scenario Coverage
Must cover OPS-S-001..OPS-S-006
```

```
## Permission & Step-Up Tests
- OPS-S-003 verifies STEP_UP_REQUIRED
- OPS-S-002 verifies FORBIDDEN for non-privileged roles

## Audit Tests
- Any allow for ops.*.update/start/end must create audit event
- Audit event must include: actorId, action, target, before/after,
correlationId

## Abuse Tests
- Rate limit ops writes
- Ensure audit feed never exposes secrets (redaction test)

## Observability Tests
- correlationId appears in logs for each ops command
- metrics increment on ops changes
```

---

6) Code Mapping (Control Plane specific)

6.1 Special Integration Requirements

Control plane requires two extra universal hooks:

A) Step-Up Gate

- `core/access/stepUp.ts` (or in your auth module)
- Expose:
  - `requireStepUp(actor, ctx) -> allow/deny`
  - `isStepUpValid(actor, ctx) -> boolean`

B) Global Lockdown Check (write-path intercept)

Every write handler in every domain must do:

1. `if (globalLockdownActive && !actor.isAdmin) deny LOCKDOWN_ACTIVE`

This should live in one place to prevent drift:

- `core/access/globalGuards.ts`

6.2 Example: policy checks (step-up + audit)

```
// server/src/core/access/stepUp.ts
export function isStepUpValid(actor, ctx) {
  const ts = actor?.stepUpAt ? new Date(actor.stepUpAt).getTime() : 0;
  const now = new Date(ctx.now).getTime();
  return now - ts <= 10 * 60 * 1000; // 10 minutes
}

// server/src/core/access/globalGuards.ts
export function enforceGlobalLockdown({ actor, action }) {
  const isWrite = action.endsWith(".create") ||
action.endsWith(".update") || action.endsWith(".delete") ||
action.includes("lockdown");
  if (!isWrite) return { ok: true };

  if (globalThis.__LOCKDOWN_ACTIVE__ && actor.role !== "admin") {
    return { ok: false, reasonCode: "LOCKDOWN_ACTIVE", userMessage:
"The app is temporarily in restricted mode." };
  }
  return { ok: true };
}
```

6.3 Example: Ops command handler (audit-first)

```
// server/src/domains/ops-control-plane/handlers/commands.ts
import { can } from "../../../core/access/accessControl";
import { isStepUpValid } from "../../../core/access/stepUp";
import { ok, fail } from "../../../core/errors/errorEnvelope";
```

```
import { audit } from "../../../core/audit/auditLog";
import { publish } from "../../../core/events/eventBus";

export async function startLockdown({ actor, reason, ctx, db }) {
  const decision = can(actor, "ops.lockdown.start", { }, ctx);
  if (!decision.allow) return fail(decision.reasonCode,
decision.userMessage, ctx);

  if (!isStepUpValid(actor, ctx)) return fail("STEP_UP_REQUIRED",
"Re-authenticate to continue.", ctx);

  const incident = await db.opsIncidents.create({ reason, state:
"open", startedBy: actor.userId });

  // Set global lock (implementation depends on your system; DB-backed
flag recommended)
  await db.opsState.setLockdown(true);

  audit("OPS.LOCKDOWN.START", actor, { type: "opsIncident", id:
incident.id }, { reason }, ctx);

  publish("ops.lockdown.started", {
    incidentId: incident.id,
    reason,
    actorId: actor.userId,
    occurredAt: new Date().toISOString(),
    correlationId: ctx.correlationId,
    producerDomain: "ops-control-plane"
  });

  return ok({ incident }, ctx);
}
```

---

7) Why this control plane example matters

It demonstrates the non-negotiables for production operations:

- step-up auth for dangerous actions
- audit-first behavior
- global guards that business domains must respect
- allowlists for config keys to prevent accidental nukes
- deny codes and safe user messages

Golden Example Contract (Minimum Counts + `roshi:verify` Gates)

This contract makes the Golden Examples enforceable. It prevents “schema-only” output by forcing every domain to meet minimum **logic + scenarios + tests** completeness before it can be locked or considered production-ready.

A) Domain Completeness Minimums (required)

A1) Domain Types

Each domain must declare `type` in `roshi/domains.json`:

- `business`
- `controlplane`
- `crosscutting`

Minimums differ by type.

A2) Minimums — Business Domains

A business domain is anything that directly powers user-facing features (community, governance, marketplace, etc.).

Required minimums before ERC lock:

- **Action Vocabulary:** ≥ **8** actions
- **Command Handlers:** ≥ **3** commands
- **Policy Rules:** ≥ **10** rules
- **State Machine:** ≥ **1** lifecycle table **OR** explicit “stateless” declaration
- **Acceptance Scenarios:** ≥ **12** scenarios
    - ≥ **6 Allow**
    - ≥ **6 Deny** (deny must include reason codes)
- **TESTPLAN:** ≥ **12** tests mapped to scenario IDs (`S-###`)
- **Reason Codes:** 100% of denies reference existing reason codes

Required minimums for “production onboarding phase”:

- At least **2 abuse/risk tests**
- At least **1 observability verification**
- At least **1 recovery scenario** (retry, idempotency, or failure handling)

A3) Minimums — Control Plane Domains

Control plane domains are high-risk ops/admin tooling and remote controls.

Required minimums before ERC lock:

- **Action Vocabulary:** ≥ **6** actions
- **Commands:** ≥ **3** commands

- **Policy Rules:** ≥ **8** rules
- **Step-Up Table: REQUIRED** for high-risk actions
- **Acceptance Scenarios:** ≥ **10** scenarios
    - ≥ **4 Deny** must include:
        - `FORBIDDEN` (role gate)
        - `STEP_UP_REQUIRED` (high-risk protection)
        - one “invalid config/key” type deny
- **Audit Coverage:** 100% of *write* actions must be audited
- **TESTPLAN:** ≥ **10** tests mapped to scenario IDs
- **Security Tests:** ≥ **2**
    - step-up expiry test
    - redaction test for audit/config exposure

A4) Minimums — Cross-Cutting Domains

Cross-cutting domains support the whole system: AccessControl, Errors, Audit, Events, Observability.

Required minimums before ERC lock:

- **Action Vocabulary:** optional (may be N/A)
- **Policy Rules:** ≥ **10** (for AccessControl) OR N/A if non-policy module
- **Error/Envelope Contract:** REQUIRED
- **Audit Contract:** REQUIRED
- **Events Contract:** REQUIRED

- **TESTPLAN:** ≥ **8** tests, including:
    - allow/deny baseline tests (if AccessControl)
    - error envelope formatting tests
    - audit write tests
    - correlationId propagation test

B) Mandatory Sections (structure gates)

For each domain, these docs must exist and contain required sections:

B1) Required docs (always)

- `DDES_<slug>.md`
- `UX_Foundations_<slug>.md`
- `UI_Constraints_<slug>.md`
- `BELS_<slug>.md`
- `DIM_<slug>.md`
- `SCREENMAP_<slug>.md`
- `TESTPLAN_<slug>.md`

B2) BELS required sections

`roshi:verify` must fail if any are missing:

- Action Vocabulary table

- Command Handlers table
- Policy Rules table (with ReasonCode + SourceRef)
- Policy Evaluation Order
- State Machine section (or explicit stateless declaration)
- Error & Recovery table
- Acceptance Scenarios table
- Open Questions section

B3) TESTPLAN required sections

- Scenario Coverage mapping
- Permission tests (verify reason codes)
- Abuse/risk tests
- Observability verification

C) `UNKNOWN` and `SourceRef` Gates (anti-invention)

C1) For “draft” documents

- `UNKNOWN` allowed
- missing info must go to Open Questions

C2) For “locked” domains (ERC exists)

`roshi:verify` must fail if any of these contain `UNKNOWN`:

- Policy Rules table
- State Machine transitions table
- Acceptance Scenarios table
- Step-Up table (if controlplane requires it)

C3) SourceRef requirement

`roshi:verify` must fail if any row in these tables lacks `SourceRef`:

- Action Vocabulary
- Policy Rules
- State transitions
- Acceptance Scenarios

D) Reason Code Gates (deny discipline)

`roshi:verify` must fail if:

- any Deny scenario lacks a reason code
- any policy deny uses a reason code not in `reason-codes.md`
- any deny has no user-facing message guidance

E) Test Mapping Gates (docs → tests)

E1) Scenario ID format (required)

- Business: `S-001`, `S-002`, …
- Control plane: `OPS-S-001`, …
- Keep stable IDs once locked

E2) Minimum test mapping

`roshi:verify` must confirm:

- Every scenario ID listed in BELS appears in TESTPLAN coverage list
- TESTPLAN includes a “test id list” section:
    - `TC_<slug>_<scenarioId>`

Example:

- `TC_community-messaging_S-001`

Fail if:

- scenario count < minimums
- deny scenario tests do not assert reason code

F) Gate behavior: Lock is impossible without passing verify

F1) Lock preflight requirement

Before `roshi:lock` writes ERC, it must run an internal verify:

- If any gate fails → no ERC created

F2) Verify is authoritative

`roshi:verify` is the final gate:

- PASS → safe to build
- FAIL → stop building

G) Add this to `roshi/domains.json` (minimums config)

To keep it universal + adjustable, include a minimums section:

```
{
  "minimums": {
    "business": { "actions": 8, "commands": 3, "policyRules": 10,
"scenarios": 12, "tests": 12 },
    "controlplane": { "actions": 6, "commands": 3, "policyRules": 8,
"scenarios": 10, "tests": 10, "requiresStepUp": true,
"requiresAuditOnWrites": true },
    "crosscutting": { "tests": 8 }
  }
}
```

H) `roshi:verify` output requirements (actionable)

When failing, `roshi:verify` must print:

- which domain failed

- which gate failed
- exact counts found vs required
- list of missing reason codes
- list of UNKNOWN locations
- next steps (what to add)

Example failure line:

- `FAIL ops-control-plane: scenarios=7 required=10 (missing 3 scenarios, include 2 deny + step-up deny)`

Roshi Verify Checklist (Human) — 1 Page Gate

Roshi Verify Checklist (Human) — 1 Page Gate

Use this checklist **right before** you run `roshi:lock`. If any item fails, do **not** lock ERC yet.

---

0) Preflight: correct sources + no drift

- RPBS exists and is current (`docs/roshi_v2/00_product/RPBS_Product.md`)
- REBS exists and is current (`docs/roshi_v2/00_product/REBS_Product.md`)
- Domain Map exists and lists this domain (`docs/roshi_v2/00_registry/domain-map.md`)
- Action Vocabulary + Reason Codes exist and are up to date
- Repo Mapping exists and is filled (if integrating into existing repo)

---

1) Domain pack exists (no missing docs)

For this domain `<slug>`:

- DDES exists
- UX Foundations exists
- UI Constraints exists
- BELS exists
- DIM exists
- SCREENMAP exists

- TESTPLAN exists

If any are missing: stop, run `roshi:gen`.

---

2) BELS completeness (logic is real)

Action vocabulary

- Action list is present and uses correct namespace prefix
- Each action has: Actor(s), Resource, Description, SourceRef, Status

Command handlers table

- There are real commands (not placeholders)
- Each command defines: inputs, validations, writes, emits, idempotency, SourceRef

Policy rules table

- Every rule has a RuleID
- Every deny has:
    - Reason Code (exists in Reason Codes registry)
    - User-facing message guidance
- Audit flag is explicit (YES/NO)
- SourceRef is present for every rule row

Policy evaluation order

- Evaluation order is written

- TrustSafety restrictions included where relevant
- Default deny is explicit

State machine (or “stateless” declared)

- State machine exists OR explicitly states “STATELESS DOMAIN”
- Every transition has:
    - guard conditions
    - deny reason code
    - SourceRef

Error & recovery

- Error table exists
- Includes at least one retry/recovery rule (idempotency, retry, fallback, cooldown)

Acceptance scenarios

- Scenarios exist with stable IDs
- Includes both allow and deny
- Every deny scenario asserts a reason code
- SourceRef present for each scenario

---

3) DIM sanity (no domain boundary violations)

- Domain OWNS vs DOES NOT OWN is clear in DDES
- No cross-domain DB writes implied
- Cross-domain effects use events or orchestrators
- DIM lists emitted events + consumers (even if minimal)

---

4) SCREENMAP sanity (UI maps to actions)

- Each screen lists primary actions (BELS actions)
- UI buttons/forms map to BELS actions
- Empty/loading/error states are defined
- Deny UX is “reason + next step” (never silent)

---

5) TESTPLAN sanity (scenarios become tests)

- Scenario Coverage section lists all scenario IDs from BELS
- Permission tests assert:
    - allow/deny outcome
    - reason code on deny
- Abuse/risk tests exist (rate limit, spam, replay, etc.)

- Observability verification exists (correlationId, audit event, metrics/logs)

---

6) UNKNOWN gate (hard stop)

For ERC lock readiness:

- Policy Rules table has **zero** `UNKNOWN`
- State transitions have **zero** `UNKNOWN`
- Acceptance scenarios have **zero** `UNKNOWN`
- (Control plane only) Step-up table has **zero** `UNKNOWN`

If any `UNKNOWN` remains in those sections: do not lock.

---

7) Minimum counts met (type-based)

Check your domain type and verify the minimums:

Business domain

- Actions ≥ required
- Policy rules ≥ required
- Scenarios ≥ required (balanced allow/deny)
- Tests ≥ required (mapped to scenario IDs)

Control plane domain

- Step-up requirements present (for high-risk actions)
- Audit coverage = 100% for writes
- Deny scenarios include FORBIDDEN + STEP_UP_REQUIRED
- Redaction test included

Cross-cutting domain

- Core contracts exist (errors/audit/events/accesscontrol)
- Baseline tests exist for contract behavior

---

8) Repo integration safety (existing apps)

If integrating into an existing app:

- You will not rename routes/params or restructure navigation
- You will use adapter layers where signatures differ
- If duplicates exist, you have a shadow-mode plan
- You documented all actual code locations in `repo-mapping.md`

---

9) Lock readiness statement (final)

- “This domain is lock-ready: policies, states, scenarios, reason codes, and tests are complete. No UNKNOWN remains in critical sections.”

Then run:

```
npm run roshi:verify -- --domain <slug>
```

```
npm run roshi:lock   -- --domain <slug> --version v1
```

Prompt Playbook (Turn-Key) — Roshi v2

Prompt Playbook (Turn-Key) — Roshi v2

How to use this

- Pick the prompt that matches the step you're on.
- Paste it into your Replit agent chat.
- Do not add extra instructions unless necessary.
- The agent must produce a **ROSHI_REPORT** at the end of every run.

---

0) Universal Guardrails (prepend to every prompt)

```
GUARDRAILS (NON-NEGOTIABLE):
- No invention: if a rule/decision is not explicitly supported by
sources, label UNKNOWN and add to Open Questions.
- No overwrite: do not overwrite existing authored domain docs under
docs/roshi_v2/02_domains/.
- Do not restructure the repo, rename routes, move screens, or change
navigation contracts.
- Reuse existing modules; use adapters instead of refactors.
- Server is source of truth; client gating is UX only.
- Every deny must include a Reason Code + user-facing message
guidance.
- Every policy/state/scenario row must include SourceRef.
- Output a ROSHI_REPORT listing created/modified/skipped/errors.
```

---

1) Prompt — Roshi Init (create doc system skeleton)

```
TASK: Implement Roshi v2 doc workspace initialization.

SCOPE:
- Create docs/roshi_v2 folder structure if missing.
- Create baseline files in:
  - 00_product/
  - 00_registry/
  - 01_templates/
  - 02_domains/
  - 03_workflows/
- Do not modify app runtime code (client/server).

REQUIREMENTS:
- Idempotent: running twice should not break or duplicate.
- If a file exists, do not overwrite; only create missing files.
- Create docs/roshi_v2/00_registry/repo-mapping.md if missing.

OUTPUT:
- Print ROSHI_REPORT.
```

Run:

```
npm run roshi:init
```

---

2) Prompt — Repo Mapping (Existing repo adoption)

```
TASK: Create/complete docs/roshi_v2/00_registry/repo-mapping.md for
this repo.

INSTRUCTIONS:
```

```
- Locate the actual backend and frontend folders.
- Find existing modules for: AccessControl/authorize, errors, audit,
events, handlers, state transitions, validators, telemetry.
- Fill the mapping table completely. Mark FOUND/MISSING.
- If something is missing, propose the minimal file(s) to create
WITHOUT refactoring existing code.
- Add “Migration Plan for Duplicates” if permission checks exist in
multiple places.

OUTPUT:
- Update repo-mapping.md only.
- Print ROSHI_REPORT.
```

---

3) Prompt — Update Templates (template-only upgrade)

```
TASK: Update Roshi templates to the fortified versions.

SCOPE:
- Only modify docs/roshi_v2/01_templates/*
- Add TEMPLATE_VERSION.md and TEMPLATE_RULES.md if missing.
- Do not touch docs/roshi_v2/02_domains/*

REQUIREMENTS:
- Include SourceRef column requirements.
- Include UNKNOWN policy + Open Questions rule.
- Ensure BELS template includes: policy evaluation order, state
machines, error/recovery, scenarios, step-up section (if
controlplane).

OUTPUT:
- Print ROSHI_REPORT.
```

---

4) Prompt — Domain List + Registry Update (domains.json + domain-map)

```
TASK: Update the domain registry.

SCOPE:
- Update roshi/domains.json with the full domain list (name, slug,
prefix, type).
- Update docs/roshi_v2/00_registry/domain-map.md to match.
- Do not generate domain docs yet.

RULES:
- Slugs and prefixes must be unique.
- Mark each domain type: business/controlplane/crosscutting.
- Include minimums config in roshi/domains.json.

OUTPUT:
- Print ROSHI_REPORT.
```

---

5) Prompt — Roshi Gen (create per-domain doc pack shells)

```
TASK: Generate Roshi per-domain doc packs (shape only).

SCOPE:
- Create missing domain doc files under
docs/roshi_v2/02_domains/<slug>/
- Use templates from docs/roshi_v2/01_templates/
- Do NOT overwrite existing domain docs.

REQUIREMENTS:
- For each domain: create DDES/UX/UI/BELS/DIM/SCREENMAP/TESTPLAN if
missing.
- If the domain folder exists, create only missing files.
```

```
- Print missing file list if any failures.

OUTPUT:
- Print ROSHI_REPORT.
```

Run:

```
npm run roshi:gen
```

---

6) Prompt — Roshi Seed (safe prefill only)

```
TASK: Seed starter scaffolding into domain docs.

SCOPE:
- Only fill empty/unstarted docs or docs with <!-- SEED_OK --> marker.
- Insert blank tables, headings, and placeholders.
- DO NOT invent real policies or states.

REQUIREMENTS:
- Add a “SEED_MARKER” comment at inserted sections for traceability.
- If a file is non-empty and lacks SEED_OK, skip it.

OUTPUT:
- Print ROSHI_REPORT.
```

Run:

```
npm run roshi:seed
```

---

7) Prompt — Inputs Pack Sync (PROJECT_OVERVIEW + sources)

```
TASK: Ensure the Inputs Pack is present and registered.
```

```
SCOPE:
- Confirm these files exist:
  - docs/roshi_v2/00_product/PROJECT_OVERVIEW.md
  - docs/roshi_v2/00_product/RPBS_Product.md
  - docs/roshi_v2/00_product/REBS_Product.md
  - docs/roshi_v2/00_registry/action-vocabulary.md
  - docs/roshi_v2/00_registry/reason-codes.md
  - docs/roshi_v2/00_registry/domain-map.md
  - docs/roshi_v2/00_registry/glossary.md
- Confirm roshi/sources.json points to them.
- Do not draft yet.

OUTPUT:
- Print ROSHI_REPORT, including missing inputs (if any).
```

---

8) Prompt — Roshi Draft (truth candidates: logic, not schema)

```
TASK: Run roshi:draft for <domain-slug> to produce truth candidates.

GUARDRAILS APPLY.

SCOPE:
- Only update clearly marked draft sections of:
  - BELS_<slug>.md
  - DIM_<slug>.md (events + interactions if discoverable)
  - TESTPLAN_<slug>.md (scenario coverage list)
- Create or update OPEN_QUESTIONS_<slug>.md.
- Do not modify LOCKED sections.

STRICT RULES:
- Extractive-only from sources listed in roshi/sources.json.
- Every row written must include SourceRef.
- If information is missing: write UNKNOWN and add to OPEN_QUESTIONS.
```

```
OUTPUT:
- Print ROSHI_REPORT.
- Print counts: actions/rules/scenarios/UNKNOWNs.
```

Run:

```
npm run roshi:draft -- --domain <domain-slug>
```

---

9) Prompt — Review Packet (find gaps + conflicts)

```
TASK: Produce a review packet for <domain-slug>.

SCOPE:
- Do not change domain truth content.
- Generate a REVIEW_PACKET_<slug>.md inside the domain folder
containing:
  - UNKNOWN list (with locations)
  - missing reason codes
  - missing required sections
  - minimum counts vs required counts
  - conflicts between sources
  - recommended next edits (human)

OUTPUT:
- Print ROSHI_REPORT.
```

Run:

```
npm run roshi:review -- --domain <domain-slug>
```

---

10) Prompt — Lock Domain (ERC) (freeze meaning)

```
TASK: Lock <domain-slug> into an ERC.

PRECONDITIONS:
- Run roshi:verify for the domain first.
- Zero UNKNOWN in policy rules, transitions, and acceptance scenarios.
- Reason codes referenced must exist.

SCOPE:
- Create ERC_<slug>_<version>.md in the domain folder.
- ERC must list locked inputs and frozen decisions.
- Do not change business meaning during lock.

OUTPUT:
- Print ROSHI_REPORT.
```

Run:

```
npm run roshi:verify -- --domain <domain-slug>
npm run roshi:lock   -- --domain <domain-slug> --version v1
```

---

11) Prompt — Build From ERC (real logic implementation)

```
TASK: Implement production logic for <domain-slug> from ERC_<slug>_v1.

GUARDRAILS APPLY.

SCOPE:
- Use repo-mapping.md to determine actual code locations.
- Implement:
  - AccessControl rules (server-side)
```

```
  - command handlers
  - state machine transitions (if applicable)
  - events (DIM emits/consumes)
  - audit requirements
  - error envelope with reason codes
  - automated tests mapped to scenario IDs

STRICT RULES:
- Do not modify navigation contracts or rename routes/params.
- Do not introduce cross-domain DB writes.
- Use adapters instead of refactoring existing app architecture.
- Every deny must return the correct reason code.

OUTPUT:
- Print ROSHI_REPORT.
- List tests added and scenario IDs covered.
```

---

12) Prompt — Safe Migration (avoid duplication/breakage)

```
TASK: Apply Roshi enforcement to an existing repo without duplication
or breakage.

SCOPE:
- Identify existing scattered permission checks and document them.
- Implement AccessControl adapter (do not rewrite callers yet).
- Add “shadow mode” option:
  - compute old decision and new can() decision
  - log differences with correlationId
  - keep old decision enforced until diff rate is low
- Add tests for allow/deny reason codes.
- Do not delete old checks yet.

OUTPUT:
- Update repo-mapping.md with duplicate locations and cutover plan.
```

```
- Print ROSHI_REPORT.
```

---

13) Prompt — Verify Gate (stop bad locks/builds)

```
TASK: Run roshi:verify and fix failures.

SCOPE:
- Run verify for <domain-slug>.
- If it fails:
  - Do not lock
  - Do not build
  - Produce a FIX_LIST_<slug>.md listing what to change in docs to
pass.

OUTPUT:
- Print ROSHI_REPORT + verify failure reasons.
```

Run:

```
npm run roshi:verify -- --domain <domain-slug>
```

Recommended “Daily Loop” prompt (copy/paste)

```
TASK: Daily Roshi loop for <domain-slug>.

1) Run roshi:draft for the domain.
2) Generate review packet.
3) Print UNKNOWN list + minimums status.
4) Do not lock; do not build.

OUTPUT: ROSHI_REPORT.
```

Doc to Func translation

Here's an efficient way to do that that works *specifically* for startup-grade apps and AI-assisted coding.

The core idea

Translate docs into **three layers** the app can actually run:

1. **Rules (Policy)** — "who can do what, where, and when"
2. **State Machines (Flows)** — "what happens next after an action"
3. **Contracts (Data + API)** — "what objects exist and how they change"

If you do only these three, your app stops being "a bunch of screens" and becomes "a system."

Step 1 — Convert your docs into a "Rules Catalog" (fast)

Make a single table (literally a checklist) with rows like:

- **Actor**: Visitor / Passport / Citizen / Mod / Admin
- **Resource**: Channel / Room / Message / Livestream / Proposal / Order / Profile
- **Action**: Read / Write / Edit / Delete / Report / Pin / Invite / Join / Vote / Pay
- **Scope**: Any / Own / Same-community / Same-channel / Paid-only / Public-only
- **Conditions**: "must be verified", "must be in good standing", "must pass step-up 2FA"
- **Result**: Allow / Deny / Allow-with-limit / Allow-with-review
- **Audit?**: yes/no (log it)

This becomes your **single source of truth**.

Why it's efficient: you stop writing paragraphs and start writing **if/then reality**.

Step 2 — Turn Rules into a “Policy Engine” (one place, reused everywhere)

Instead of sprinkling `if (role === 'ADMIN')` all over the app:

- Create one function on the server: `can(actor, action, resource, context) -> decision`
- Every protected endpoint calls it.
- Client uses it only to **hide/disable UI** (server is truth).

Result: Your documentation becomes code **once**, and your whole app obeys it.

Startup-grade pattern:

- `PolicyDecision = { allow: boolean, reason?: string, constraints?: {...} }`
- Constraints can include rate limits, “write disabled”, “read only”, “requires step-up”.

Step 3 — Define “Interaction Flows” as state machines (not text)

Docs usually say: “Members can request access… mods approve…”

Turn that into explicit states:

Example: **Join Community**

- `new` → `requested` → (`approved` | `rejected`) → `member`
- Events: `request_join`, `approve_join`, `reject_join`
- Side effects: notifications, audit logs, welcome message, role upgrade

Do this for your top 5 flows (chat posting, joining channels, going live, making purchases, voting, etc.).

Why this matters for vibe coding: AI writes code way better when you give it **states + events**.

Step 4 — Convert each flow into API contracts + DB tables

For each flow, define:

- **Entities** (tables): `join_requests`, `moderation_actions`, `messages`, `roles`
- **Endpoints**: `POST /join/request`, `POST /join/:id/approve`
- **Domain events** (optional but powerful): `JoinApproved`, `MessageReported`

This is where your “rules + flows” become real.

Step 5 — Vibe coding workflow that stays non-chaotic

Use a tight loop:

1. Pick **one domain** (ex: Channels + Messages)
2. Write **Rules Catalog rows** for only that domain
3. Write **2–3 flows** as state machines
4. Generate:
    - policy checks
    - endpoints
    - UI gating (disabled composer, hidden buttons)
5. Add **tests** for policy + flow transitions

6. Move to next domain

This prevents the “AI spaghetti app” problem.

The fastest “translation artifact” to create right now

Create a file called:

`interaction_contract.md`

With these sections (short, structured):

- Roles
- Entities (list)
- Actions (verbs list)
- Rules Catalog (table)
- Flows (state diagrams in bullet form)
- Non-functional rules (rate limits, logging, privacy)

Then every time you vibe code, you only work from that file.

Vibecoding Terminology

Below is the exact language that reliably pushes an agent past scaffolding.

The words that trigger logic (use these)

1) “Implement domain logic”

Tell it: *“Stop scaffolding. Implement the domain logic for X.”*

2) “Business rules” / “policy enforcement”

Tell it: *“Enforce business rules on the server (source of truth) and mirror them in the UI as gating.”*

3) “State machine / lifecycle”

Tell it: *“Model this as a state machine and implement state transitions + invalid transition errors.”*

4) “Command handlers”

Tell it: *“Implement command handlers for: create, update, publish, approve, revoke…”*
(Command handler = function that performs validation + state change + side effects.)

5) “Invariants”

Tell it: *“Define invariants that must always be true, and validate them.”*
Example: “A Visitor can never WRITE messages anywhere.”

6) “Side effects”

Tell it: *“Implement side effects: audit log, notifications, counters, fan-out events.”*

7) “Acceptance criteria”

Tell it: *“Implement these scenarios as tests; code is not complete without passing tests.”*

Why you keep getting scaffolding

Most prompts sound like:

> “Build the chat system / set up schemas / add routes”

That reads like “create the skeleton.”

To get logic, you must specify:

- **who** is acting (role)
- **what** action they attempt
- **on what resource**
- **under what conditions**
- **what must happen in DB**
- **what must happen in UI**
- **what errors must happen when denied**
- **how to test it**

That’s “behavior spec,” not “build a feature.”

The “Logic-First Prompt Pattern” (copy/paste)

Use this structure every time:

A) Domain + Goal

- “Domain: Channels + Messages”
- “Goal: enforce permissions + message lifecycle”

B) Rules (policy)

- List 8–15 rules as bullets (“Visitor can READ public channels; cannot WRITE anywhere”)

C) Flows (state transitions)

- “Message states: draft? sent, edited, deleted, reported”
- “Report flow: open → reviewed → actioned”

D) Server requirements

- “All enforcement server-side in a single `can()` policy module”
- “Endpoints must call policy before DB write”
- “Return structured errors: code + reason”

E) Client requirements

- “UI must hide/disable actions based on policy”
- “Show reason tooltips/toasts from server reason”

F) Tests

- “Write tests for 10 scenarios (allow + deny).”

If your prompt includes A–F, you get logic.

A concrete example prompt (Chat domain)

Here’s a prompt that forces domain logic instead of scaffolding:

```
Implement DOMAIN LOGIC (not scaffolding) for Channels + Messages.

You must implement:
1) Policy enforcement (server is source of truth):
- Roles: VISITOR, PASSPORT, CITIZEN, MODERATOR, ADMIN
- Rules:
  - VISITOR: can READ only channels where isPublic=true. cannot WRITE
anywhere.
  - PASSPORT/CITIZEN: can READ channels they are members of; can WRITE
in channels they can READ unless channel is locked.
```

  - MODERATOR: can READ/WRITE everywhere; can DELETE any message; can MUTE users.
  - ADMIN: full access + can change channel settings (lock/unlock, public/private).
  - Users can EDIT only their own messages within 15 minutes; after that read-only.
  - Any user can REPORT a message they can read; reporting creates a moderation queue item.
  - Blocked users cannot DM each other (deny both directions).

2) State transitions:
- Message lifecycle: created -> edited (optional) -> deleted (optional)
- Report lifecycle: open -> reviewed -> actioned (no action | delete msg | mute user | ban user)

3) Required implementation:
- Create `can(actor, action, resource, ctx)` module returning {allow, reason, constraints}
- Every endpoint must call `can()` BEFORE any DB write.
- Implement endpoints:
  - POST /channels/:id/messages (create)
  - PATCH /messages/:id (edit)
  - DELETE /messages/:id (delete)
  - POST /messages/:id/report
  - GET /moderation/reports (mod/admin)
  - POST /moderation/reports/:id/action (mod/admin)

4) Client gating:
- Disable composer for users who cannot WRITE (show reason)
- Hide edit/delete buttons unless allowed

5) Tests (required):
- Add tests for at least 12 scenarios covering allow+deny, including the 15-minute edit window.

Do not add new scaffolding or placeholder logic. Implement full behavior and return structured errors.

That prompt produces logic consistently.

Domain-logic “verbs” you should use per domain

Use these words depending on what you’re building:

- **Community**: join, request, approve, reject, kick, ban, mute, unmute
- **Channels**: create, lock, unlock, archive, makePublic, makePrivate
- **Chat**: send, edit, delete, react, mention, pin, report
- **Live**: start, end, join, leave, promoteSpeaker, demoteSpeaker, muteSpeaker
- **Marketplace**: list, purchase, refund, dispute, fulfill
- **Fintech-ish**: authorize, capture, transfer, settle, reconcile

Agents respond very well to “implement handlers for these commands.”

The one sentence that fixes your vibe coding

When you notice it drifting into schema/scaffolding, say:

“Stop scaffolding. Implement the domain logic + policy enforcement + state transitions + tests for (domain).”

Global Docs Templates

Global Docs

- **RPBS_Product.md**
- **REBS_Product.md**
- **Domain_Map.md**
- **Glossary.md**
- **Action_Vocabulary.md**
- **Reason_Codes.md**
- **ALRP.md** *(Agent Logic & Role Prompt)*
- **TIES.md** *(Implementation discipline / build rules)*
- **Definition_of_Done.md**
- **Prompt_Playbook.md**
- **Review_Checklists.md**
- **Domain_Build_Order.md**
- **Changelog.md**
- **README.md**

RPBS_Product.md

RPBS — {{APP_NAME}} (v{{VERSION}})

0) Authority
- RPBS is the top-level product truth.
- If anything conflicts, RPBS wins.
- “Truth for build” is what is later locked via ERC, but ERC must never contradict RPBS.

1) Product Summary (one screen)
- Product in one sentence:
- Who it is for:
- Main outcomes it creates (top 3):
- What it is NOT:
- Primary “north star” behavior:

2) Problem & Motivation
- Current problem:
- Why this matters now:
- What fails without this product:
- Existing alternatives and why they fail:

3) Target Users, Personas, and Roles
Personas
- Persona A:
- Persona B:

Roles (system roles used for access control)
List the roles the app recognizes:
- Role:
- Role:
- Role:

Role Powers Table (high-level capability matrix)
Capability	Role A	Role B	Role C	Admin/Mod
Sign up / authenticate				
View public content				
View restricted content				
Create content				
Edit own content				
Delete own content				
Interact (comment/chat/react)				
Invite/approve members				
Moderate content/users				
Manage configuration				

| Access admin tools | | | | |

4) Core Use Cases (Jobs To Be Done)
Write 5–15 JTBD statements:
- JTBD-001:
- JTBD-002:

5) Non-Negotiables (must always be true)
Behavior Non-Negotiables
- NNB-001:
- NNB-002:

Trust & Safety Non-Negotiables
- NNT-001:
- NNT-002:

Operational Non-Negotiables
- NNO-001:
- NNO-002:

6) Hard Rules Catalog (executable truth fuel)
Write 25–150 rules as short, testable statements. These are primary inputs for BELS + AccessControl + tests.

Identity & Access
- HR-IA-001:
- HR-IA-002:

Permissions / AccessControl
- HR-AC-001:
- HR-AC-002:

Content Visibility & Publishing
- HR-PUB-001:
- HR-PUB-002:

Social / Interaction (chat, comments, reactions, follows, etc.)
- HR-SOC-001:
- HR-SOC-002:

Marketplace / Payments (if applicable)
- HR-MKT-001:
- HR-MKT-002:

Trust & Safety / Moderation
- HR-TS-001:
- HR-TS-002:

Notifications & Activity
- HR-NOTIF-001:
- HR-NOTIF-002:

Operations Control Plane (feature flags, remote config, kill switches)
- HR-OPS-001:
- HR-OPS-002:

Compliance / Data Rights
- HR-CDR-001:
- HR-CDR-002:

7) Global State Model (standardize cross-domain lifecycles)
Define global states that multiple domains will reference so they don't get reinvented.

User Account Status (example)
- active
- locked
- disabled
- pending_verification

Enforcement Status (example)
- none
- muted
- suspended
- banned

Publish Status (example)
- draft
- in_review
- published
- archived

Verification / Trust Signals (example)
- unverified
- verified
- flagged

> Rule: Domains may extend states locally, but the global set must remain consistent.

8) Event Guarantees (cross-domain determinism)
List events that must always occur when certain actions happen.

Guarantee ID	When this happens	The system must emit	Consumers (examples)
EG-001	A user account is created	UserCreated	Analytics, Notifications
EG-002	A moderation action is taken	EnforcementActionRecorded	ActivityLog, Admin
EG-003	Content is published	ContentPublished	Search, Notifications
EG-004	A payment succeeds	PaymentSucceeded	Ledger, Notifications

9) Data Classification (security + compliance baseline)
Classification Levels
- Public: safe to display broadly
- Internal: non-public but not sensitive
- Sensitive: PII or private user content
- Restricted: financial, credentials, high-risk data

Handling Rules (minimum)
- Logging redaction rules:
- Admin visibility rules:
- Export/delete rules:
- Retention rules:

10) Onboarding Definition (activation truth)
Onboarding Success Criteria
- Minimum steps:
- Must-see screens/content:
- Required settings (if any):
- When privileges unlock:

Onboarding Failure / Recovery
- If user skips step X:
- If verification fails:
- If user is restricted (mute/suspend):

11) Product Definition of Done (RPBS-level)
This is the product-level “ready to ship onboarding” checklist.

- Accounts: sign up, sign in, sign out, password reset / recovery works
- Roles: role assignment works and changes reflect immediately
- Gating: restricted actions show reason + next step (never silent failure)
- Safety minimum: report/block exists (if social), enforcement is respected everywhere
- Data basics: basic export/delete pathway exists or explicitly documented as unavailable
- Support: user can reach help/contact and see basic troubleshooting

- Operations: maintenance/kill switch plan exists (even if simple)
- Observability: onboarding funnel is measurable (activation checkpoints)

12) System Limits & SLAs (behavior constraints)
Define practical constraints so build and tests are realistic.

- Latency targets (p95): reads __ms, writes __ms
- Availability target: __%
- Max payload size (uploads): __
- Max message length / post length: __
- Rate limits expectations (examples): login, signup, create content, chat send
- Concurrency assumptions (if realtime): __ active rooms, __ msgs/sec
- Offline tolerance: what must work offline vs not

13) Minimum Viable Safety (anti-abuse baseline)
List the minimum defenses required for launch (adapt to app type).

- Abuse reporting: report content/user available
- Blocking: block user (if social) with clear effect
- Throttling: basic spam/rate limit for high-risk actions
- Moderation: basic review/queue (if needed) OR clear enforcement workflow
- Audit: admin/mod actions are logged (who/what/when/why)
- Appeals/support path: basic “contact support” for enforcement issues

14) Permission Philosophy (global stance)
Define consistent behavior across the app.

- Default stance: allow-by-default OR deny-by-default (pick one)
- Hide vs disable vs gate: when each is used
- Client gating is UX only; server enforcement is required for security
- Deny experience must include: reason + next step

15) Data Residency / Export / Delete Promise (baseline)
Even if simple, define your promise so domains don’t contradict each other.

- Data export: available? what scope?
- Account deletion: available? hard delete vs anonymize?
- Timeline expectations: within __ days
- Exceptions: what cannot be deleted and why (legal/audit)

16) Canonical Identifiers (prevent entity confusion)
List IDs used across the system and what they represent.

- userId:

- orgId / workspaceId (if multi-tenant):
- deviceId:
- sessionId:
- contentId types (postId, messageId, etc.):
- paymentId / ledgerEntryId (if applicable):
- correlationId (for observability and audits):

17) Scope
In Scope
- …

Out of Scope
- …

18) Success Metrics
- Activation:
- Retention:
- Engagement:
- Conversion (if applicable):
- Safety / abuse:
- Reliability / uptime:
- Support load:

19) Product Principles (decision filters)
- Principle 1:
- Principle 2:
- Principle 3:

20) Feature Inventory (high-level)
- Feature area:
- Feature area:

21) Domain Inventory Reference
- Domain Map: {{LINK_DOMAIN_MAP}}
- Domain List: {{LINK_DOMAIN_LIST}}

22) Risks & Mitigations
Security Risks
- Risk:
- Mitigation:

Abuse Risks
- Risk:
- Mitigation:

Compliance / Data Risks
- Risk:
- Mitigation:

Product Risks
- Risk:
- Mitigation:

23) Release Constraints
- Platforms:
- Minimum OS versions:
- Accessibility baseline:
- Performance baseline:
- Offline tolerance expectations:
- Observability baseline:

24) Open Questions (product-level)
- Q-001:
- Q-002:

25) Appendices
- Links:
- Supporting notes:

REBS_Product.md

REBS — {{APP_NAME}} Engineering Blueprint (v{{VERSION}})

0) Authority
- REBS defines how engineering (and agents) are allowed to think and build.
- RPBS overrides REBS on product truth.
- ERC-locked docs are build-authoritative for a domain, but must not contradict RPBS/REBS.

1) Engineering Goals
- Primary build goal (production readiness):
- Non-goals (explicitly not optimizing for):
- Long-term maintainability goals:

2) System Architecture Principles (global)
Separation of Concerns
- UI renders state and triggers actions; it does not decide truth.
- Server enforces policy; client gating is UX only.
- Domain modules do not cross-write each other's storage.

System of Record Rule (Ownership)
- Every shared concept has exactly one owning domain (system of record).
- Non-owning domains may consult (read-only) or react via events.

Contract-Driven Boundaries
- Every cross-domain interaction must use:
  - a public function/contract OR
  - an event contract (published schema)
- No "reach into another domain's internals."

3) Deterministic Behavior Requirements
- No "mystery changes": all state transitions must be explainable.
- All writes are produced by explicit commands.
- All commands must validate inputs deterministically.
- Idempotency required for high-risk operations (payments, publishing, moderation, voting, onboarding steps).

4) Enforcement Model (Policy & Access Control)
Single Gate Requirement
- All permission checks must flow through a single mechanism:
  - `AccessControl.can(actor, action, resource, ctx)`
- Domain code must not implement custom permission logic except via AccessControl calls.

Evaluation Order (global)
1) Authentication / session validity
2) Account status (active/locked/disabled/etc.)

3) Enforcement status (mute/suspend/ban, etc.)
4) Resource existence + visibility rules
5) Role + conditions policy allow
6) Default deny

Deny Standard (mandatory)
Every deny must include:
- `reasonCode` (from Reason_Codes.md)
- user-facing message guidance (clear + actionable)
- optional `nextStep` hint (route/action)

5) Error Model (global)
Error Shape
Define a standard error envelope:
- code (ReasonCode or ErrorCode)
- message (user-safe)
- detail (optional, developer-safe)
- correlationId
- retryable (true/false)

Reason Codes vs Error Codes
- Reason Codes = user blocked by policy (authorization/eligibility)
- Error Codes = system failures (validation, timeouts, DB, etc.)

6) Data & Persistence Rules
- Canonical identifiers and their usage (userId, sessionId, etc.)
- Soft delete vs hard delete policy
- Data classification handling rules (Public/Internal/Sensitive/Restricted)
- Logging redaction requirements for Sensitive/Restricted fields
- Retention requirements (audit logs vs user content)

7) Observability & Audit Requirements
Logging
- Every request must produce a correlationId.
- Every domain command must log:
  - actorId
  - action
  - resourceId(s)
  - result (allow/deny)
  - reasonCode if denied

Audit Events (mandatory for certain categories)
Required audit categories:
- moderation/enforcement actions

- publishing lifecycle changes
- governance actions (proposal/vote/result)
- admin config changes
- security events (login, session revoke, etc.)

Metrics
- onboarding funnel metrics
- permission denials rate
- command success/fail rate
- latency p95 targets

8) Testing & Verification Discipline
Minimum Coverage
- Policy tests: allow + deny for every critical action
- State machine tests: valid + invalid transitions
- Integration tests for cross-domain event flows
- E2E tests for onboarding + one meaningful action

Test Source of Truth
- TESTPLAN scenarios MUST be derived from BELS acceptance scenarios.
- If a behavior is not in BELS/TESTPLAN, it is not considered “done.”

9) Roshi Doc Stack Requirements (global)
Each domain must have:
- DDES_<Domain>.md
- UX_Foundations_<Domain>.md
- UI_Constraints_<Domain>.md
- BELS_<Domain>.md
- DIM_<Domain>.md
- SCREENMAP_<Domain>.md
- TESTPLAN_<Domain>.md
- ERC_<Domain>_v#.md (required before build)
- SROL_<Domain>.md (post-build optimization review)

UPGRADE 1: Canonical Contracts (global API/event style)
10) Canonical Contract Standards

Command Handler Contract (writes)
All state change operations must map to a command handler with a consistent signature.

- Input:
  - actor (userId, roles, restrictions, sessionId)

  - action (string)
  - resource (id + type)
  - payload (validated)
  - ctx (correlationId, clientInfo, ip/device if available)
- Output:
  - ok (true/false)
  - result payload (success)
  - error envelope (failure)

Query Contract (reads)
Standardize:
- pagination (cursor or page/limit)
- filtering (typed filters)
- sorting (explicit)
- consistent response envelopes

Event Contract (pub/sub)
All events share a common envelope:
- eventName
- eventId
- occurredAt
- actorId (if applicable)
- resourceRef(s)
- correlationId
- payload (typed)
- idempotencyKey (if applicable)

UPGRADE 2: Policy DSL (how rules must be written)
11) Policy Rule Language (BELS standard)

Required Rule Fields
Every policy rule must specify:
- Action
- Allow conditions (boolean logic)
- Deny reasonCode
- Deny user message
- SourceRef

Allowed Condition Vocabulary (recommended predicates)
Standard predicates (extendable):
- isAuthenticated
- hasRole(role)

- hasAnyRole(roles[])
- isOwner(actor, resource)
- isPublic(resource)
- isVisibleTo(actor, resource)
- isMemberOf(actor, org/community/etc.)
- isVerified(actor)
- isRestricted(actor, restrictionType)
- withinRateLimit(actor, action)
- featureEnabled(flag)

Precedence Rules (global)
- Restrictions override roles unless explicitly stated.
- Resource visibility overrides “role allow.”
- Default deny if any required condition is UNKNOWN.

UPGRADE 3: State Machine Implementation Standard
12) State Machine Standard (lifecycles)

Requirements
- Every lifecycle entity must have:
  - explicit states
  - explicit transitions
  - guard conditions
  - deny reason codes for blocked transitions

Implementation Pattern
- Transition function:
  - validate current state
  - evaluate guard conditions (policy + invariants)
  - apply transition (write)
  - emit event(s)
  - write audit record if required

Illegal Transition Handling
- Must return:
  - error envelope with reason code (INVALID_TRANSITION or domain-specific)
  - correlationId
- Must not “force” state changes.

UPGRADE 4: Migration Playbook (existing apps)

13) Migration Discipline (prevent duplication/breakage)

Overlay-First Rule
- Add Roshi docs and generators without touching runtime code initially.

Shadow Mode (safe enforcement migration)
For policy refactors:
- Evaluate old decision + new decision
- Log differences (do not block yet)
- Switch to enforce only after review

De-duplication Rule
- Do not remove old checks until:
  - tests exist for behavior
  - AccessControl is enforcing the same behavior
  - shadow-mode diff is clean

Backward Compatibility
- Avoid renaming routes/params unless explicitly locked by ERC + migration plan.

---

14) Build Gating (anti-drift rules)
ERC Gate
- No code changes for a domain are allowed unless that domain has an ERC lock.
- If meaning changes, bump ERC version (v2, v3…)—never mutate locked meaning silently.

SourceRef Rule (mandatory)
- Every policy rule and lifecycle transition in BELS must include SourceRef.

No Invention Rule (mandatory)
- Agents must not guess.
- If not defined by sources, mark as UNKNOWN and add to Open Questions.

Draft vs Truth Rule
- Draft docs are not authoritative.
- Truth = ERC-locked docs.

15) Agent Operating Rules (high-level)
- Agents must operate in this order:
  1) Read RPBS/REBS/Domain Map
  2) Draft domain docs (DDES/UTCS/BELS/DIM/SCREENMAP/TESTPLAN)
  3) Surface UNKNOWNs
  4) Wait for lock (ERC)

5) Build strictly from locked inputs
- Agents must output diffs and list what they changed.

16) Definition of Done (engineering)
A feature/domain is “done” only if:
- Policy enforced server-side (AccessControl)
- State transitions enforced (no illegal transitions)
- Tests cover allow/deny + transitions
- Reason codes are consistent and user messaging is defined
- Audit/notifications behavior defined where relevant
- Observability includes correlation IDs and required logs

17) Security Baseline (global)
- Auth/session standards:
- Rate limiting standards:
- Secret handling:
- Data storage rules (secure storage on device):
- Threats explicitly in scope (spam, brute force, privilege escalation):

18) Open Questions (REBS-level)
- Q-001:
- Q-002:

19) Change Control
- When RPBS changes: what must be re-locked
- When REBS changes: which domains need ERC bumps
- Template versioning rules

Domain_Map.md

Domain Map — {{APP_NAME}} (v{{VERSION}})

0) Authority
- Domain Map defines bounded contexts, ownership, and cross-domain interaction rules.
- RPBS defines product truth; REBS defines engineering governance.
- If a domain boundary is unclear, it must be resolved here before code changes.

1) Domain Registry (Canonical List)
Every domain must have:
- Name
- Slug
- Prefix (for action vocabulary)
- Type (Business / Control Plane / Cross-Cutting)
- System-of-record scope (what it owns)

#	Domain Name	Slug	Prefix	Type	Owns (System of Record)	Primary Surfaces	Notes
1							

> Rule: A concept can only have ONE owning domain.

2) Global Boundary Rules (Non-Negotiable)
2.1 No Cross-Domain DB Writes
- A domain may only write its own tables/storage.
- Cross-domain effects must happen through:
  - events OR
  - explicit public service calls (contracted interfaces).

2.2 Read-Only Consultation Only
- A domain can query another domain only through:
  - read models, or
  - public query contracts.
- No direct coupling to internal schema.

2.3 Orchestration Rule (Multi-step flows)
If a flow touches multiple domains:
- Use an orchestrator layer (service/workflow) that:
  - calls domain contracts in order
  - handles retries and partial failures
  - emits final workflow events
- Orchestrator must not become a “god domain.”

2.4 Default Deny for Undefined Ownership
- If ownership is unclear, the action is blocked until ownership is assigned.

3) Cross-Domain Interaction Patterns (Standard)
Approved patterns (no improvisation):

Pattern A — Event-Driven Side Effects (preferred)
- Owning domain writes + emits event
- Subscriber domains react and write their own data

Pattern B — Synchronous Contract Call (allowed)
- Owning domain exposes a stable contract (function/API)
- Caller uses it as a black box
- No schema coupling

Pattern C — Shared Read Model (allowed with care)
- Owner publishes read model
- Consumers cannot modify it

UPGRADE 1: Concept Ownership Ledger (granular, executable)
4) Concept Ownership Ledger (Canonical Truth Table)

This is stricter than the Domain Registry. It lists every cross-domain concept and nails:
- owner (system of record)
- what “writes” mean (actions)
- what read surfaces exist
- what events MUST be emitted

Concept ID	Concept	Owner Domain	Write Actions (canonical)	Read Surfaces (contracts)	Required Events (guarantees)	Non-Owner Allowed	Non-Owner Forbidden
C-USER	User identity/profile					consult/read	direct writes
C-ROLE	Roles & membership					consult/read	invent role logic
C-RESTRICT	Restrictions/enforcement					consult/read	apply enforcement
C-PUBLISH	Publish state					consult/read	change publish state
C-NOTIF	Notifications					request via event	send directly
C-AUDIT	Audit log					append via contract	silent admin actions
C-PAY	Payments					consult/read	money math outside owner
C-LEDGER	Ledger entries					consult/read	edit ledger

> Rule: If a concept exists in the system and crosses domains, it MUST be registered here.

---

5) Global Roles & Trust States
Roles (canonical)
- {{ROLE_1}}
- {{ROLE_2}}
- {{ROLE_3}}

Global Account States
- active / locked / disabled / pending_verification (example)

Global Enforcement States
- none / muted / suspended / banned (example)

> Rule: Domains may not invent new global states. Local states must be scoped to the domain.

6) Global Action Namespace Rules
Action Format
`<domain>.<resource>.<verb>`

Naming Constraints
- verbs are lowercase imperative: create, update, delete, publish, approve, revoke, ban, mute
- resources are singular nouns: user, message, proposal
- domain prefix must match registry prefix

Examples
- `community.message.create`
- `identity.session.revoke`
- `trustsafety.enforcement.apply`

7) Global Reason Code Rules
- Reason codes are canonical and live in: {{LINK_REASON_CODES}}
- Deny reasons must be stable identifiers (not free text).
- User-facing messages must map to reason codes.

8) Domain Dependency Graph (High-Level)
Prevent circularity and “everything depends on everything.”

Domain	Hard Dependencies (must exist)	Soft Dependencies (optional)	Emits Events To	Consumes Events From	Circular Risk?
					YES/NO

> Guidance: AccessControl, Identity, TrustSafety, Ops are typically upstream.

9) Cross-Domain Use Cases (Reality Check / DIM Anchors)

List 10–30 real cross-domain flows. These become DIM anchors and orchestrator candidates.

Use Case	Primary Domain	Supporting Domains	Primary Actions	Required Events	Orchestrator Needed?
Onboarding new user	Identity	AccessControl, Notifications, ActivityLog, Ops			YES/NO

10) Data Classification Ownership
Tie classification to owner domains so privacy doesn't drift.

Data Type	Classification	Owner Domain	Redaction Rules	Retention Rules	Access Notes

---

UPGRADE 2: Overlap Resolution Protocol (how to handle domain collisions)
11) Overlap Resolution Protocol (Mandatory)

When two domains overlap, follow this protocol—no exceptions.

11.1 Identify the collision type
Choose one:
- Type A: Shared UI surface, single truth (most common)
- Type B: Shared entity referenced everywhere (must assign owner)
- Type C: Multi-step workflow touching many domains (needs orchestrator)
- Type D: Cross-cutting enforcement (TrustSafety/AccessControl)

11.2 Resolve ownership (system-of-record)
Answer in order:
1) Which domain owns the entity/state?
2) Which domain owns the policies/permissions?
3) Which domain owns the lifecycle transitions?
4) Which domains only consume via events?

If any answer is unknown → register an Open Question and block build changes for affected areas.

11.3 Decide integration pattern
Use only approved patterns:
- Event-driven side effects (preferred)
- Synchronous contract call (allowed)
- Shared read model (careful)

11.4 Update required artifacts (in this order)
1) Update Domain Map registry + Concept Ownership Ledger
2) Update DDES (both domains) ownership sections
3) Update DIM (interaction tables + event catalog)
4) Update BELS (policy rules + transitions)
5) If build-authoritative meaning changed → bump ERC version(s)

11.5 Create a new domain vs subsystem rules
Create a new domain if:
- it owns a system-of-record concept AND
- it has unique policies + lifecycle + workflows

Create a subsystem if:
- it is complex but still owned by a single domain and doesn't deserve separate ownership.

11.6 Prevent duplication (enforcement rule)
- Policies must not be duplicated across domains.
- AccessControl is the single gate for permission truth.
- TrustSafety is the single owner of enforcement truth.
- If a duplicate is found, it must be migrated into the owning domain and removed after tests exist.

12) Open Questions (Map-level)
- Q-001: Any concept without a clear owner?
- Q-002: Any domain that should be split/merged?
- Q-003: Any orchestrator needed for onboarding?
- Q-004: Any duplicate policy logic existing in multiple domains?

13) Change Control
- Any change to ownership requires:
  - updating this map
  - updating affected domain DDES/DIM
  - potentially bumping ERC versions

Glossary.md

Glossary — {{APP_NAME}} (v{{VERSION}})

0) Authority
- This Glossary defines canonical meaning for terms used across RPBS/REBS/Domain Map and all domain docs.
- If two docs use the same term differently, the Glossary wins.
- Terms must be stable; changes require changelog entry and may require ERC bumps.

1) How to Use This Glossary (rules)
- Every domain doc must use glossary terms as written.
- If a new term is introduced, it must be added here.
- Avoid synonyms that create drift (pick one canonical term).
- If a UI label differs from a canonical term, record the alias in “Aliases.”

2) Global Roles (Canonical)
Define every role used by AccessControl and UI gating.

Role	Description	Primary Capabilities	Restrictions/Notes	SourceRef
{{ROLE_1}}				RPBS §__
{{ROLE_2}}				RPBS §__

3) Global States (Canonical)
These are cross-domain and must match RPBS/Domain Map.

Account Status
State	Meaning	Who sets it	Where enforced	Notes	SourceRef
active			AccessControl		RPBS §__

Enforcement Status
State	Meaning	Who sets it	Where enforced	Notes	SourceRef
muted		TrustSafety	AccessControl + Domain policies		RPBS §__

Publish Status (if applicable)
State	Meaning	Who sets it	Where enforced	Notes	SourceRef
draft		Publishing Control Plane	UI + server checks		RPBS §__

4) Canonical Entities (Cross-Domain Objects)
Define shared “nouns” that appear in multiple domains.

| Term | Definition | Owner Domain | Primary IDs | Sensitive? | Notes | SourceRef |

|---|---|---|---|---|---|---|
User		Identity & Access	userId	Yes		Domain Map §4
Session		Identity & Access	sessionId	Yes		REBS §10
Restriction		TrustSafety	restrictionId	Yes		Domain Map §4

5) Canonical Actions & Verbs (Language Rules)
Canonical Verbs (use consistently)
- create / update / delete
- publish / unpublish
- approve / reject
- invite / join / leave
- revoke / suspend / ban / mute / unmute
- report / block / unblock
- enroll / verify / confirm
- start / stop (for realtime sessions)

Verb Rules
- Use “revoke” for access removal (sessions/tokens/roles)
- Use “ban/suspend/mute” only for enforcement outcomes (TrustSafety)
- Use “publish” only for publish state transitions (Publishing Control Plane owner)

6) Permission Language (Canonical)
These terms prevent confusion in AccessControl and BELS.

Term	Meaning	Where used	Notes
allow	policy permits action	AccessControl + BELS	always logged
deny	policy blocks action	AccessControl + BELS	must include reasonCode
gate	UI prevents user attempt	UI Constraints	UX only, never security
enforce	server blocks action	API/services	security truth

7) Observability Terms (Canonical)
Term	Meaning	Required?	Notes
correlationId	ties logs/events/errors together	Yes	always returned in errors
audit event	immutable record of sensitive action	For admin/mod/publish/payment	stored in AuditLog
metric	numeric measurement for health	Yes	funnels + denials + latency

8) Data Classification Terms (Canonical)
Classification	Meaning	Examples	Handling Rules
Public	safe to display	public posts, landing copy	can log safely
Internal	non-public, low risk	feature flags, config	avoid in client logs

| Sensitive | PII/private content | email, DMs | redact in logs |
| Restricted | financial/credentials | tokens, payment refs | never log raw |

9) UI Copy Dictionary (Optional but powerful)
Define canonical UI phrases so the app stays consistent.

Concept	Canonical Phrase	Alternatives (avoid)	Notes
Restricted action	“You don’t have access to do that yet.”	“Not allowed.”	include next step

10) Aliases (If the product uses multiple labels)
Record allowed UI/display synonyms mapped to canonical terms.

Canonical Term	Allowed Alias	Where used	Notes
Member	“Citizen”	UI	role label only

11) Open Questions (Glossary-level)
- Q-001: Any conflicting terms across domains?
- Q-002: Any term used in UI that doesn’t exist here?

12) Change Control
- Any change to roles/states/entities must:
  - update this Glossary
  - update Domain Map Concept Ownership Ledger if affected
  - potentially bump ERC versions for impacted domains
  - be recorded in Changelog

Action_Vocabulary.md

Action Vocabulary — {{APP_NAME}} (v{{VERSION}})

0) Authority
- This is the canonical registry of all actions the system recognizes.
- AccessControl policies and BELS rules must reference actions from this registry.
- If an action is not listed here, it is not allowed to be implemented as “real behavior” (add it first).
- Actions must be stable identifiers; changes require Changelog entry and may require ERC bumps.

1) Action Naming Standard (Global)
Format
`<domain>.<resource>.<verb>`

Rules
- `domain` must match Domain Map prefix/slug.
- `resource` is a singular noun: user, session, message, proposal.
- `verb` is a canonical verb (see Glossary verb list).
- Avoid synonyms: choose one verb and keep it.

Verb Set (Canonical)
- create, update, delete
- view, list
- publish, unpublish
- approve, reject
- invite, join, leave
- revoke
- report, block, unblock
- mute, unmute, suspend, unsuspend, ban, unban (TrustSafety-owned)
- start, stop (realtime/live)
- enroll, verify, confirm (onboarding/identity)
- export, erase (data rights)

2) Action Ownership Rules
- Every action has exactly one owning domain.
- Owning domain defines:
  - write behavior (command handler)
  - state transitions (if any)
  - events emitted
- Other domains can only:
  - consult (read-only)
  - react (events)

3) Global Action Registry (Canonical List)

Action	Owning Domain	Category	Description	Default Sensitivity	Requires Auth?	Requires Audit?	SourceRef
				Public/Internal/Sensitive/Restricted	YES/NO	YES/NO	RPBS §__

Category (recommended set)
- Identity
- AccessControl
- Community
- Content
- TrustSafety
- Notifications
- Admin
- Payments
- Governance
- Operations
- DataRights
- Analytics

4) Required Action Metadata (per action)
For each action you register, specify:

4.1 Resource Model
- resourceType:
- primaryIdField:
- secondaryIds (optional):
- ownership rules (who is “owner”):

4.2 Risk Tier
Pick one:
- Low (safe read/list)
- Medium (user write, reversible)
- High (irreversible/destructive, security-sensitive)
- Critical (payments, enforcement, admin config)

4.3 Idempotency Requirement
- required? yes/no
- idempotency key pattern (if yes):

4.4 Audit Requirement
- audit required? yes/no
- audit category (admin/mod/publish/payment/security):
- minimum audit fields:
  - actorId

  - action
  - resourceRef(s)
  - result
  - reasonCode (if deny)
  - correlationId

5) Action Deprecation Rules
If an action must change:
- Never silently rename in place.
- Use one of:
  - Deprecate old action, introduce new action
  - Keep action name, change internals only if behavior remains consistent with locked docs
- Document the change in Changelog.
- If used in locked ERC domains, bump ERC versions.

6) Action Coverage Expectations
- Every action must be covered in:
  - BELS (policy + command handler + scenarios) for its owning domain
  - TESTPLAN (at least 1 allow and 1 deny scenario if gated)
- High/Critical actions must have:
  - explicit deny reason codes
  - audit requirement
  - abuse/rate-limit expectations

7) Cross-References
- Domain Map: {{LINK_DOMAIN_MAP}}
- Reason Codes: {{LINK_REASON_CODES}}
- Glossary: {{LINK_GLOSSARY}}

8) Open Questions (Action Vocabulary)
- Q-001: Any actions implemented in code but missing here?
- Q-002: Any actions listed here without owning domain docs yet?
- Q-003: Any verb collisions (same meaning, different verb)?

9) Change Control
- Any addition/removal/rename requires:
  - update this registry
  - update owning domain BELS
  - update AccessControl policies
  - update tests
  - record in Changelog

Reason_Codes.md

Reason Codes — {{APP_NAME}} (v{{VERSION}})

0) Authority
- Reason Codes are canonical identifiers for policy denials and eligibility blocks.
- Every deny in AccessControl and BELS must return a reasonCode from this registry.
- UI must map denies to user-facing messages using these codes.
- Codes must be stable; changes require Changelog entry and may require ERC bumps.

1) Reason Code vs Error Code (strict distinction)
- ReasonCode = user is blocked by policy/eligibility (authorization / restrictions / visibility).
- ErrorCode = system failure (validation, timeout, DB error, crash).

> Rule: A deny must never return a generic error if the real issue is policy.

2) Naming Standard (Global)
Format
Use one consistent pattern:
- Option A (recommended): `{{DOMAIN_PREFIX}}_{{REASON}}`
  - Example: `COMMUNITY_NOT_A_MEMBER`
- Option B (generic): `FORBIDDEN_{{REASON}}`
  - Example: `FORBIDDEN_NOT_A_MEMBER`

Pick one and stick to it.

Rules
- UPPER_SNAKE_CASE
- No spaces, no punctuation
- Meaning must be obvious without reading docs
- Must not encode UI language (messages live separately)

3) Deny Envelope Requirements (mandatory)
Every deny must include:
- `reasonCode` (from this doc)
- `messageKey` OR canonical “User Message” guidance
- optional `nextStep` hint (actionable)
- `correlationId`

4) Reason Code Taxonomy (Categories)
A) Authentication & Session
- NOT_AUTHENTICATED
- SESSION_EXPIRED
- MFA_REQUIRED
- EMAIL_NOT_VERIFIED

B) Account Status / Eligibility
- ACCOUNT_LOCKED
- ACCOUNT_DISABLED
- ACCOUNT_NOT_ELIGIBLE
- ONBOARDING_INCOMPLETE

C) Role / Permission Deny
- INSUFFICIENT_ROLE
- MISSING_PERMISSION
- NOT_OWNER
- NOT_A_MEMBER

D) Visibility / Publishing / Access Scope
- RESOURCE_NOT_FOUND_OR_NOT_VISIBLE
- PRIVATE_RESOURCE
- NOT_PUBLISHED
- REGION_RESTRICTED (optional)

E) Trust & Safety Enforcement
- USER_MUTED
- USER_SUSPENDED
- USER_BANNED
- CONTENT_RESTRICTED
- ACTION_BLOCKED_BY_SAFETY_POLICY

F) Rate Limiting & Abuse Controls
- RATE_LIMITED
- SPAM_DETECTED
- TOO_MANY_ATTEMPTS
- COOLDOWN_ACTIVE

G) Operational / Feature Flags
- FEATURE_DISABLED
- MAINTENANCE_MODE
- APP_VERSION_UNSUPPORTED

H) Compliance / Data Rights
- EXPORT_NOT_AVAILABLE
- DELETE_NOT_AVAILABLE
- LEGAL_HOLD_ACTIVE

> Note: These are category headers; the actual canonical list is below.

5) Canonical Reason Code Registry (the real list)

Every row must be usable by:
- AccessControl (server)
- UI (message + next step)
- tests (assert exact code)

Reason Code	Category	When Used (plain language)	User Message (guidance)	Next Step (optional)	HTTP Status (guidance)	Audit Required?	SourceRef
NOT_AUTHENTICATED	Auth	user has no valid session	“Please sign in to continue.”	go_to_login	401	NO	REBS §4
INSUFFICIENT_ROLE	Role	user role lacks required power	“You don’t have access to do that.”	request_access	403	NO	RPBS §__
USER_MUTED	TrustSafety	user is muted and action requires posting	“You can’t post right now.”	view_restrictions	403	YES	RPBS §__
RATE_LIMITED	Abuse	action exceeded allowed rate	“Slow down and try again.”	wait_seconds	429	NO	REBS §13
FEATURE_DISABLED	Ops	feature flag is off	“This feature is currently unavailable.”	none	403/503	YES (if admin action)	RPBS §__

6) Message Guidelines (universal)
User Message Rules
- clear, short, and actionable
- never reveals sensitive enforcement internals
- never leaks whether a resource exists if visibility is restricted
  - use “not found or not visible” pattern

“Next Step” Rules
- must be actionable (button/link/flow)
- examples: `go_to_login`, `complete_onboarding`, `verify_email`, `contact_support`, `view_rules`

Security Messaging Rules
For restricted visibility:
- prefer `RESOURCE_NOT_FOUND_OR_NOT_VISIBLE`
- do not say “this exists but you can’t see it”

7) Mapping Rules (ReasonCode → UI Treatment)
Define global treatment so UI is consistent.

Reason Code	UI Treatment	Show CTA?	CTA Label	Notes
NOT_AUTHENTICATED	redirect/inline gate	YES	“Sign in”	
ONBOARDING_INCOMPLETE	inline gate	YES	“Finish setup”	
USER_BANNED	blocked screen	YES	“Contact support”	keep vague

| RATE_LIMITED | toast/banner | NO/YES | “Try again” | show cooldown |

8) Testing Requirements (mandatory)
- Every gated action must have at least one deny test asserting:
  - exact reasonCode
  - correct HTTP status guidance
- High/Critical actions must assert reasonCode + audit behavior (if required).

9) Open Questions (Reason Codes)
- Q-001: Any denies currently returning generic errors instead of reason codes?
- Q-002: Any duplicate codes with same meaning?
- Q-003: Any missing “visibility-safe” deny patterns?

10) Change Control
- Changing a reason code identifier is a breaking change.
- If a code is replaced:
  - deprecate old code
  - add new code
  - update UI mapping + tests
  - record in Changelog
  - bump ERC for affected domains

ALRP.md

ALRP — Agent Logic & Role Prompt ({{APP_NAME}} v{{VERSION}})

0) Authority
- This ALRP governs any agent operating in this repo.
- RPBS is product truth. REBS is engineering governance. Domain Map is ownership truth.
- ERC-locked domain docs are build-authoritative for that domain.
- If instruction conflicts: RPBS > REBS > Domain Map > ERC > ALRP > everything else.

1) Agent Role
You are an implementation agent. Your job is to:
- translate locked documentation into working code
- enforce policy and state machines
- produce tests that prove behavior
- avoid drifting from authoritative truth

You are NOT allowed to:
- invent missing requirements
- change product scope
- refactor unrelated code
- create new domains without updating Domain Map + registry

2) Inputs (Hierarchy)
The agent must read and obey in this order:
1) RPBS_Product.md (product truth)
2) REBS_Product.md (engineering truth)
3) Domain_Map.md (ownership + boundaries)
4) Glossary.md, Action_Vocabulary.md, Reason_Codes.md (global canon)
5) Domain doc pack (DDES, UX, UI, BELS, DIM, SCREENMAP, TESTPLAN)
6) ERC lock for the domain being built (required to build)

3) Operating Modes (Mandatory)
Mode A — Doc Mode (no code)
Allowed when:
- domain docs are missing or incomplete
Outputs:
- drafts of missing domain docs
- lists of UNKNOWNs
Forbidden:
- modifying runtime code

Mode B — Build Mode (code allowed)
Allowed ONLY when:
- the target domain has an ERC lock file
- ERC references the correct input docs

Outputs:
- code changes limited to target domain and approved integration points
- tests matching TESTPLAN scenarios
- a diff summary of changes

Mode C — Migration Mode (existing app overlay)
Allowed when:
- integrating Roshi behavior into an existing system
Must use:
- shadow mode evaluation (old vs new decision logging) for policy changes
- incremental de-duplication after tests prove parity

4) Non-Negotiable Build Order (per domain)
When building a domain, do these steps in order:

1) Confirm domain ownership (Domain Map)
2) Confirm action vocabulary exists (Action_Vocabulary)
3) Confirm reason codes exist (Reason_Codes)
4) Confirm BELS is complete (policy + handlers + state machines + scenarios)
5) Confirm TESTPLAN covers BELS scenarios
6) Implement AccessControl enforcement (server-side)
7) Implement command handlers + state transitions
8) Emit required events (DIM)
9) Implement screens/components (SCREENMAP) if requested
10) Write tests (policy + transitions + key flows)
11) Provide “Release Receipt” (what changed + what is verified)

5) “No Scaffolding-Only” Rule (Critical)
If asked to build a domain, you must produce:
- policy rules (allow/deny) enforced server-side
- state machine enforcement for lifecycle entities
- tests proving allow/deny + transitions

If you only produce schema/scaffolding without the above, the work is incomplete.

6) No-Invention Rule (Critical)
If anything is undefined:
- mark it as UNKNOWN
- add it to Open Questions in the relevant doc
- stop and request human decision OR proceed only with explicitly permitted defaults.

7) Allowed Defaults (only these)
Agents may assume ONLY these defaults if not specified:
- Default deny for undefined permissions

- Visibility-safe deny: RESOURCE_NOT_FOUND_OR_NOT_VISIBLE
- Correlation ID required in errors/logs
- Client gating is UX-only; server enforcement required

Any other assumption requires an explicit SourceRef.

8) Drift Prevention Rules
- Never rename actions/roles/reason codes without updating registries + changelog.
- Never implement cross-domain DB writes.
- Never duplicate AccessControl logic in domain modules.
- Never implement TrustSafety enforcement outside the TrustSafety domain.

9) Output Format (Mandatory)
Every response must include:
1) What mode you are operating in (Doc / Build / Migration)
2) Inputs consulted (links/paths)
3) Changes made (files + summary)
4) Verification (tests run, scenarios covered)
5) Open Questions (UNKNOWNs discovered)

10) Stop Conditions (Mandatory)
The agent must STOP (no code) when:
- ERC is missing for a domain build request
- Domain ownership is unclear
- Required registries (actions/reasons) are missing
- a requirement is UNKNOWN and not covered by Allowed Defaults

11) Safety & Security Requirements
- Never log secrets or restricted data
- Enforce rate limits for abuse-prone actions
- Always return reason codes for denies
- Always include correlationId in failures

12) Review Checklist (Agent Self-Check)
Before finalizing any build:
- Are policies enforced server-side through AccessC

TIES.md

TIES — Technical Implementation & Execution Standards ({{APP_NAME}} v{{VERSION}})

0) Authority
- TIES governs HOW code is implemented.
- RPBS governs WHAT must be true; REBS governs engineering truth; Domain Map governs boundaries.
- ERC-locked domain docs are build-authoritative.
- If TIES conflicts with REBS/RPBS, TIES must yield.

1) Architecture Style (global)
Functional Core / Imperative Shell
- Business logic lives in pure-ish functions (functional core).
- IO (DB, network, time, filesystem) lives in the shell.
- Shell calls core; core never calls shell.

Deterministic Data Flow
- All writes occur through command handlers.
- Reads occur through query handlers / repositories.
- Side effects are explicit and event-driven where possible.

Domain Isolation
- Domain code cannot directly mutate other domain storage.
- Cross-domain collaboration via contracts/events only.

2) Code Organization Standard
(Keep universal; adapt paths to your repo)

Per Domain Structure (recommended)
- `domain/{{domainSlug}}/`
  - `actions/` (action constants + mapping)
  - `policies/` (domain-specific policy helpers, calls AccessControl)
  - `commands/` (write handlers)
  - `queries/` (read handlers)
  - `state/` (state machine + transitions)
  - `events/` (emitters + consumers)
  - `models/` (types, DTOs, validation)
  - `tests/` (unit + integration)

> Rule: No “misc” buckets. If it doesn’t fit, the boundary is unclear.

3) Contracts & Interfaces
Command Handler Contract (writes)
All command handlers must:
- validate input

- check policy via AccessControl
- enforce state transitions
- write only owned entities
- emit required events
- return standardized envelope (success/error)

Query Contract (reads)
All queries must:
- enforce visibility rules (if required)
- return stable response shapes
- support pagination/filtering when relevant

Event Contract (pub/sub)
Events must use the canonical event envelope from REBS.

4) Validation & Security Discipline
Input Validation
- Validate at the boundary (API handler / command handler).
- Use typed schemas (e.g., zod) for payload validation.
- Reject unknown fields for critical actions (payments/admin/moderation).

Authorization
- Never rely on UI for security.
- All gated actions must call AccessControl.
- Denies must include reasonCode + correlationId.

Rate Limiting (minimum)
- Identify high-risk actions (auth, posting, reporting, moderation, payments).
- Apply rate limits and return RATE_LIMITED reason codes.

5) State Machine Discipline
- Lifecycle entities must not be mutated outside a transition function.
- Every transition must:
  - validate current state
  - apply guard conditions
  - emit events
  - write audit logs if required
- Illegal transitions must return INVALID_TRANSITION (or domain reason).

6) Error Handling & Recovery
Error Envelope (required)
- code
- message (user-safe)
- correlationId

- retryable flag
- optional detail (dev-safe)

Idempotency
Required for:
- payments
- publishing
- moderation
- voting
- onboarding steps that can be retried

Retry Semantics
- safe retries for idempotent operations
- do not retry non-idempotent writes without keys

7) Observability & Auditing
Correlation IDs
- Every request/action must carry correlationId end-to-end.
- Include in logs, errors, audit events.

Audit Logging
Required for:
- admin config changes
- moderation actions
- publishing transitions
- security events (login/session revoke)
- payments/ledger changes

8) Testing Discipline
Required Test Types
- Unit: command handler logic, policy decisions, state transitions
- Integration: DB writes + events emitted + consumers react
- E2E (minimum): onboarding + one meaningful action flow

Test Source of Truth
- TESTPLAN scenario IDs must map to actual tests.
- BELS scenarios are the authoritative behavior specification.

Regression Protection
- Any bug fix must add a test that would have caught it.
- No deletion of a test without replacing coverage.

9) Dependency & Version Discipline
- Pin critical dependencies.

- Avoid introducing new libraries without documenting why.
- Keep domain dependencies minimal.

10) Performance & Resource Discipline
- Avoid N+1 queries in high-volume paths.
- Use pagination for lists.
- Use caching only with explicit invalidation strategy.
- Prefer incremental loading in UI.

11) Security & Privacy Baseline (implementation)
- Never log secrets or restricted content.
- Redact sensitive fields in logs.
- Use secure storage mechanisms for tokens on device (mobile).
- Protect against common abuse patterns (spam, brute force, privilege escalation).

12) Pull Request / Change Discipline (even if solo)
Every change must include:
- what domain it belongs to
- which actions it affects
- which tests prove it
- what events/audits are emitted
- any migration notes

13) Definition of Done (TIES-level)
A change is “done” when:
- policies enforced server-side
- state transitions enforced
- reason codes returned on denies
- required events emitted
- audit events emitted for sensitive actions
- tests added and passing
- no cross-domain DB writes introduced

Definition_of_Done.md

Definition of Done — {{APP_NAME}} (v{{VERSION}})

0) Authority
- This DoD is the minimum acceptance bar for work to be considered complete.
- RPBS defines product truth; REBS defines engineering governance; TIES defines implementation discipline.
- ERC-locked domain docs define domain truth for builds.

1) Levels of Done (Use the right one)
L0 — Draft (not shippable)
- doc exists but is incomplete
- UNKNOWNs remain unresolved for core flows
- no tests

L1 — Buildable (internal)
- compiles/runs
- core screens/handlers exist
- basic happy path works

L2 — Production-Ready (required for release)
- policies enforced server-side
- tests prove behavior
- observability + audits present
- failure recovery defined

> Default requirement: L2 for anything user-facing or security-sensitive.

2) Product DoD (user experience)
Onboarding / Entry
- User can create an account (or authenticate)
- User can recover access (reset/verify where applicable)
- Onboarding definition in RPBS is measurable and implemented
- User reaches at least one meaningful “success moment”

Permission UX
- Any denied action shows:
  - reason (mapped from Reason Codes)
  - next step (actionable)
- No silent failures
- No confusing “it just doesn’t work” behaviors

Trust & Safety Minimum (if app is social / user-generated content)
- Reporting exists (content/user)
- Blocking exists (if relevant)

- Enforcement effects are consistent (muted means no posting, etc.)
- Moderator/admin actions have audit trail

3) Engineering DoD (logic + governance)
Policy Enforcement (mandatory)
- All gated actions call `AccessControl.can(...)`
- Deny returns:
  - reasonCode (from registry)
  - correlationId
- No policy duplication across domains

State Machine Enforcement (mandatory where applicable)
- Lifecycle entities cannot be mutated outside transitions
- Invalid transitions return deterministic reason codes
- Transitions emit required events

Cross-Domain Discipline
- No cross-domain DB writes
- All cross-domain effects via:
  - events OR
  - public contracts

4) Testing DoD (must prove truth)
Minimum Test Set
- For every critical action:
  - at least 1 allow test
  - at least 1 deny test asserting reasonCode
- For every lifecycle entity:
  - valid transition tests
  - invalid transition tests
- For one major flow:
  - integration test (DB + events)
- For onboarding:
  - at least 1 E2E test (or equivalent integration suite)

Test Traceability
- Tests must reference:
  - BELS scenario IDs
  - TESTPLAN sections
- If behavior changes, tests must change with it.

5) Observability DoD (production visibility)
Correlation IDs
- Generated or propagated for every request/action

- Present in:
  - server logs
  - error envelopes
  - audit events

Logging (minimum)
- actorId (or anonymous)
- action name
- resource ids
- result allow/deny
- reasonCode for denies

Metrics (minimum)
- onboarding funnel checkpoints
- action success/fail rate
- deny rate by reason code
- latency p95 for key endpoints

6) Security & Privacy DoD
- No secrets or restricted data in logs
- Sensitive fields are redacted
- Rate limiting exists for high-risk actions (auth, posting, reporting, admin)
- Secure storage used for tokens (mobile)
- Basic abuse protections exist per RPBS

7) Documentation DoD (Roshi compliance)
Domain Docs
A domain is “doc-complete” when:
- DDES exists and ownership is clear
- UX Foundations exists
- UI Constraints exists
- BELS exists (policies + handlers + state machines + scenarios)
- DIM exists (events & interactions)
- SCREENMAP exists
- TESTPLAN exists

ERC Lock
A domain is “build-authorized” when:
- ERC exists and references the correct inputs
- Frozen decisions are listed
- Acceptance gate defined

8) Release Receipt (required output for any build)
Every build must produce a short “receipt”:

- mode (Doc / Build / Migration)
- inputs consulted (paths)
- files changed
- tests added/updated
- scenarios covered
- open questions remaining
- known risks

9) Exceptions (rare)
If something cannot meet L2:
- record exception
- explain why
- define a follow-up task
- define risk and mitigation

10) Change Control
- Any change that alters meaning requires:
  - updating docs
  - updating tests
  - potentially bumping ERC
  - recording in Changelog

Prompt_Playbook.md

Prompt Playbook — {{APP_NAME}} (v{{VERSION}})

0) Authority
- This playbook defines approved prompt patterns for agents working in this repo.
- Agents must follow ALRP, REBS, TIES, and the relevant ERC locks.
- If a prompt conflicts with those, the prompt is invalid.

1) Prompt Contract (mandatory format)
Every agent prompt must include:

1) Objective (what outcome you want)
2) Mode (Doc / Build / Migration)
3) Scope (domain(s), files, what NOT to touch)
4) Inputs (paths to RPBS/REBS/Domain Map + relevant domain docs)
5) Deliverables (what must be produced)
6) Verification (tests/scenarios that must pass)
7) Stop Conditions (when the agent must stop and ask)

---

2) Global “Hard Constraints” Snippets (copy/paste)
A) Docs-only constraint
- Do not modify runtime code.
- Only create/update docs under: `docs/roshi_v2/...`

B) Domain-only constraint
- Only touch files inside: `domain/{{domainSlug}}/` and approved integration points.
- Do not refactor unrelated areas.

C) No-invention constraint
- If requirement is missing, mark UNKNOWN and add Open Questions.
- Do not guess.

D) Logic-not-scaffolding constraint
- Must implement policy enforcement + state transitions + tests.
- Scaffolding-only output is invalid.

E) ERC gate constraint
- Do not code unless ERC exists for the domain.

---

3) Starter Prompts (Universal)

3.1 Roshi Workspace Setup (docs-only)
Use when: initializing Roshi v2 folders + templates.
- Objective:
- Mode: Doc
- Scope:
- Inputs:
- Deliverables:
- Verification:
- Stop conditions:

3.2 Roshi Template Lock (docs-only)
Use when: hardening templates before generation.

Prompt:
- Objective: Fortify templates to meet RPBS/REBS/TIES requirements.
- Mode: Doc
- Scope: `docs/roshi_v2/01_templates/*`
- Deliverables:
  - add missing sections for policy/state/tests
  - create TEMPLATE_VERSION + rules
- Stop conditions: any contradiction discovered

3.3 Domain Doc Pack Generation (`roshi:gen`)
Use when: generating empty domain doc packs from templates.

Prompt:
- Objective: Generate Roshi doc packs for these domains: {{DOMAIN_LIST}}
- Mode: Doc
- Scope: only create files under `docs/roshi_v2/02_domains/*`
- Inputs: templates + registries
- Deliverables:
  - create doc files with placeholders filled (domain name/slug/prefix)
  - no invented content
- Verification: file list output

3.4 Domain Doc Seeding (`roshi:seed`)
Use when: populating docs from existing RPBS or legacy notes.

Prompt:
- Objective: Seed {{DOMAIN_NAME}} docs using SourceRefs from RPBS and legacy docs.
- Mode: Doc
- Scope: only these docs: DDES/UX/UI/BELS/DIM/SCREENMAP/TESTPLAN
- Constraints:
  - only copy/transform from sources

  - mark UNKNOWN where missing
- Deliverables:
  - populated tables
  - open questions
- Verification: list of SourceRefs used

3.5 Build Domain (requires ERC)
Use when: implementing real logic.

Prompt:
- Objective: Implement {{DOMAIN_NAME}} per ERC lock.
- Mode: Build
- Scope: domain folder + approved integration points only
- Inputs: ERC + domain docs
- Deliverables:
  - AccessControl allow/deny enforcement
  - command handlers + state transitions
  - events emitted
  - tests mapping TESTPLAN scenarios
- Verification:
  - run tests
  - include a Release Receipt
- Stop conditions:
  - missing ERC
  - UNKNOWN requirement not covered by allowed defaults

3.6 Migration Shadow Mode (existing app)
Use when: moving policy checks into AccessControl safely.

Prompt:
- Objective: Migrate {{FEATURE}} to AccessControl with shadow mode diff logging.
- Mode: Migration
- Deliverables:
  - oldCan/newCan diff logs
  - tests proving parity
  - staged switch plan
- Stop conditions: parity not achieved

---

4) Debug Prompts (Universal)
4.1 “Why is the agent only generating scaffolding?”
Prompt:
- Objective: Identify why logic is missing and generate missing BELS/TestPlan items.

- Mode: Doc
- Scope: only docs, no code
- Deliverables:
  - missing actions
  - missing policy rules
  - missing state machine definitions
  - missing acceptance scenarios
  - updated open questions list

4.2 “Generate logic for this domain”
Prompt:
- Objective: Convert domain interactions into BELS policy + state machines.
- Mode: Doc
- Scope: BELS + DIM only
- Deliverables:
  - Rule table + transitions + scenarios
  - reason codes required

---

5) Prompt Quality Checklist
Before using a prompt:
- Did you specify Mode?
- Did you include inputs (paths)?
- Did you constrain scope?
- Did you demand logic + tests (if Build mode)?
- Did you define stop conditions?

6) Change Control
- Add new prompt patterns only if they produce repeatable results.
- Record major changes in Changelog.

Review_Checklists.md

Review Checklists — {{APP_NAME}} (v{{VERSION}})

0) Authority
- These checklists define the minimum review gates for docs and code changes.
- RPBS/REBS/Domain Map + ERC locks are the sources of truth.
- If a checklist fails, the work is not approved.

A) Global Doc Review Checklist (RPBS/REBS/Domain Map/Registries)

A1) RPBS Review
- [ ] Roles are listed and consistent with Glossary
- [ ] Role Powers Table is filled (not blank)
- [ ] Non-Negotiables are testable statements (not vibes)
- [ ] Hard Rules Catalog has enough rules (25–150) and is explicit
- [ ] Global State Model is defined (account/enforcement/publish/verification)
- [ ] Event Guarantees are listed for major actions
- [ ] Data Classification rules exist (logging/admin/export/delete)
- [ ] Onboarding definition is explicit and measurable
- [ ] Product Definition of Done is clear and release-aligned
- [ ] System limits/SLAs are realistic
- [ ] Minimum viable safety baseline is defined
- [ ] Permission philosophy is stated (hide/disable/gate, default deny stance)
- [ ] Canonical identifiers list exists
- [ ] Open Questions list is present and honest

A2) REBS Review
- [ ] AccessControl “single gate” requirement is explicit
- [ ] Policy evaluation order is defined
- [ ] Deny standard includes reasonCode + correlationId + nextStep guidance
- [ ] Canonical contracts defined (commands/queries/events)
- [ ] Policy DSL defined (predicates + precedence)
- [ ] State machine standard defined (guards + illegal transitions)
- [ ] Observability requirements are explicit (logs/metrics/audit categories)
- [ ] Testing discipline aligns with BELS/TESTPLAN
- [ ] ERC gate and no-invention rule are explicit
- [ ] Migration discipline exists (shadow mode + dedupe rules)

A3) Domain Map Review
- [ ] Domain Registry is complete (name/slug/prefix/type/owns)
- [ ] Concept Ownership Ledger is filled for cross-domain concepts
- [ ] No cross-domain DB writes rule is explicit
- [ ] Interaction patterns are defined and limited

- [ ] Dependency graph has no obvious circularities
- [ ] Cross-domain use cases list exists (10–30)
- [ ] Overlap resolution protocol exists and is usable
- [ ] Open Questions include unresolved ownership conflicts

A4) Registries Review (Glossary / Actions / Reason Codes)
- [ ] Glossary includes roles, states, key nouns, and canonical verbs
- [ ] Action vocabulary has owning domain per action
- [ ] Action names follow the global format
- [ ] Reason codes registry includes deny taxonomy and UI mapping rules
- [ ] “Visibility-safe deny” pattern exists (not found or not visible)

B) Domain Doc Pack Review Checklist
(DDES/UX/UI/BELS/DIM/SCREENMAP/TESTPLAN/ERC)

B1) DDES Review
- [ ] Domain purpose is one sentence and clear
- [ ] System-of-record ownership is explicit (OWNS vs DOES NOT OWN)
- [ ] Core entities table is populated
- [ ] Invariants list exists
- [ ] Key flows list exists (business flows, not UI)
- [ ] Cross-domain dependencies are explicit
- [ ] Failure/recovery semantics exist
- [ ] Abuse/safety considerations exist
- [ ] Observability requirements exist
- [ ] Open Questions list exists

B2) UX Foundations Review
- [ ] User mental model is described
- [ ] UX laws are explicit (clarity/feedback/recovery)
- [ ] JTBD set exists and maps to screens
- [ ] Key moments defined (first/success/failure/recovery)
- [ ] Trust & safety UX is addressed
- [ ] Accessibility rules included
- [ ] Copy/tone rules included
- [ ] Open Questions list exists

B3) UI Constraints Review
- [ ] Entry points + nav patterns defined
- [ ] Screen types required are listed
- [ ] Interaction constraints defined (forms/destructive/rate-limited)
- [ ] Permission UX standard defined (reason + next step)

- [ ] Loading/empty/error patterns defined
- [ ] Component rules align with design system
- [ ] Open Questions list exists

B4) BELS Review (the critical one)
- [ ] Action vocabulary table is populated (not placeholder)
- [ ] Command handlers table is populated (inputs/validations/writes/events)
- [ ] Policy rules table exists with:
  - [ ] RuleIDs
  - [ ] allow conditions
  - [ ] deny reasonCode
  - [ ] user message guidance
  - [ ] SourceRef
- [ ] Policy evaluation order matches REBS
- [ ] State machines are defined for lifecycle entities
- [ ] Error/recovery model table exists
- [ ] Acceptance scenarios exist (>=10) and include allow/deny/edge
- [ ] Open Questions list exists (UNKNOWNs are visible)

B5) DIM Review
- [ ] Interaction matrix exists
- [ ] Event catalog exists (emitted and consumed)
- [ ] Consumers and failure behavior are specified
- [ ] No cross-domain DB writes implied

B6) SCREENMAP Review
- [ ] Screen inventory is complete
- [ ] Every screen maps to BELS actions
- [ ] Forms + validation defined
- [ ] Empty/loading/error states defined
- [ ] Deep links/entry points listed

B7) TESTPLAN Review
- [ ] Coverage targets defined (unit/integration/e2e)
- [ ] Scenario set references BELS scenario IDs
- [ ] Permission tests assert reason codes
- [ ] State machine tests assert invalid transitions
- [ ] Abuse/risk tests exist for high-risk actions
- [ ] Observability verification included

B8) ERC Review (build gate)
- [ ] ERC lock statement exists
- [ ] All referenced inputs exist and are correct
- [ ] Frozen decisions are listed

- [ ] Build boundaries are explicit
- [ ] Acceptance gate defined
- [ ] Version bumped if meaning changed

C) Code Review Checklist (Build Mode)

C1) Scope & Drift
- [ ] Changes limited to target domain + approved integration points
- [ ] No cross-domain DB writes introduced
- [ ] No action/role/reason code rename without registry update + changelog
- [ ] No duplicated policy logic outside AccessControl

C2) Policy & State Correctness
- [ ] Every gated write calls AccessControl
- [ ] Denies return reasonCode + correlationId
- [ ] Visibility-safe denies used where needed
- [ ] State transitions enforced and illegal transitions blocked

C3) Events, Audit, Observability
- [ ] Required events emitted
- [ ] Admin/mod/publish/payment actions generate audit entries
- [ ] Logs include correlationId and action metadata
- [ ] Sensitive data redacted

C4) Testing
- [ ] Tests added/updated
- [ ] Tests cover allow + deny (reasonCode asserted)
- [ ] Tests cover state transitions
- [ ] Integration/E2E coverage meets TESTPLAN requirements
- [ ] Release Receipt included

D) Security Review Checklist (High-Risk Work)

Trigger this checklist if work involves:
- auth/session/mfa
- admin tools
- moderation/enforcement
- payments/ledger
- data export/delete
- realtime chat/live

- [ ] Rate limits in place for abuse-prone endpoints
- [ ] No secrets logged
- [ ] PII redaction verified
- [ ] Authorization checks server-side
- [ ] Replay/idempotency strategy for critical actions
- [ ] Audit trail present for sensitive operations
- [ ] Error messages don’t leak existence/visibility

E) Release Review Checklist (Pre-Production)

- [ ] RPBS Product DoD satisfied for release scope
- [ ] Critical domains have ERC locks
- [ ] No unresolved ownership conflicts (Domain Map)
- [ ] Onboarding funnel measurable
- [ ] Crash reporting/metrics configured (as applicable)
- [ ] Support pathway exists
- [ ] Known risks documented and accepted
- [ ] Version recorded in Changelog

Domain_Build_Order.md

Domain Build Order — {{APP_NAME}} (v{{VERSION}})

0) Authority
- This document defines the approved sequencing and prerequisites for domain work.
- It must align with RPBS/REBS/Domain Map.
- If dependencies change, update this doc before starting the next build.

1) Build Order Philosophy (universal)
- Build upstream truth before downstream features.
- Build enforcement + observability before growth features.
- Build one “vertical slice” to production readiness before expanding breadth.
- No domain enters Build Mode without:
  - complete doc pack AND
  - ERC lock.

2) Domain Readiness States (not time-based)
State S0 — Unscoped
- Domain exists in name only.

State S1 — Defined
- Domain Map entry exists (ownership + boundaries).

State S2 — Documented
- DDES/UX/UI/BELS/DIM/SCREENMAP/TESTPLAN exist (drafts allowed).

State S3 — Locked
- ERC exists and freezes meaning.

State S4 — Built
- Code implemented per ERC + tests pass.

State S5 — Hardened
- Abuse cases + observability + audits verified; regressions prevented.

3) Build Tracks (universal lanes)
These tracks help you decide what to build next without dates.

Track A — Core Access & Enforcement (Upstream)
Domains typically include:
- Identity & Access
- AccessControl
- TrustSafety
- Ops Control Plane (feature flags/config)
- Audit/Activity (if separate)

Track B — Core Product Experience
Domains that create the “main thing” users do:
- Community / Content / Marketplace / Fintech / Streaming / Enterprise tools
(Depends on the product)

Track C — Growth & Reach
- Notifications
- Search/Discovery
- Invites/Referrals
- Analytics

Track D — Operations & Admin
- Admin Console / Control Panel
- Moderation tooling
- Support tooling

> Rule: Track A prerequisites must exist before Track B can ship safely.

4) Prerequisite Matrix (Dependency Truth)
Fill this once and it becomes your deterministic build guide.

Domain	Requires (Hard)	Benefits From (Soft)	Blocks (Downstream)	Notes
Identity	—	Ops	AccessControl, Community, Payments	
AccessControl	Identity	Reason Codes, Actions	All gated domains	
TrustSafety	Identity, AccessControl	Audit	Community, Content	
Ops	Identity	Audit	Feature rollout safety	

5) Build Order Queue (Canonical)
This is the actual prioritized list. No dates—just order + prerequisites.

Order	Domain	Track	Readiness State (S0–S5)	Entry Criteria Met?	Exit Criteria (DoD)	Notes
1		Track A/B/C/D	S0	YES/NO	S4 (Built)	

6) Entry Criteria (to start Domain Docs)
You can begin drafting docs for a domain when:
- Domain exists in Domain Map registry
- Ownership is clear (system-of-record defined)
- Actions can be named (Action Vocabulary prefix exists)

7) Entry Criteria (to lock ERC)

You can lock a domain when:
- DDES complete (ownership + flows + invariants)
- BELS complete (policy + state machines + scenarios)
- TESTPLAN complete (scenarios mapped)
- UNKNOWNs resolved or explicitly deferred

8) Entry Criteria (to start Build Mode)
You can start code build only when:
- ERC exists (S3 Locked)
- Required upstream dependencies are at least S3
- Registries are updated (actions + reason codes)

9) Exit Criteria (S4 Built)
A domain is “built” when:
- Policies enforced server-side via AccessControl
- State transitions enforced
- Required events emitted (DIM)
- Tests pass and map to TESTPLAN
- Release Receipt written

10) Exit Criteria (S5 Hardened)
A domain is “hardened” when:
- Abuse/rate-limit tests exist for high-risk actions
- Audit trail verified for sensitive actions
- Observability verified (correlationId, metrics)
- Regression suite covers critical flows

11) How to Choose the Next Domain (Decision Procedure)
When choosing what to build next:
1) pick domains that unblock the most downstream domains
2) prefer domains with clear ownership and minimal UNKNOWNs
3) prefer completing S5 for one vertical slice before expanding breadth
4) never build “growth” before enforcement

12) Open Questions (Build Order)
- Q-001: Which domain is the primary “product experience” domain?
- Q-002: Any dependency conflicts in the matrix?
- Q-003: Any domain should be split into subsystem(s)?

13) Change Control
- Any reorder requires:
  - update to the Prerequisite Matrix
  - update to the Queue table
  - record in Changelog

Changelog.md

Changelog — {{APP_NAME}}

0) Authority
- This changelog records meaningful changes to:
  - product truth (RPBS)
  - engineering governance (REBS/TIES/ALRP)
  - domain boundaries (Domain Map)
  - registries (Glossary/Actions/Reason Codes)
  - ERC locks (domain truth)
- Any change that affects behavior must be recorded here.
- “Silent changes” are not allowed.

1) Versioning Rules
Doc System Version
- Use semantic versioning for the doc system:
  - MAJOR: breaking meaning changes (actions renamed, role model changes, domain boundary shifts)
  - MINOR: additive changes (new domain, new action, new reason code)
  - PATCH: clarifications (wording improvements, examples)

Entry Requirements
Each entry must include:
- date
- version
- type (MAJOR/MINOR/PATCH)
- scope (global or specific domain)
- what changed
- why it changed
- impacts (what must be updated)
- references (links to docs/erc)

2) Current Version
- Current doc system version: {{DOC_SYSTEM_VERSION}}

---

3) Entries

{{YYYY-MM-DD}} — v{{X.Y.Z}} ({{MAJOR|MINOR|PATCH}})
Scope
- Global / Domain: {{DOMAIN_NAME if applicable}}

Change Summary
- {{one line}}

What Changed
- RPBS:
  - …
- REBS:
  - …
- Domain Map:
  - …
- Registries:
  - Glossary:
  - Action Vocabulary:
  - Reason Codes:
- Domain Docs:
  - …
- ERC Locks:
  - …

Why
- …

Impact / Required Follow-ups
- [ ] Update tests for: …
- [ ] Bump ERC for: …
- [ ] Update UI copy mapping for reason codes: …
- [ ] Run migration steps: …

Risk / Mitigation
- Risk:
- Mitigation:

References
- Links/paths:
  - `docs/...`
  - `execution/ERC_...`

README.md

{{APP_NAME}} — Roshi Doc System (v{{VERSION}})

0) What This Is
This repository uses the Roshi documentation system to:
- define product truth (RPBS)
- define engineering governance (REBS)
- define domain boundaries (Domain Map)
- define canonical vocabulary (Glossary / Actions / Reason Codes)
- define per-domain behavior truth (BELS + TESTPLAN)
- lock meaning before building (ERC)
- implement with discipline (TIES)

1) Source of Truth Order (Read This First)
If anything conflicts, resolve using this hierarchy:
1) RPBS_Product.md
2) REBS_Product.md
3) Domain_Map.md
4) Registries (Glossary, Action Vocabulary, Reason Codes)
5) Domain docs (DDES, UX, UI, BELS, DIM, SCREENMAP, TESTPLAN)
6) ERC locks (build-authoritative for that domain)
7) TIES + ALRP (how to build)

2) Repo Doc Layout (Recommended)

```
/docs/roshi_v2/
 /00_registry/
 Domain_Map.md
 Glossary.md
 Action_Vocabulary.md
 Reason_Codes.md
 /01_global/
 RPBS_Product.md
 REBS_Product.md
 ALRP.md
 TIES.md
 Definition_of_Done.md
 Prompt_Playbook.md
 Review_Checklists.md
 Domain_Build_Order.md
 Changelog.md
 /02_domains/
 /{{domainSlug}}/
 DDES_{{Domain}}.md
```

UX_Foundations_{{Domain}}.md
UI_Constraints_{{Domain}}.md
BELS_{{Domain}}.md
DIM_{{Domain}}.md
SCREENMAP_{{Domain}}.md
TESTPLAN_{{Domain}}.md
/03_execution/
ERC_{{Domain}}_v1.md

> You can change paths, but keep the structure.

3) How Work Happens (The Only Valid Flow)
Doc Flow
RPBS → REBS → Domain Map → Registries → Domain Docs → ERC Lock

Build Flow
ERC Lock → Build (policy + state + events + tests) → Release Receipt

Optimize Flow
Post-build SROL reviews → refactor safely (with tests)

4) Operating Modes
Mode A — Doc Mode (no code)
- Use when docs are missing or evolving.
- Output: drafts + UNKNOWNs.

Mode B — Build Mode (code allowed)
- Allowed only when ERC exists for the domain.
- Must produce: logic + policy + tests.

Mode C — Migration Mode (existing app)
- Use overlay-first + shadow mode diffs for safety.

5) “No Scaffolding Only” Rule
If you build a domain, you must produce:
- server-side policy enforcement (AccessControl)
- state transition enforcement
- tests mapping TESTPLAN scenarios

If not, the work is incomplete.

6) Where To Start (Practical)
1) Confirm RPBS + REBS are accurate

2) Lock Domain Map + registries
3) Pick first domain in Domain_Build_Order
4) Draft domain doc pack
5) Lock ERC
6) Build with tests

7) Release Receipt Template (required)
Every build must include:
- Mode:
- Inputs consulted:
- Files changed:
- Tests added/updated:
- Scenarios covered:
- Open questions:
- Risks:

8) Common Failure Modes (and fixes)
- “Agent only scaffolds” → BELS/TESTPLAN missing or ERC missing
- “Domain overlap” → fix Domain Map ownership + overlap protocol
- “Permission bugs” → add reason codes + deny tests
- “Breakage during migration” → use shadow mode + parity tests

9) Quick Links
- RPBS: `docs/roshi_v2/01_global/RPBS_Product.md`
- REBS: `docs/roshi_v2/01_global/REBS_Product.md`
- Domain Map: `docs/roshi_v2/00_registry/Domain_Map.md`
- Actions: `docs/roshi_v2/00_registry/Action_Vocabulary.md`
- Reason Codes: `docs/roshi_v2/00_registry/Reason_Codes.md`
- Domain Build Order: `docs/roshi_v2/01_global/Domain_Build_Order.md`
- Execution Locks (ERCs): `docs/roshi_v2/03_execution/`

Per-Domain Doc Pack templates

Per-Domain Doc Pack (repeat for each domain)

- **DDES_<Domain>.md**
- **UX_Foundations_<Domain>.md**
- **UI_Constraints_<Domain>.md**
- **BELS_<Domain>.md**
- **DIM_<Domain>.md**
- **SCREENMAP_<Domain>.md**
- **TESTPLAN_<Domain>.md**
- **ERC_<Domain>_v#.md**
- **SROL_<Domain>.md** *(post-build optimization review)*_

DDES_<Domain>.md

DDES — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- This document defines the domain's structure and truth at the "what exists" level.
- RPBS/REBS/Domain Map/Registries override this doc.
- This doc must not define UI layouts (that is SCREENMAP/UI Constraints).
- This doc must not define policy truth in full (that is BELS), but it MUST define invariants and boundaries.

1) Domain Summary (one paragraph)
- Domain purpose:
- Primary user outcome this domain enables:
- What this domain is NOT responsible for:

2) Ownership & Boundaries (System of Record)
Owns (System of Record)
List concepts/entities this domain owns (writes truth for):
- OWNS-001:
- OWNS-002:

Does NOT Own
List concepts/entities this domain may reference but must never write:
- NOTOWN-001:
- NOTOWN-002:

Upstream Dependencies
Domains that must exist first:
- DEP-HARD-001:
- DEP-HARD-002:

Downstream Consumers
Domains that depend on this domain:
- DEP-DOWN-001:
- DEP-DOWN-002:

Cross-Domain Interaction Pattern (choose)
- [ ] Event-driven side effects (preferred)
- [ ] Synchronous contract calls
- [ ] Shared read model (careful)

3) Canonical Entities (Domain Nouns)
Define the domain's primary entities.

| Entity | Description | Primary ID | Owner? | Lifecycle? | Sensitive? | Notes |

|---|---|---|---|---|---|---|
| | | | YES/NO | YES/NO | YES/NO | |

Entity Notes
For each lifecycle entity:
- Current states (name only):
- State owner (this domain):
- Related events (name only):

4) Invariants (must always be true)
These are non-negotiable truths used by BELS and tests.

Invariant ID	Statement (testable)	Applies To	Failure Behavior	SourceRef
INV-001			deny with reasonCode	RPBS §__ / Domain Map §__

Examples:
- “A message cannot be edited after it is archived.”
- “A resource marked private is never returned to non-members.”

5) Domain Flows (business flows, not UI)
List the core flows this domain supports. Keep them narrative and structural.

Flow ID	Flow Name	Trigger	Steps (high level)	Writes	Emits Events	Cross-Domain Touchpoints	Notes
FL-001							

6) Commands & Queries (what the domain exposes)
This is the contract surface the rest of the system can rely on.

6.1 Commands (Writes)
Command	Action Vocabulary	Description	Inputs	Writes	Emits Events	Idempotent?	Risk Tier	SourceRef
						YES/NO	Low/Med/High/Critical	

6.2 Queries (Reads)
Query	Action Vocabulary	Description	Inputs	Returns	Visibility Rules?	Pagination?	SourceRef
					YES/NO	YES/NO	

7) Lifecycle Models (if applicable)

For each lifecycle entity, define states and transition names (not the full policy; BELS will enforce guards).

Entity	States	Allowed Transitions (names only)	Terminal States	Notes

8) Data Classification & Privacy (domain-specific)
Tie to the global classification system.

Data Field / Concept	Classification	Storage Location	Logging Rules	Admin Visibility	Retention Notes

9) Abuse / Misuse Cases (domain-specific)
List how this domain could be abused and what mitigations are expected.

Abuse Case ID	Abuse Scenario	Likely Impact	Expected Mitigation	Where Enforced	SourceRef
AB-001			rate limits / throttles	AccessControl/BELS	RPBS §__

10) Failure & Recovery Semantics
Define what happens when things fail.

Failure Case	When It Happens	User Experience	System Behavior	Retryable?	Notes
				YES/NO	

11) Observability Requirements (domain-specific)
Signal	What to record	Why	Where	Notes
Logs	actorId, action, resourceId, result	debugging	API layer	
Metrics	success/fail, denies by reason	health	metrics	
Audits	admin/mod sensitive actions	compliance	audit log	

12) Open Questions (DDES-level)
- Q-001:
- Q-002:

13) Change Control
- Any change that affects:
  - ownership

  - invariants
  - contract surface (commands/queries)
  - lifecycle states
requires:
- Domain Map update if ownership shifts
- BELS + TESTPLAN updates
- potential ERC bump if locked

UX_Foundations_<Domain>.md

UX Foundations — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- UX Foundations defines the experience truth for this domain.
- It must align with RPBS product principles, glossary terms, and global permission philosophy.
- It must not define implementation details (TIES) or policy enforcement (BELS).
- If UX conflicts with RPBS non-negotiables or safety rules, RPBS wins.

1) Domain Mental Model (how users think about it)
Describe the user's "story" and how they perceive the domain.

- What users believe this domain is:
- What it actually is:
- Common misconceptions to prevent:
- What success feels like (in plain language):
- What failure feels like (and what we must avoid):

2) Primary Jobs To Be Done (JTBD)
List 5–15 JTBD statements (domain-specific).

JTBD ID	Statement	Priority	Notes
JTBD-001			

3) User Journey Moments (required)
Define the major moments that must be supported.

First-Time Moment
- First-time goal:
- First-time obstacles:
- First-time "aha" moment:
- Required guidance/help:
- Required guardrails:

Routine Moment
- Most common repeat behavior:
- Expected speed/friction:
- What must be one-tap / low-friction:

Failure Moment
- Most common failure:
- What the user should see:
- Recovery path:
- What must NEVER happen (rage triggers):

Restricted/Denied Moment (permission UX)
This must follow global “deny standard.”
- Common deny situations:
- What the message must communicate:
- Required next steps:
- What must not be disclosed (security/visibility):

4) Trust, Safety, and Social Friction (domain lens)
Define the domain’s safety experience rules.

- User protections needed:
- What “unsafe” looks like in this domain:
- Reporting expectations:
- Blocking expectations (if relevant):
- Enforcement UX expectations (mute/suspend/ban):
- Harassment/spam friction level (low/medium/high):

5) Interaction Laws (domain-specific)
Write 8–15 laws as testable UX rules.

UX Law ID	Law (testable)	Why	Applies To	Notes
UX-001	The user must always see confirmation after a destructive action.	trust	delete flows	

Examples:
- “Every send/post action must show immediate feedback (sent/sending/failed).”
- “Users must be able to undo accidental destructive actions when possible.”
- “Denied actions must show reason + next step, never silent.”

6) Information Architecture Principles (domain-specific)
- What users expect to find first:
- What belongs 1 tap away:
- What should never be buried:
- What can be advanced/hidden:

7) Content Rules (if domain displays user/system content)
- What content types exist:
- Ordering rules (newest, pinned, relevance):
- What “featured/pinned” means:
- Visibility rules the user should understand:

8) Feedback & Status Language (user-facing)

Define consistent language for this domain.

Status Types
- loading:
- empty:
- success:
- partial success:
- error:
- restricted/deny:

Copy Style Rules
- Tone:
- Reading level:
- Avoid phrases:
- Required phrases (if any):

9) Accessibility & Inclusion (domain-specific)
- Minimum requirements (screen reader, contrast, touch targets):
- Keyboard navigation (if web):
- Captions/transcripts (if media):
- Motion sensitivity rules:

10) Performance Expectations (user-perceived)
- What must feel instant:
- What can take time with progress:
- What should be deferred/backgrounded:
- What must never block primary interaction:

11) Open Questions (UX-level)
- Q-001:
- Q-002:

12) Change Control
- Any change to UX laws or primary journey moments must:
  - update SCREENMAP + UI Constraints
  - update TESTPLAN scenarios where relevant
  - bump ERC if the domain is locked and meaning changes

UI_Constraints_<Domain>.md

UI Constraints — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- UI Constraints defines structural UI rules for this domain.
- Must align with UX Foundations and global navigation/permission philosophy.
- Must not define business logic or policy enforcement (BELS).
- If any UI rule conflicts with RPBS safety/non-negotiables, RPBS wins.

1) Entry Points (how users arrive here)
List all valid entry points.

Entry Point ID	From Where	Trigger	Preconditions	Expected Landing Screen	Notes
EP-001					

2) Navigation Model (allowed structures only)
Allowed Navigation Types (choose)
- [ ] Tab entry
- [ ] Stack entry
- [ ] Modal
- [ ] Deep link
- [ ] Notification entry
- [ ] Inline panel (within a screen)

Navigation Constraints (hard rules)
- Max depth allowed:
- Back behavior rules:
- Modal usage rules:
- Deep link rules:
- “Return to context” rules:

3) Screen Grouping Rules
Define which screens belong together and how they are grouped.

Group	Description	Screens Included	Max Screens	Notes
G-001				

4) Screen Types Allowed (domain-specific)
Choose which screen types are allowed and the constraints on each.

Screen Type	Allowed?	When Used	Forbidden Uses	Notes
List/Feed	YES/NO			

Detail/View	YES/NO			
Create Form	YES/NO			
Edit Form	YES/NO			
Review/Confirm	YES/NO			
Settings/Config	YES/NO			
Admin Tools	YES/NO			

5) Interaction Pattern Constraints
Forms
- Validation timing: inline / on-submit / mixed
- Required field markings:
- Error message standard:
- Disabled submit standard:
- Save states: saving/saved/failed

Destructive Actions
- Must require confirm? YES/NO
- Undo availability? YES/NO
- Copy guidelines for warnings:

Realtime / Streaming / Live (if applicable)
- Presence indicators rules:
- Connection state rules:
- Retry / reconnect rules:
- Latency display rules (if any):

6) Permission UX Standard (mandatory)
This section must align with Reason Codes and global deny rules.

Hide vs Disable vs Gate (choose rules)
- Hide when:
- Disable when:
- Gate with explanation when:

Denied Action Response
If server denies an action:
- UI must display:
  - user-safe message mapped from reasonCode
  - nextStep (if provided)
  - non-leaking behavior for visibility
- UI must NOT:
  - assume resource exists
  - show internal enforcement details
  - silently fail

Required Deny Components
- Deny Banner / Sheet:
- Inline Gate Component:
- “Request access” CTA (if applicable):

7) Loading / Empty / Error Standard
Loading
- Skeleton vs spinner rule:
- Max time before showing retry:
- Progressive loading rules:

Empty States
- Must explain why empty:
- Must show next action:
- Must show permissions if gated:

Error States
- Must show correlationId? YES/NO (recommended for support)
- Retry allowed when:
- Escalation path:

8) Content & Moderation UI (if user content exists)
- Report entry points:
- Block entry points:
- “Hidden content” display rules:
- “Removed by moderation” display rules:

9) Accessibility Constraints (UI-specific)
- Touch target minimum:
- Contrast requirements:
- Screen reader labeling rules:
- Captions/transcripts requirement (if media):

10) Performance & Rendering Constraints
- Max items rendered in lists:
- Pagination/infinite scroll rules:
- Image/video loading rules:
- Caching rules (UI layer):
- Avoid expensive re-renders rules:

11) UI Contract with Domain Logic
List what the UI is allowed to assume and what it must ask the server.

UI May Assume (UX-only)
- local optimistic states (temporary)
- cached data freshness windows

UI Must Ask Server For (truth)
- permissions (AccessControl allow/deny)
- enforcement effects
- visibility rules
- publish status truth

12) Open Questions (UI-level)
- Q-001:
- Q-002:

13) Change Control
- Any change to entry points, navigation constraints, or deny UX must:
  - update SCREENMAP
  - update TESTPLAN
  - bump ERC if locked meaning changes

BELS_<Domain>.md

BELS — Behavior & Enforcement Logic Spec — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- BELS is the authoritative behavior specification for this domain (pre-ERC).
- It converts RPBS/REBS/Domain Map + DDES into enforceable logic.
- When ERC locks this domain, the ERC references this BELS as build-authoritative.
- If anything conflicts: RPBS/REBS/Domain Map/Registries override BELS.

1) Domain Context Snapshot
- Domain purpose (1 sentence):
- System-of-record owned concepts:
- Non-owned concepts referenced:
- Primary user outcomes:
- Risk tier of domain (Low/Med/High/Critical):

2) Action Inventory (must match Action_Vocabulary)
List every domain action this BELS defines behavior for.

Action	Resource	Verb	Owning Domain Confirmed?	Risk Tier	Idempotent?	Audit Required?	SourceRef
{{domain}}.{{resource}}.{{verb}}			YES		YES/NO	YES/NO	Action_Vocabulary §__

> Rule: If an action is missing from Action_Vocabulary, STOP and add it there first.

3) Actor & Context Model (what AccessControl evaluates)
Define the canonical context needed for decisions.

3.1 Actor Fields
- actor.userId
- actor.roles[]
- actor.accountStatus
- actor.enforcementStatus
- actor.trustSignals (optional)
- actor.orgIds / membershipIds (if applicable)

3.2 Request Context Fields
- ctx.correlationId
- ctx.deviceId (optional)
- ctx.sessionId
- ctx.clientInfo (appVersion/platform)
- ctx.ipHash (optional)

3.3 Resource Fields (per resource type)

List the minimum fields needed for policy checks:
- resource.ownerId
- resource.visibility (public/private/members)
- resource.publishStatus (if applicable)
- resource.state (lifecycle)
- resource.communityId/orgId (if applicable)

4) Policy Evaluation Order (must match REBS)
1) Authentication/session validity
2) Account status gating
3) Enforcement gating (mute/suspend/ban)
4) Resource visibility/existence gating
5) Role + conditions allow
6) Default deny

5) Policy Rules Table (the core)
Every rule must be deterministic, testable, and include SourceRef.

5.1 Rule Format Requirements
Each rule row MUST include:
- RuleID
- Action
- Applies To (resource type)
- Allow Conditions (boolean logic using predicates)
- Deny ReasonCode
- User Message Guidance
- Next Step (optional)
- Audit? (yes/no)
- SourceRef

5.2 Canonical Predicates (use these words)
- isAuthenticated
- hasRole(role)
- hasAnyRole([roles])
- isOwner(actor, resource)
- isMemberOf(actor, resource.communityId/orgId)
- isVisibleTo(actor, resource)
- isPublic(resource)
- isRestricted(actor, type)
- withinRateLimit(actor, action)
- featureEnabled(flag)
- stateIs(resource, state)
- publishIs(resource, status)

5.3 Policy Rules

RuleID	Action	Resource	Allow Conditions	Deny ReasonCode	User Message Guidance	Next Step	Audit?	SourceRef
PR-001							YES/NO	RPBS §__ / DDES §__
PR-002							YES/NO	

> Rule: If multiple rules apply, the evaluation order in section 4 decides precedence.
> Restrictions override roles unless explicitly stated.

6) Command Handlers (write behaviors)

Define what happens when an allowed action is executed.

6.1 Handler Requirements (mandatory)

Every handler must:
- validate input (schema)
- call AccessControl.can(...)
- enforce state transition (if lifecycle)
- write only domain-owned entities
- emit required events (DIM)
- write audit records when required
- return standardized envelope

6.2 Handler Table

HandlerID	Action	Inputs	Validation Rules	Writes (entities/fields)	State Transition (if any)	Emits Events	Idempotency Strategy	Audit Category	SourceRef
H-001									DDES §__
H-002									

7) State Machines (lifecycle enforcement)

For every lifecycle entity, define states + transitions + guard rules.

7.1 State Machine Table

Entity	States	Transition	From → To	Guard Conditions (predicates)	Deny ReasonCode	Emits Events	SourceRef

7.2 Illegal Transition Standard

- If transition not listed: deny with `INVALID_TRANSITION` (or domain-specific)
- Must include correlationId
- Must not partially write

8) Rate Limits & Abuse Controls (domain-specific)
Define constraints for abuse-prone actions.

ControlID	Action	Limit Type	Threshold	Window	Deny ReasonCode	Notes	SourceRef
RL-001		per-user			RATE_LIMITED		RPBS §__

9) Visibility & “Not Found” Rules (security-safe)
Define when to return “not found or not visible.”

Scenario	Required Behavior	ReasonCode	Notes
user requests private resource they can’t access	respond as not found	RESOURCE_NOT_FOUND_OR_NOT_VISIBLE	prevents leaks

10) Error & Recovery Semantics (domain layer)
Error Case	When	Response	Retryable?	Notes
validation fails	invalid input	VALIDATION_ERROR	NO	include field hints
conflict	state changed	CONFLICT	YES/NO	depends
rate limit	threshold exceeded	RATE_LIMITED	YES	include cooldown

11) Events & Side Effects (must match DIM)
List emitted events and their triggers.

Event	Trigger Action/Transition	Payload Notes	Consumers	Failure Handling	SourceRef
					DIM §__

12) Acceptance Scenarios (must be testable)
These scenarios drive TESTPLAN directly.

Scenario Format (required)
- ScenarioID
- Preconditions
- Action
- Expected Result (allow/deny + reason code)
- Expected State Change (if any)
- Expected Events
- Expected Audit (if any)
- SourceRef

Scenario Table

ScenarioID	Preconditions	Action	Expected Result	State Change	Events	Audit	SourceRef
SC-001			allow				RPBS §__
SC-002			deny (REASON_CODE)	none	none	no	

> Minimum: 10 scenarios per domain; high-risk domains should have 25+.

13) Open Questions (BELS-level)
- Q-001:
- Q-002:

14) Change Control
- Any change to:
  - policy rules
  - handler behavior
  - state machines
  - reason codes

requires:
- TESTPLAN update
- possibly Action Vocabulary update
- ERC bump if locked
- Changelog entry

DIM_<Domain>.md

DIM — Domain Interaction Map — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- DIM defines how this domain interacts with other domains and the outside world.
- Domain Map boundaries override this doc.
- DIM must align with BELS (events/side effects) and Action Vocabulary.

1) Domain Positioning (context)
- Domain purpose:
- System-of-record owned concepts:
- Primary upstream dependencies:
- Primary downstream consumers:
- Cross-domain pattern used most (event / contract / read model):

2) Interaction Principles (hard rules)
- No cross-domain DB writes.
- Cross-domain effects via events or explicit contracts only.
- Orchestrators may coordinate multi-step flows but must not own domain truth.
- Visibility-safe behavior must be preserved (no existence leaks).

3) External Interfaces (outside the app)
List non-domain dependencies.

Interface Type	System	Purpose	Auth Required	Failure Mode	Retry Strategy	Notes
Email/SMS		notify users	YES/NO	queued/dropped	retry/backoff	
Push						
Payments						
Media						

4) Cross-Domain Contract Calls (Synchronous)
List explicit contract calls this domain makes to others (or exposes).

4.1 Outbound Calls (this domain calls others)
Call ID	Caller (this domain)	Callee Domain	Contract Name	Purpose	Inputs	Output	Failure Handling	SourceRef
CC-OUT-001	{{DOMAIN_NAME}}							BELS §__

4.2 Inbound Calls (others call this domain)
Call ID	Caller Domain	Callee (this domain)	Contract Name	Purpose	Inputs	Output	Rate Limit?	SourceRef
CC-IN-001		{{DOMAIN_NAME}}					YES/NO	DDES §__

> Rule: Contract calls must have stable DTOs and cannot expose internal schema.

5) Event Catalog (Canonical)
This is the most important part. Define every event emitted/consumed.

5.1 Event Envelope (must match REBS)
All events must carry:
- eventName
- eventId
- occurredAt
- correlationId
- actorId (optional)
- resourceRef(s)
- payload
- idempotencyKey (optional)

5.2 Emitted Events (this domain is producer)
Event Name	Trigger (Action/Transition)	Payload Fields (summary)	Consumers	Delivery Semantics	Failure Handling	SourceRef
EVT-001				at-least-once	retry + idempotency	BELS §11

5.3 Consumed Events (this domain is subscriber)
Event Name	Producer Domain	When Consumed	Effect (writes)	Idempotency Strategy	Failure Handling	SourceRef
EVT-IN-001						

6) Orchestrations (multi-step workflows)
Define workflows that touch multiple domains.

6.1 Orchestrator Registry
Workflow ID	Workflow Name	Primary Domain	Orchestrator Location	Steps (high level)	Domains Touched	Trigger	Terminal Outcomes	SourceRef
WF-001								Domain Map §9

6.2 Workflow Failure & Compensation
Workflow ID	Failure Point	Expected Behavior	Compensation (if any)	User Experience	Notes
WF-001					

7) Derived Read Models (if used)
If this domain publishes/consumes read models:

Read Model	Owner	Consumers	Update Trigger	Freshness	Notes

8) Notification & Audit Hooks (if relevant)
Tie side effects to canonical events rather than ad-hoc sends.

Side Effect	Trigger	Event Driven?	Owner Domain	Failure Handling	SourceRef
push notification	EVT-___	YES	Notifications	retry/queue	RPBS §__
audit record	admin action	YES	AuditLog/Ops	must persist	REBS §7

9) Abuse & Safety Interactions
List how TrustSafety/AccessControl influences this domain.

Interaction	Source Domain	Behavior	Where Enforced	Reason Codes	Notes
user muted	TrustSafety	block posting	AccessControl + BELS	USER_MUTED	

10) Testing Implications (DIM → TESTPLAN)
List integration tests required because of interactions.

Test ID	What it proves	Producer	Consumer	Expected Result	Notes
IT-001	event causes downstream update	{{DOMAIN_NAME}}	Notifications	push queued	

11) Open Questions (DIM-level)
- Q-001:
- Q-002:

12) Change Control
- Any change to event names/contracts requires:
  - update this DIM
  - update Action Vocabulary if actions changed
  - update TESTPLAN integration tests
  - bump ERC if locked
  - record in Changelog

SCREENMAP_<Domain>.md

SCREENMAP — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- SCREENMAP defines the UI surfaces for this domain and how they connect.
- Must align with UX Foundations and UI Constraints.
- Must reference BELS actions (no invented actions).
- Must not define policy truth (BELS) or domain ownership (Domain Map).

1) Navigation Context
- Primary entry points (from UI Constraints):
- Navigation type (tab/stack/modal/deeplink/notification):
- Route prefix (if applicable):
- Back behavior rules:
- Deep link patterns (if any):

2) Screen Inventory (Canonical)
List every screen this domain owns or heavily contributes to.

Screen ID	Screen Name	Route/Key	Type (List/Detail/Form/etc.)	Primary Purpose	Primary Actions (from Action Vocabulary)	Requires Auth?	Notes
SCR-001						YES/NO	

> Rule: Every Primary Action must exist in Action_Vocabulary.md.

3) Screen Wiring (How screens connect)
Define the allowed navigation edges. This prevents random jumps.

From Screen	To Screen	Trigger (tap/CTA/event)	Preconditions	Params Passed	Notes

4) State & Data Requirements (per screen)
For each screen define required data, caching, and loading behavior.

Screen: {{SCR-001 Screen Name}}
- Required data (queries):
- Optional data:
- Refresh rules:
- Cache rules:
- Pagination rules (if list):
- Offline behavior (if any):
- Loading state UX:
- Empty state UX:

- Error state UX (include correlationId display?):

Repeat for each screen.

5) UI Components Inventory (domain-level)
Define reusable components this domain uses/owns.

Component	Purpose	Props (summary)	Used On Screens	Notes

6) Forms & Validation (critical)
List every form and what it validates. This must align with BELS handler validations.

Form ID	Form Name	Screen	Submit Action	Fields	Validation Rules	Error UX	Notes
F-001			{{domain}}.{{resource}}.{{verb}}				

7) Permission & Restriction UX Mapping (Reason Codes)
List expected deny codes on each screen and how UI responds.

Screen	Action	Possible Deny Reason Codes	UI Response	CTA/Next Step
			show inline gate	“Request access”

> Rule: Denies must be mapped using Reason_Codes.md guidance.

8) Notifications & Entry (if domain can be entered via notif)
Define how notifications land users here.

Notification Type	Landing Screen	Params	Preconditions	Fallback Behavior	Notes

9) Analytics / Telemetry (screen-level)
Define what events to track (UI telemetry only; audits are server-side).

Screen	UI Event	When Fired	Purpose	Notes

10) Error, Recovery, and Support
Define what users see when something fails.

- Global error component usage:
- Retry rules by screen:
- When to show “Contact support”:
- CorrelationId display rules:

11) Performance Constraints (UI)
- Max list item count per page:
- Image/video constraints:
- Debounce rules (search):
- Rendering constraints:

12) Accessibility Checklist (screen-level)
- Screen reader labels:
- Focus order:
- Contrast/touch targets:
- Captions/transcripts (if media):

13) Open Questions (SCREENMAP-level)
- Q-001:
- Q-002:

14) Change Control
- Any change to screens/routes/actions must:
  - update this SCREENMAP
  - update UI Constraints (if navigation rules change)
  - update TESTPLAN (UI + integration tests)
  - bump ERC if locked meaning changes

TESTPLAN_<Domain>.md

TESTPLAN — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- TESTPLAN defines how domain truth is verified.
- BELS scenarios are the authoritative behavior specs.
- If a test conflicts with BELS, either the test is wrong or BELS must be updated (with SourceRef).
- ERC-locked domains require TESTPLAN coverage before Build Mode completes.

1) Test Strategy (domain-specific)
- Domain risk tier (Low/Med/High/Critical):
- Minimum required coverage level (L1/L2/L3):
  - L1: unit-heavy (low risk)
  - L2: unit + integration (default)
  - L3: unit + integration + E2E + abuse testing (high/critical)
- Environments needed (dev/staging/prod-like):

2) Test Inventory (Canonical)
This is the master list. Every test should map back to BELS Scenario IDs.

2.1 Unit Tests (fast)
Test ID	Type	What it proves	BELS Scenario IDs	Component Under Test	Expected Result	Notes
UT-001	unit	policy allow/deny	SC-___	AccessControl.can	deny = REASON_CODE	

2.2 State Machine Tests (required if lifecycle)
Test ID	Type	Entity	Transition	BELS Reference	Expected Result	Notes
SM-001	unit			BELS §7	invalid transition denied	

2.3 Integration Tests (DB + events)
Test ID	Type	What it proves	Producer	Consumer	BELS Scenario IDs	Expected Result	Notes
IT-001	integration	event emitted and consumed	{{DOMAIN_NAME}}	Notifications	SC-___	push queued	

2.4 API Contract Tests (recommended)
Test ID	Type	Endpoint/Contract	What it proves	BELS Scenario IDs	Expected Result	Notes
API-001	contract		returns correct envelope	SC-___	has correlationId	

2.5 E2E Tests (minimum when user-facing)

Test ID	Type	User Flow	Screens	BELS Scenario IDs	Expected Result	Notes
E2E-001	e2e	onboarding + first action	SCR-___	SC-___	completes	

2.6 Abuse / Rate Limit Tests (required for high-risk)

Test ID	Type	Abuse Case	BELS Control IDs	Expected Result	Notes
ABT-001	integration	spam posting	RL-001	RATE_LIMITED	

3) Scenario Coverage Map (BELS → Tests)

This is the traceability table.

BELS Scenario ID	Scenario Summary	Required Test Type	Test IDs Covering It	Pass Criteria
SC-001		unit/integration/e2e	UT-___, IT-___	asserts allow + events

> Rule: Every SC-xxx must map to at least one test ID.

4) Permission Deny Assertions (mandatory)

For any gated action:
- tests must assert exact `reasonCode`
- tests must verify visibility-safe behavior if applicable
- tests must assert correlationId exists

Deny Assertion Table

Action	Deny Reason Codes to Assert	Test IDs	Notes

5) Event Assertions (DIM-driven)

For emitted events:
- assert event name
- assert payload minimum fields
- assert idempotency behavior where required

Event	Minimum Fields	Test IDs	Notes

6) Audit Assertions (sensitive actions)

For audit-required actions:
- assert audit record exists

- assert actorId/action/resourceRef/correlationId present

Action	Audit Category	Test IDs	Notes

7) Performance/Load Checks (optional but recommended)
If applicable:
- list endpoints requiring pagination
- list realtime paths requiring soak tests

Surface	Check Type	Target	Notes
	latency p95		

8) Test Data & Fixtures
- canonical fixtures needed:
- seeding approach:
- cleanup approach:
- deterministic IDs guidance:

9) Release Gate (domain-specific)
Define the minimum tests that must pass to consider the domain “release ready.”

Required to Merge
- [ ] UT: ...
- [ ] SM: ...
- [ ] IT: ...
- [ ] API: ...

Required to Release
- [ ] E2E: ...
- [ ] ABT: ... (if high-risk)

10) Open Questions (TESTPLAN-level)
- Q-001:
- Q-002:

11) Change Control
- Any change to BELS scenarios requires:
  - updating this plan
  - updating/adding tests
  - bumping ERC if locked meaning changes

ERC_<Domain>_v#.md

ERC — Execution Readiness Contract — {{DOMAIN_NAME}} (v{{VERSION}})

0) Lock Statement (Mandatory)
This ERC locks the meaning and requirements for the {{DOMAIN_NAME}} domain for execution.
- From this point forward, implementation must follow the locked inputs below.
- Any meaning change requires a new ERC version (v{{NEXT_VERSION}}) and a Changelog entry.

1) Execution Mode
- [ ] Build Mode (code allowed)
- [ ] Migration Mode (overlay/shadow allowed)

2) Scope of Work (what is being built)
Included
- Actions included in this execution:
  - {{domain}}.{{resource}}.{{verb}}
  - …

Excluded (explicitly not building now)
- Actions excluded:
  - …
- Screens excluded:
  - …
- Integrations excluded:
  - …

> Rule: Anything not listed in Included is not build-authorized in this ERC.

3) Locked Inputs (Authoritative References)
List the exact doc versions/paths used as authority.

Global Inputs
- RPBS: `{{PATH_RPBS}}` (v{{RPBS_VERSION}})
- REBS: `{{PATH_REBS}}` (v{{REBS_VERSION}})
- Domain Map: `{{PATH_DOMAIN_MAP}}` (v{{MAP_VERSION}})
- Glossary: `{{PATH_GLOSSARY}}` (v{{GLOSSARY_VERSION}})
- Action Vocabulary: `{{PATH_ACTIONS}}` (v{{ACTIONS_VERSION}})
- Reason Codes: `{{PATH_REASONS}}` (v{{REASONS_VERSION}})
- ALRP: `{{PATH_ALRP}}` (v{{ALRP_VERSION}})
- TIES: `{{PATH_TIES}}` (v{{TIES_VERSION}})
- DoD: `{{PATH_DOD}}` (v{{DOD_VERSION}})

Domain Inputs

- DDES: `{{PATH_DDES}}` (v{{DDES_VERSION}})
- UX Foundations: `{{PATH_UX}}` (v{{UX_VERSION}})
- UI Constraints: `{{PATH_UI}}` (v{{UI_VERSION}})
- BELS: `{{PATH_BELS}}` (v{{BELS_VERSION}})
- DIM: `{{PATH_DIM}}` (v{{DIM_VERSION}})
- SCREENMAP: `{{PATH_SCREENMAP}}` (v{{SCREENMAP_VERSION}})
- TESTPLAN: `{{PATH_TESTPLAN}}` (v{{TESTPLAN_VERSION}})

4) Frozen Decisions (No Drift Allowed)
These are decisions the agent must not change during this execution.

4.1 Ownership & Boundaries
- Owning domain for core entities: {{LIST}}
- No cross-domain DB writes allowed.
- Integration pattern(s) approved: {{event/contract/read model}}

4.2 Policy Truth
- AccessControl is the single gate.
- Policy evaluation order is frozen (REBS + BELS).
- Reason code mappings are frozen.

4.3 State Machines
- Lifecycle states and transitions are frozen.
- Illegal transitions must be denied with {{INVALID_TRANSITION or domain reason}}.

4.4 Events
- Event names and payload minimums are frozen.
- Delivery semantics are frozen (at-least-once, idempotency keys).

4.5 UI/Navigation (if included)
- Screen list and wiring are frozen (SCREENMAP).
- Permission UX behavior is frozen (UI Constraints + Reason Codes).

5) Implementation Boundaries (files and folders)
Agent is allowed to change ONLY:

Allowed Paths
- `{{DOMAIN_CODE_PATH}}/**`
- `{{APPROVED_INTEGRATION_POINTS}}`

Forbidden Paths
- navigation contracts (if frozen)
- other domains' storage/repositories
- global auth unless explicitly included

6) Acceptance Gates (must be proven)
These are the conditions for marking the execution complete.

6.1 Behavior Gates (BELS)
- [ ] All Included actions implemented
- [ ] All policy allow/deny rules enforced server-side
- [ ] All required reason codes returned
- [ ] All lifecycle transitions enforced
- [ ] All required events emitted
- [ ] Audit records created when required

6.2 Test Gates (TESTPLAN)
- [ ] Scenario coverage map complete (SC-xxx → Test IDs)
- [ ] Allow tests pass
- [ ] Deny tests assert reasonCode
- [ ] State machine tests assert invalid transitions
- [ ] Integration tests assert events + consumers
- [ ] E2E tests pass (if required)

6.3 Observability Gates
- [ ] correlationId present end-to-end
- [ ] logs include action + result + reasonCode for denies
- [ ] audits present for sensitive actions

7) Migration Plan (only if Migration Mode checked)
7.1 Shadow Mode
- [ ] old decision evaluated
- [ ] new decision evaluated
- [ ] diffs logged with correlationId
- [ ] enforcement remains old until parity confirmed

7.2 Cutover
- [ ] parity tests added
- [ ] switch to new enforcement
- [ ] remove duplicate logic after burn-in

8) Release Receipt (required output)
Agent must output:
- Mode (Build/Migration)
- Inputs consulted
- Files changed
- Tests run
- Scenarios covered

- Open questions (if any)
- Known risks (if any)

9) Open Questions (ERC-level)
- Q-001:
- Q-002:

10) Versioning Rules
Create a new ERC version when:
- actions change (add/remove/rename)
- reason codes change
- policy evaluation order changes
- state machine states/transitions change
- event names/payload minimums change
- domain ownership changes

SROL_<Domain>.md

SROL — System Review & Optimization Ledger — {{DOMAIN_NAME}} (v{{VERSION}})

0) Authority
- SROL is a post-build review document.
- It does not change product truth (RPBS) or engineering truth (REBS/TIES).
- It proposes optimizations only after behavior is proven with tests.
- Optimizations must not change meaning unless a new ERC is issued.

1) Build Snapshot
- Domain version built under ERC: `{{PATH_ERC}}` (v{{ERC_VERSION}})
- Date reviewed:
- Reviewer:
- Build scope summary (what was implemented):

2) Reality Check (Does the domain do what it claims?)
- Intended outcomes (from DDES):
- Observed outcomes (from tests/usage):
- Gaps:
- Unexpected behaviors:

3) Domain Purity Audit (Boundaries)
3.1 Ownership Compliance
- Does the domain write only its own data? YES/NO
- Any cross-domain DB writes detected? YES/NO
  - If yes: list files/locations

3.2 Contract Surface Health
- Commands exposed:
- Queries exposed:
- Are other domains calling internal functions? YES/NO
- Contract DTO stability issues:

3.3 Event Discipline
- Emitted events list:
- Consumed events list:
- Any missing idempotency protections? YES/NO

4) Policy & Enforcement Audit
- Are all gated actions enforced server-side? YES/NO
- Any duplicated policy checks outside AccessControl? YES/NO
- Deny reason code coverage:
  - % denies mapped to canonical reason codes:
- Visibility-safe deny compliance (resource existence leaks)? YES/NO

5) State Machine Audit (if applicable)
- Lifecycle entities:
- Invalid transition protections verified? YES/NO
- Transition coverage in tests: % (estimate)
- Any “state mutation without transition” detected? YES/NO

6) Complexity & Change Cost Audit
6.1 Hotspots
List high-churn or risky areas.

Hotspot	Why it’s risky	Symptoms	Proposed fix	Priority
				High/Med/Low

6.2 Duplication Audit
Duplicate Behavior	Locations	Why it happened	Consolidation target	Risk

6.3 Coupling Audit
Coupling	Domains involved	Impact	Decoupling approach	Notes

7) Performance & Reliability Audit
- p95 latency for key actions:
- N+1 or inefficient queries found? YES/NO
- Memory/CPU hotspots:
- Realtime stability issues (if applicable):
- Retry/idempotency correctness:

8) Observability & Ops Audit
- Logs quality (action/result/reason/correlationId):
- Metrics coverage (success/fail/deny by reason):
- Audit log completeness for sensitive actions:
- Alerting gaps:

9) UX Regression Audit (Post-build)
- Any UX laws violated from UX Foundations?
- Any permission UX confusion?
- Any empty/error state issues?
- Accessibility concerns:

10) Optimization Plan (Safe, Non-Meaning-Changing)

List proposals that do NOT change meaning.

Proposal ID	Change	Why	Risk	Requires Tests Update?	Requires ERC bump?
OPT-001			Low/Med/High	YES/NO	NO

> Rule: If meaning changes, ERC bump is required.

11) Deferred Improvements (Meaning-changing or scope expansion)
List improvements that require new ERC or RPBS changes.

Deferred ID	Change	Why	What must change (docs)	New ERC needed?
DEF-001			BELS + TESTPLAN + ERC	YES

12) Open Questions (SROL-level)
- Q-001:
- Q-002:

13) Sign-off
- [ ] Reviewed for boundaries
- [ ] Reviewed for policy
- [ ] Reviewed for state machines
- [ ] Reviewed for tests/coverage
- [ ] Reviewed for ops/observability
- [ ] Optimization plan approved

Upgrades

Upgrades

My revision: what I'd improve on top of what you already have

You already have strong discipline: **integration surfaces**, **repo mapping**, **no duplicate policy engines**, **no cross-domain writes**, **command handler order**, **verify gates**, **reason codes**, **observability fields**, and **lock readiness rules**.

Roshi system start up guide (1)

Roshi system start up guide (1)

Roshi system start up guide (1)

Here are high-leverage upgrades that improve quality without changing your philosophy:

Upgrade A — Make config + schemas "machine-checkable"

Add a `roshi.schema.json` (or zod) for:

- `roshi/domains.json` (domain list, minimums)
  Roshi system start up guide (1)
- `roshi/sources.json` (source inventory)
- `ROSHI_REPORT.json` (stable output format)

Why: ensures tools don't drift and makes CI gating reliable.

Upgrade B — Add 3 missing "pro" commands

1. **`roshi:diff`**
   Diff ERC versions + show what changed in locked inputs (and block if locked meaning changed without new version).
2. **`roshi:lint`**
   Structural lint: required headings/tables exist (same gate rules as verify, but faster + editor-friendly). Your verify already fails on missing structure; lint is "instant feedback."
   Roshi system start up guide (1)
3. **`roshi:ci`**
   A single CI entrypoint that runs:

- `roshi:verify` for changed domains
- enforces "no lock without verify" rule
- prints a concise summary

Upgrade C — First-class “Concept Ownership Ledger”

Your doc already proposes this as **UPGRADE 1** (super valuable). Make it mandatory in the registry layer and make verify fail if a cross-domain concept exists but is not registered.

Roshi system start up guide (1)

Upgrade D — Output artifacts for engineering tools

Generate “exports” from locked ERC:

- OpenAPI (or at least endpoint stubs) from Command Handlers table
- Test case skeleton list from Scenario IDs (you already require mapping)
  Roshi system start up guide (1)
- A “reason-code catalog” JSON used by clients for consistent UX messaging (you already enforce reason codes + deny UX)
  Roshi system start up guide (1)

Upgrade E — Strengthen the repo mapping contract into an enforcement gate

You already require repo-mapping completeness and “stop if blank.” Make `roshi:verify` fail if integration surfaces are marked FOUND but the required exported APIs are missing. (Example: AccessControl must expose `can(actor, action, resource, ctx)`.)

Roshi system start up guide (1)